

1. Introdução

Esta documentação tem como objetivo contextualizar a evolução da API **Quero Trabalhar Backend**, desde seu estado inicial até a consolidação do MVP. O foco não é apenas listar endpoints ou tecnologias utilizadas, mas registrar o raciocínio por trás das principais decisões técnicas, arquiteturais e de negócio tomadas durante o desenvolvimento.

A API nasceu como uma base funcional, mas ainda exploratória, voltada a conectar candidatos, recrutadores e oportunidades de emprego. No estado inicial, já existiam alguns conceitos importantes do domínio, como usuários, recrutadores, oportunidades, tipos de emprego, experiências profissionais, interesses e indicações. Porém, a estrutura ainda apresentava limitações relevantes: entidades JPA eram expostas diretamente como contrato HTTP, empresa e localidade eram tratadas como textos simples, não havia paginação ou filtros consistentes, o tratamento de erros não era padronizado e alguns fluxos sensíveis dependiam de IDs enviados pelo cliente, criando risco de acesso indevido a recursos de outros usuários.

Ao longo da refatoração, a API foi reorganizada para se aproximar de um padrão mais profissional de backend com Spring Boot. Foram introduzidos DTOs para proteger os contratos HTTP, services passaram a concentrar regras de negócio, repositories ficaram responsáveis pela persistência, specifications passaram a centralizar filtros dinâmicos, e o tratamento de erros foi padronizado. Também foram consolidados autenticação e autorização com JWT, endpoints baseados no usuário autenticado, documentação Swagger/OpenAPI, JavaDoc técnico nos módulos mais críticos, execução com Docker e uma base mais organizada de testes.

Um dos pontos centrais da evolução foi o módulo de **Localidade**. No início, informações como cidade e estado eram tratadas como texto livre, o que dificultava validação, busca, padronização e exposição segura dos recursos. No estado final, a API passou a trabalhar com localidade validada como fonte oficial e com **LocalidadePendente** como mecanismo técnico temporário para casos em que a localização informada ainda não pode ser resolvida com segurança. Essa decisão

permitiu manter o cadastro flexível sem expor publicamente empresas ou oportunidades com localização incerta.

Esta documentação também considera a integração com o frontend. Por isso, além de explicar a arquitetura e as regras internas da API, ela também registra como os contratos devem ser consumidos: diferença entre endpoints públicos, endpoints `/me` e endpoints administrativos; uso do token JWT; leitura de respostas paginadas; interpretação dos erros; e tratamento de recursos com localidade pendente.

Portanto, este documento deve ser lido como uma documentação de continuidade do projeto. Ele explica o que existe hoje, por que certas escolhas foram feitas, quais regras não devem ser quebradas sem cuidado e quais pontos ficaram conscientemente planejados para uma evolução pós-MVP.

Em resumo, a API foi transformada de uma base inicial de CRUDs e entidades expostas diretamente em um backend mais estruturado, com separação de responsabilidades, contratos documentados, regras de negócio mais claras, maior segurança nos fluxos autenticados e uma estratégia mais robusta para lidar com localidade, empresas, oportunidades e usuários.

2. Para quem esta documentação foi escrita

Esta documentação foi escrita para orientar a continuidade da API **Quero Trabalhar Backend**. Ela não foi pensada apenas como um registro técnico do que existe no código, mas como um material de transição para quem vai manter, evoluir ou consumir a API daqui em diante.

O projeto envolve regras de negócio importantes, decisões arquiteturais específicas e alguns pontos sensíveis que não ficam totalmente claros apenas lendo os endpoints ou os nomes das classes. Por isso, esta documentação busca atender tanto quem vai trabalhar no backend quanto quem vai integrar o frontend.

2.1 Desenvolvedor backend

O primeiro público desta documentação é o próximo desenvolvedor backend que assumir a manutenção da API.

Para essa pessoa, o documento deve explicar como a API está organizada hoje, quais decisões foram tomadas durante a refatoração e quais pontos precisam ser preservados para evitar regressões. A API deixou de ser apenas um conjunto de CRUDs e passou a ter regras mais claras envolvendo autenticação, ownership, localidade, visibilidade pública, vínculos entre recrutador e empresa, moderação de tipos de emprego, paginação, filtros e tratamento de erros.

O desenvolvedor backend deve usar esta documentação para entender:

- como a API estava no início;
- quais problemas técnicos motivaram a refatoração;
- como a arquitetura final está organizada;
- por que os controllers devem permanecer finos;
- por que as regras de negócio ficam nos services;
- por que DTOs são usados nos contratos HTTP;
- como funcionam os endpoints públicos, `/me` e `admin`;
- como `Localidade` e `LocalidadePendente` foram modeladas;
- quais regras de negócio não devem ser quebradas sem cuidado;
- quais pontos ficaram para evolução pós-MVP.

Um exemplo importante é o módulo de Localidade. Sem contexto, um novo desenvolvedor poderia achar estranho existir `LocalidadePendente` em vez de simplesmente bloquear o cadastro quando a localidade não for encontrada. Esta documentação explica que essa foi uma decisão consciente: a API permite salvar o recurso internamente, mas impede que ele apareça publicamente enquanto a localização não estiver validada.

Outro exemplo é o uso de endpoints `/me`. Eles existem para reduzir risco de IDOR e evitar que o cliente informe IDs sensíveis de usuário em operações que deveriam pertencer ao próprio usuário autenticado.

2.2 Pessoa responsável pelo frontend

O segundo público desta documentação é a pessoa responsável pelo frontend.

Para o frontend, esta documentação deve funcionar como um guia de integração. Ela não substitui o Swagger/OpenAPI, mas explica comportamentos que muitas vezes não ficam evidentes apenas pelo contrato formal de request e response.

A pessoa do frontend precisa entender principalmente:

- como fazer login e armazenar o JWT;
- quais endpoints são públicos;
- quais endpoints exigem autenticação;
- quais endpoints exigem perfil administrativo;
- quando usar endpoints `/me`;
- como consumir endpoints paginados;
- como enviar filtros;
- como interpretar erros padronizados;
- como lidar com recursos que possuem localidade pendente;
- qual a diferença entre DTOs públicos e DTOs internos;
- quais fluxos ainda não existem no MVP.

Essa distinção é importante porque a API tem regras diferentes dependendo do contexto do endpoint. Por exemplo, uma oportunidade com localidade pendente não aparece em `GET /api/oportunidades`, porque esse endpoint é público. Porém, essa mesma oportunidade pode aparecer em `GET /api/recrutadores/me/oportunidades`, porque o recrutador autenticado precisa visualizar os próprios recursos, inclusive os que ainda não foram publicados por pendência de localidade.

Para o frontend, isso significa que a ausência de uma vaga na listagem pública não necessariamente indica erro. Pode significar que ela ainda está aguardando validação de localidade. Essa informação deve ser tratada em telas internas do usuário, não em telas públicas.

2.3 Avaliadores, colaboradores e pessoas que precisam entender o projeto

Além de backend e frontend, esta documentação também pode ser útil para avaliadores, orientadores, colaboradores futuros ou qualquer pessoa que precise entender o projeto de forma mais ampla.

Para esse público, o documento mostra a evolução do sistema como um processo de amadurecimento técnico. Ele explica que o projeto não foi apenas “incrementado”, mas reorganizado em torno de contratos, regras de negócio, segurança, documentação, testes e execução.

Essa visão é importante porque muitas decisões do estado final só fazem sentido quando comparadas com o estado inicial. Por exemplo:

- Empresa deixou de ser apenas texto e passou a ser entidade.
- Localidade deixou de ser texto livre e passou a ter modelo próprio.
- Oportunidade deixou de ser um recurso sem dono claro e passou a nascer no contexto do recrutador autenticado.
- Endpoints com IDs sensíveis foram substituídos ou complementados por endpoints `/me`.
- Listagens críticas passaram a ser paginadas e filtráveis.
- Erros passaram a ter estrutura padronizada.

Assim, a documentação também serve como registro da evolução do MVP e das decisões que tornaram a API mais próxima de um backend real de produção.

2.4 Como cada público deve usar esta documentação

O backend deve usar esta documentação como referência de manutenção e evolução. Antes de alterar Localidade, Empresa, Oportunidade, autenticação, paginação ou DTOs, é importante ler as seções correspondentes para entender as decisões existentes.

O frontend deve usar esta documentação como guia de integração. A documentação ajuda a entender quais endpoints usar para cada tela, como interpretar respostas e

como lidar com estados específicos, como localidade pendente, erro de regra de negócio ou resposta paginada.

O Swagger/OpenAPI deve continuar sendo a referência principal para detalhes exatos de contrato. Esta documentação complementa o Swagger com contexto, regras e raciocínio de produto.

Em outras palavras:

- Swagger responde “qual é o contrato?”;
- README responde “como executar?”;
- docs/testing.md responde “como os testes estão organizados?”;
- esta documentação responde “por que a API foi construída assim e como continuar sem quebrar decisões importantes?”.

2.5 O que o leitor deve ter em mente

Quem ler esta documentação deve ter em mente que a API foi finalizada como um **MVP evoluído**, não como um produto fechado em todos os detalhes.

Isso significa que algumas decisões foram tomadas para entregar valor com segurança e coerência, mas sem antecipar todas as funcionalidades futuras. Exemplos disso são:

- confirmação manual de localidade sugerida ficou para depois;
- notificação de recurso oculto por localidade pendente ficou para depois;
- ownership formal de Empresa ficou para depois;
- contrato JSON de paginação pode ser estabilizado futuramente;
- alguns endpoints pessoais ou de catálogo continuam retornando [List](#).

Esses pontos não são esquecimentos. São decisões de escopo. A documentação deixa claro o que foi implementado no MVP e o que deve ser tratado como evolução futura.

A principal expectativa é que a próxima pessoa que continuar o projeto entenda o estado atual antes de propor mudanças. Isso evita reverter decisões importantes ou tratar como erro algo que foi uma escolha consciente de arquitetura e produto.

3. Estado inicial da API

Antes da refatoração, a API **Quero Trabalhar Backend** já possuía uma base funcional. Ela permitia trabalhar com usuários, recrutadores, oportunidades de emprego, tipos de emprego, experiências profissionais, interesses e indicações. Ou seja, o projeto já carregava a ideia central do produto: conectar pessoas em busca de trabalho com oportunidades cadastradas por recrutadores.

Apesar disso, a API ainda estava em um estágio exploratório. A estrutura era suficiente para validar conceitos iniciais, mas não estava madura para sustentar um MVP mais seguro, documentado e preparado para evolução. O diagnóstico inicial identificou 42 arquivos Java em `src/main`, apenas 3 arquivos em `src/test`, ausência de Docker, ausência de tratamento global de erros, ausência de paginação/filtros avançados e ausência de uma documentação Swagger/OpenAPI real além de menções no README.

3.1 Visão geral do estado inicial

O estado inicial pode ser resumido como uma API CRUD funcional, mas ainda pouco protegida do ponto de vista arquitetural e de contrato.

A API já tinha separação básica entre controllers, services e repositories. Também já utilizava Spring Boot, Spring Data JPA, Spring Security, JWT, H2 e criptografia de senha com BCrypt. Esses pontos eram aproveitáveis e serviram como base para a evolução posterior.

O problema é que essa separação ainda era superficial. Muitos controllers recebiam e retornavam entidades JPA diretamente, os services frequentemente apenas repassavam chamadas para repositories, e as regras de negócio ainda não estavam bem centralizadas. Além disso, quase não existiam DTOs, não havia camada clara de exceptions, não havia specifications, não havia documentação real da API e os testes eram mínimos.

Na prática, isso significava que a API já funcionava, mas ainda misturava conceitos de persistência, contrato HTTP e regra de negócio.

3.2 Stack e configuração inicial

O projeto inicial já usava uma stack moderna, mas havia divergências e inconsistências na configuração.

O Maven estava configurado com Java 21, mas o README mencionava Java 17. A aplicação usava Spring Boot 3.4.3, Spring Web, Spring Data JPA, Spring Validation, Spring Security, JWT com `java-jwt`, Lombok, H2 e dependência runtime para MySQL. O perfil ativo apontava para H2 em memória, com H2 console exposto, e havia segredo JWT hardcoded em `application.properties`.

Essa situação mostrava que a API já tinha alguns componentes importantes, mas ainda precisava de ajustes para ficar coerente e segura. A divergência entre README e configuração real também indicava que a documentação não acompanhava o código.

3.3 Arquitetura inicial

A arquitetura inicial seguia uma divisão básica:

- controllers;
- services;
- repositories;
- entidades;
- alguns DTOs;
- security;
- enums.

Essa base foi útil, porque indicava uma tentativa inicial de separação por camadas. Porém, a implementação ainda era muito próxima de CRUD direto.

Os principais problemas arquiteturais eram:

- controllers expondo entidades JPA diretamente;
- services com pouca regra de negócio;
- ausência de DTOs para a maior parte dos fluxos;
- ausência de tratamento global de erros;

- ausência de specifications para filtros;
- ausência de estrutura específica para documentação, configuração avançada ou regras de domínio;
- pouca cobertura de testes.

O diagnóstico inicial apontou que vários controllers recebiam e retornavam entidades JPA, misturando persistência, contrato de API e serialização. Também foi observado que muitos services apenas executavam `findAll`, `findById`, `save` e `deleteById`, sem concentrar regras importantes de negócio.

3.4 Domínio inicial

O domínio inicial já carregava conceitos importantes do produto, mas ainda de forma incompleta.

As principais entidades eram:

- `Usuario`;
- `UsuarioRecrutador`;
- `OportunidadeDeEmprego`;
- `TipoDeEmprego`;
- `ExperienciaProfissional`;
- `Indicacoes`;
- `InteresseEmOportunidades`;
- `InteresseEmEmprego`.

O `Usuario` representava o candidato/autenticável e implementava `UserDetails`. O `UsuarioRecrutador`, por outro lado, era uma entidade separada, com nome, email, telefone e empresa, mas não fazia parte diretamente da autenticação. A oportunidade de emprego carregava descrição, tipo de emprego, modalidade, cidade, estado e empresa como texto.

Alguns conceitos importantes ainda não existiam como entidades próprias:

- `Empresa`;

- `PerfilCandidato`;
- `PerfilRecrutador`;
- `Localidade`;
- `LocalidadePendente`;
- vínculo formal entre recrutador e empresa;
- modelo consistente de candidato interessado em vaga;
- modelo de localização normalizada.

Isso mostra que o domínio já tinha uma direção, mas ainda não estava suficientemente modelado para sustentar regras de negócio mais complexas.

3.5 Segurança e autorização no estado inicial

A API inicial já tinha Spring Security e JWT. Isso foi um ponto positivo, porque havia uma base de autenticação sobre a qual evoluir.

O fluxo inicial era baseado em login por email e senha, com retorno de token. O usuário implementava `UserDetails`, e já existiam roles como `ADMIN` e `USER`. A senha já era criptografada com BCrypt.

Porém, o problema maior não era a autenticação, mas a autorização.

O diagnóstico apontou risco alto de IDOR. Muitos endpoints operavam com IDs vindos do path sem verificar se o usuário autenticado era realmente dono do recurso. Por exemplo, um usuário autenticado poderia informar o ID de outro usuário em determinados fluxos, como interesse em oportunidade. Também havia risco de cadastro indevido de administrador, porque a role podia ser aceita no payload de cadastro.

Em outras palavras: a API sabia quem estava autenticado, mas ainda não usava essa identidade de forma consistente para proteger recursos.

Esse foi um dos motivos para a evolução posterior em direção a endpoints `/me`, nos quais a identidade passa a ser derivada do token JWT e não de um ID informado pelo cliente.

3.6 Localidade no estado inicial

O módulo de localidade praticamente não existia.

As informações geográficas eram tratadas como texto livre. Em oportunidade, havia campos como cidade e estado em formato `String`. Em interesses/preferências, havia listas de cidades, estados e países também como `String`. Não havia entidade `País`, `Estado`, `Cidade`, `Localidade` ou `LocalidadePendente`. Também não havia integração com Google Maps, validação de localidade ou qualquer fluxo de resolução de localização incerta.

Isso criava vários problemas:

- dificuldade de filtrar por cidade ou estado;
- risco de dados duplicados por diferença de acento, caixa ou abreviação;
- possibilidade de publicar vaga com localização inválida;
- ausência de normalização geográfica;
- impossibilidade de tratar localidade ambígua;
- ausência de mecanismo para pendência ou validação posterior.

Esse foi um dos maiores problemas do estado inicial. A API precisava aceitar informações de localidade, mas não tinha mecanismo confiável para validar, padronizar ou proteger a exposição pública de recursos com localização incerta.

3.7 Empresa e recrutador no estado inicial

No estado inicial, `Empresa` não existia como entidade.

O recrutador possuía um campo `empresa` em formato texto, e a oportunidade também possuía empresa como `String`. Não havia relacionamento real entre recrutador e empresa, não havia aprovação de vínculo, não havia dono de empresa e não havia administração de empresa.

Essa modelagem limitava muito o sistema.

A API não conseguia responder de forma confiável:

- qual recrutador pertence a qual empresa;
- quem pode publicar vaga em nome de uma empresa;
- quais empresas existem no sistema;
- quais oportunidades pertencem a uma empresa real;
- quem pode aprovar ou recusar vínculos;
- quais dados públicos uma empresa possui.

Na prática, empresa era apenas um texto repetido em lugares diferentes. Isso podia funcionar para uma validação inicial, mas não era suficiente para um MVP com listagem pública de empresas, vínculo de recrutadores e publicação de oportunidades em nome de empresa.

3.8 Oportunidade de emprego no estado inicial

A oportunidade de emprego existia, mas ainda sem regras fortes de domínio.

Ela possuía descrição, tipo de emprego, modalidade, cidade, estado e empresa como texto. A criação recebia a entidade JPA diretamente no corpo da requisição. Não havia vínculo com recrutador autenticado, nem vínculo com entidade empresa, nem autorização de dono para editar ou excluir. Também não havia paginação, filtros ou distinção clara entre listagem pública e listagem interna.

O diagnóstico também identificou inconsistência de tipo no ID da oportunidade: a entidade usava `Integer`, enquanto o repository usava `Long`.

O principal problema era que a oportunidade não tinha um dono de negócio bem definido. Como consequência, não havia como garantir que apenas o recrutador responsável pudesse alterar ou remover uma vaga.

Essa limitação foi uma das razões para, no estado final, a oportunidade passar a nascer no contexto do recrutador autenticado.

3.9 Candidato, experiências, preferências e indicações no estado inicial

No estado inicial, o candidato era representado diretamente por `Usuario`. Ainda não existia `PerfilCandidato` separado.

O usuário podia ter experiências profissionais e interesses em oportunidades. Porém, os fluxos ainda eram frágeis. O interesse em vaga, por exemplo, era feito com endpoint contendo `usuarioId` no path, o que permitia risco de manipulação de outro usuário se não houvesse validação de ownership. Também havia problema de inconsistência entre o nome do path variable declarado na rota e o nome usado no método de remoção de interesse.

As experiências profissionais tinham CRUD parcial, referenciavam usuário e tipo de emprego, mas não tinham DTO próprio nem validação consistente de ownership. As preferências existiam como entidade, mas não havia controller correspondente. As indicações eram um CRUD simples e ainda não tinham modelagem clara de autor, destinatário e ownership.

Esses pontos mostravam que os conceitos já existiam, mas ainda não tinham sido organizados em fluxos seguros e orientados ao usuário autenticado.

3.10 Paginação e filtros no estado inicial

Não havia paginação nem filtros avançados.

O diagnóstico inicial não encontrou uso de `Pageable`, `Page`, `Specification` ou filtros estruturados. As listagens principais retornavam `List`: usuários, recrutadores, oportunidades, tipos de emprego, experiências, indicações e interesses.

Isso trazia riscos práticos:

- respostas grandes demais;
- piora de performance com crescimento da base;
- dificuldade para o frontend montar telas de listagem;
- ausência de busca e refinamento;
- necessidade de filtros client-side, o que não escala.

Esse foi o motivo para, no estado final, as listagens principais passarem a usar `Page`, `Pageable`, `FilterDTO` e `Specification`.

3.11 Tratamento de erros no estado inicial

O tratamento de erros ainda não era padronizado.

Não havia `@RestControllerAdvice`, `@ExceptionHandler`, `StandardError`, `ValidationError` ou mapeamento consistente de status HTTP. Os erros apareciam de formas variadas: `ResponseEntity.notFound()`, `ResponseEntity.badRequest().build()`, `RuntimeException`, `IllegalArgumentException`, `EntityNotFoundException` e até `System.out.println` em situações de negócio.

Isso dificultava muito a integração com o frontend.

Sem um padrão de erro, o frontend não consegue prever com segurança onde ler a mensagem, como identificar erro de validação, como diferenciar regra de negócio de erro técnico, ou qual comportamento adotar para cada status.

Essa lacuna motivou a criação posterior de um tratamento global com modelos como `StandardError`, `ValidationError` e `FieldMessage`.

3.12 Documentação no estado inicial

A documentação inicial era limitada e desatualizada.

Havia um README, mas ele mencionava Java 17 enquanto o Maven usava Java 21. Também dizia que Spring Security seria uma implementação futura, embora a API já tivesse segurança. O README citava Swagger, mas não havia dependência real de SpringDoc nem configuração OpenAPI. Também havia problemas de encoding no texto.

Não havia:

- Swagger/OpenAPI real;
- JavaDoc técnico relevante;
- documentação de endpoints;
- documentação de testes;

- documentação de Docker;
- documentação de variáveis de ambiente.

Isso tornava difícil para outra pessoa entender, executar ou integrar a API.

3.13 Docker e execução no estado inicial

Não havia Docker.

O diagnóstico inicial não encontrou `Dockerfile`, `docker-compose.yml` nem `.dockerignore`. A execução dependia do ambiente local com Maven e da configuração ativa com H2. Também havia dependência MySQL no `pom.xml`, mas o perfil ativo usava H2 em memória.

Essa situação dificultava a reprodução do ambiente por outra pessoa, especialmente para avaliação, demonstração ou continuidade do projeto.

3.14 Testes no estado inicial

A suíte de testes era mínima.

Foram encontrados apenas testes muito básicos, como `contextLoads` e um teste de `UsuarioService`. O diagnóstico também apontou indício de teste quebrado, porque um objeto `user` usado em `Optional.of(user)` não estava inicializado em trecho comentado.

Não havia testes relevantes para:

- controllers;
- repositories;
- segurança/JWT;
- localidade;
- oportunidades;
- recrutador/empresa;
- interesses;
- indicações;

- experiências.

Isso significava que mudanças importantes no domínio poderiam quebrar comportamento sem serem detectadas automaticamente.

3.15 Principais riscos do estado inicial

O estado inicial tinha riscos técnicos e de negócio importantes.

Os principais eram:

- exposição de entidades JPA como contrato da API;
- ausência de DTOs na maior parte dos fluxos;
- risco alto de IDOR;
- ausência de ownership;
- possibilidade de role indevida no cadastro;
- segredo JWT hardcoded;
- empresa como texto;
- localidade como texto livre;
- oportunidade sem dono/recrutador;
- ausência de paginação/filtros;
- ausência de tratamento global de erros;
- documentação desatualizada;
- ausência de Docker;
- testes insuficientes;
- inconsistências de encoding;
- services com pouca regra de negócio.

Esses riscos não significam que a API inicial não tinha valor. Pelo contrário: ela já provava a ideia do produto. Mas indicavam que, para chegar a um MVP mais sólido, seria necessário reorganizar a base técnica.

3.16 O que já era aproveitável

Apesar dos problemas, o estado inicial tinha pontos positivos importantes.

A API já usava Spring Boot, JPA, validação, BCrypt, JWT, roles, enum de modalidade, tipo de emprego com unicidade de título, DTOs para usuário evitando retorno de senha e H2 para facilitar desenvolvimento inicial. Além disso, o domínio já apontava os conceitos principais do produto.

Esses elementos foram fundamentais para a evolução posterior.

A refatoração não começou do zero. Ela aproveitou a base existente, preservou os conceitos úteis e reorganizou aquilo que precisava de mais segurança, clareza e escalabilidade.

3.17 Síntese do estado inicial

O estado inicial da API pode ser entendido como uma primeira versão funcional, boa para validar ideias, mas ainda inadequada para sustentar um MVP com segurança, documentação e manutenção de longo prazo.

A API já tinha o embrião do produto: usuários, recrutadores, oportunidades, tipos de emprego, experiências, interesses e indicações. Porém, ainda faltava amadurecer a arquitetura, proteger recursos por ownership, normalizar localidade, transformar empresa em entidade, padronizar erros, criar DTOs, adicionar paginação, ampliar testes e documentar os contratos.

A refatoração posterior nasceu exatamente dessas necessidades. O objetivo não foi apenas adicionar funcionalidades, mas transformar uma base inicial de CRUDs em uma API mais coerente, segura, documentada e preparada para integração com frontend e continuidade por outros desenvolvedores.

4. Problemas encontrados

Durante o diagnóstico do estado inicial da API, ficou claro que o projeto já possuía uma base funcional, mas ainda carregava problemas importantes de arquitetura, segurança, modelagem de domínio, documentação e preparação para evolução.

Esses problemas não devem ser entendidos apenas como “falhas” do projeto original. Eles representam o estado natural de uma primeira versão exploratória: a

API já validava parte da ideia do produto, mas ainda precisava ser reorganizada para se tornar um MVP mais seguro, compreensível e sustentável.

A refatoração foi orientada justamente por esses problemas. Entender essa seção é importante porque muitas decisões do estado final só fazem sentido quando vistas como resposta direta às limitações iniciais.

4.1 Exposição direta de entidades JPA

Um dos principais problemas encontrados foi o uso de entidades JPA como contrato direto da API.

No estado inicial, vários controllers recebiam e retornavam entidades diretamente. Isso misturava três responsabilidades que deveriam estar separadas:

- modelo de persistência;
- contrato HTTP;
- serialização da resposta para o cliente.

Essa abordagem funciona em protótipos simples, mas cria problemas à medida que o sistema cresce. Quando uma entidade é exposta diretamente, qualquer alteração interna no modelo JPA pode quebrar o contrato da API. Além disso, campos sensíveis, relações internas ou detalhes de persistência podem acabar aparecendo no JSON sem intenção.

No estado final, essa decisão foi corrigida com a adoção de DTOs e Java Records para contratos de request, response e filtros. A regra consolidada foi: **entidade JPA representa persistência; DTO representa contrato HTTP**.

Essa mudança foi essencial para proteger a API contra acoplamento indevido entre banco de dados e frontend.

4.2 Services com pouca regra de negócio

Outro problema era que muitos services atuavam apenas como repasse direto para os repositories. Em vários fluxos, a camada de service chamava métodos como

`findAll`, `findById`, `save` e `deleteById`, sem concentrar validações relevantes de domínio.

Isso tornava a aplicação frágil, porque parte da regra acabava espalhada entre controller, entidade, repository ou simplesmente não existia.

O objetivo da refatoração foi reposicionar os services como a camada principal de regra de negócio. No estado final, os services passaram a cuidar de decisões como:

- validar usuário autenticado;
- verificar ownership;
- impedir duplicidades;
- aplicar regra de publicação de oportunidade;
- validar vínculo entre recrutador e empresa;
- acionar resolução de localidade;
- decidir se um recurso pode ou não ser exibido publicamente;
- lançar exceptions específicas para o handler global.

Essa mudança fortaleceu a arquitetura porque deixou mais claro onde novas regras devem ser implementadas.

4.3 Ausência de DTOs na maior parte dos fluxos

A API inicial possuía alguns DTOs para autenticação e usuário, mas a maioria dos módulos ainda dependia diretamente das entidades. Isso afetava especialmente oportunidades, experiências, indicações, recrutadores e interesses.

A ausência de DTOs gerava três problemas principais.

Primeiro, o contrato HTTP ficava instável. Segundo, o frontend ficava dependente da estrutura interna do banco. Terceiro, era difícil separar respostas públicas de respostas internas.

Essa diferença passou a ser muito importante no estado final. Por exemplo, uma oportunidade pública não deve expor detalhes de `LocalidadePendente`, enquanto uma oportunidade retornada no painel do recrutador pode precisar indicar que a localidade ainda está pendente.

Por isso, a refatoração consolidou a separação entre DTOs públicos e DTOs internos. Essa decisão é especialmente importante para `Empresa` e `OportunidadeDeEmprego`.

4.4 Risco de IDOR e ausência de ownership

Um dos problemas mais críticos era o risco de IDOR, ou seja, acesso indevido a recursos de outro usuário por manipulação de IDs.

No estado inicial, alguns endpoints recebiam IDs de usuário diretamente no path para executar ações sensíveis. Isso permitia cenários em que um usuário autenticado poderia tentar operar sobre recursos de outro usuário apenas alterando o ID na URL. O diagnóstico inicial apontou esse risco especialmente em fluxos como interesse em oportunidade e manipulação de dados vinculados ao usuário.

A solução arquitetural foi priorizar endpoints `/me`.

Em vez de o cliente informar qual usuário deseja manipular, a API passa a extrair a identidade do token JWT. Assim, ações pessoais são sempre feitas no contexto do usuário autenticado.

A regra consolidada foi:

Quando a ação pertence ao próprio usuário, o backend deve derivar o usuário a partir do token, não confiar em `usuarioId` enviado pelo cliente.

Essa decisão impactou vários fluxos, como:

- dados do usuário autenticado;
- perfil de candidato;
- perfil de recrutador;
- experiências profissionais;
- interesses em vagas;
- oportunidades do recrutador;
- vínculo do recrutador com empresa.

4.5 Autenticação existente, mas autorização insuficiente

A API inicial já possuía Spring Security e JWT. Isso era um ponto positivo. O problema era que a autenticação não era acompanhada por uma autorização de domínio suficientemente forte.

A API sabia quem estava logado, mas ainda não usava essa informação de forma consistente para proteger recursos específicos. Também havia risco no cadastro público, pois a role poderia ser aceita no payload dependendo do contrato usado. O diagnóstico inicial também apontou segredo JWT hardcoded e H2 console exposto como pontos sensíveis.

No estado final, a segurança foi reorganizada com maior clareza:

- endpoints públicos foram definidos explicitamente;
- endpoints `/me` passaram a depender do usuário autenticado;
- endpoints admin foram agrupados em `/api/admin/**`;
- `ROLE_ADMIN` passou a proteger fluxos administrativos;
- respostas 401 e 403 foram padronizadas;
- services passaram a validar ownership quando necessário.

Essa evolução foi importante para transformar a API de uma autenticação genérica para uma autorização mais orientada ao domínio.

4.6 Empresa modelada apenas como texto

No estado inicial, `Empresa` não existia como entidade própria. A empresa aparecia como uma `String` dentro de recrutador e oportunidade.

Essa modelagem era insuficiente para o produto.

Com empresa como texto, a API não conseguia responder corretamente:

- quais empresas existem;
- quais recrutadores estão vinculados a cada empresa;
- quem pode publicar em nome de uma empresa;
- quais oportunidades pertencem a uma empresa real;

- se o vínculo entre recrutador e empresa foi aprovado;
- quais dados públicos uma empresa possui;
- qual localidade está associada à empresa.

Por isso, uma das decisões importantes da evolução foi transformar **Empresa** em entidade real do domínio.

No estado final, empresa passou a ter dados próprios, localidade, endpoints públicos, vínculo com recrutador e fluxo administrativo de aprovação/recusa de vínculo. Ainda assim, o ownership formal da empresa foi deixado para pós-MVP, porque exigiria uma decisão maior sobre dono, administradores, conta empresarial ou login por CNPJ.

4.7 Recrutador separado do usuário autenticável

Inicialmente, havia uma separação frágil entre **Usuario** e **UsuarioRecrutador**. O recrutador existia como entidade separada, mas não era integrado de forma robusta à identidade autenticada.

Isso criava dificuldade para responder perguntas fundamentais:

- qual usuário está atuando como recrutador;
- quais oportunidades pertencem ao recrutador autenticado;
- quem pode alterar ou remover uma oportunidade;
- como um mesmo usuário poderia atuar como candidato e recrutador.

A evolução do domínio resolveu isso com a introdução de perfis.

No estado final, **Usuario** é a base autenticável, e os papéis de candidato e recrutador são representados por perfis associados. Isso permite que um mesmo usuário tenha perfil de candidato, perfil de recrutador ou ambos.

Essa decisão simplifica a autenticação e melhora a modelagem do produto.

4.8 Oportunidade sem dono claro

A oportunidade de emprego existia no estado inicial, mas não tinha dono de negócio bem definido. Ela possuía empresa como texto, cidade e estado como texto, e não estava vinculada de forma consistente a um recrutador autenticado.

Isso gerava riscos importantes:

- qualquer usuário autenticado poderia criar ou remover oportunidades sem vínculo claro;
- não havia como saber quem era responsável pela vaga;
- não havia como validar publicação em nome de empresa;
- não havia regra de visibilidade pública baseada em localidade;
- não havia filtros ou paginação;
- não havia distinção clara entre painel do recrutador e listagem pública.

No estado final, a oportunidade passou a nascer obrigatoriamente no contexto do recrutador autenticado. O cliente não informa `recrutadorId`; o backend deriva essa informação do token.

Essa foi uma das decisões mais importantes para segurança e coerência do domínio.

4.9 Localidade como texto livre

O problema de localidade foi um dos mais críticos da API inicial.

Cidade, estado e país eram tratados como texto livre em diferentes partes do sistema. Isso tornava os dados inconsistentes e difíceis de filtrar. O diagnóstico inicial apontou ausência de entidade `Localidade`, ausência de `LocalidadePendente`, ausência de integração com Google Maps, ausência de validação e ausência de fluxo para localidades incertas.

Esse modelo criava vários riscos:

- cidade escrita com variações diferentes;
- estado com abreviação ou nome completo sem padrão;

- erros de digitação;
- acentos inconsistentes;
- impossibilidade de filtro confiável;
- publicação de recurso com localização inválida;
- dificuldade de relacionar empresa, oportunidade e preferências de candidato.

A solução final foi criar um módulo de localidade mais robusto.

A decisão central foi:

Localidade validada é a fonte oficial. LocalidadePendente é uma fila técnica temporária para casos que ainda não puderam ser resolvidos com segurança.

Essa decisão permitiu manter o cadastro flexível sem comprometer a qualidade das listagens públicas.

4.10 Ausência de LocalidadePendente

No estado inicial, quando a localidade era inválida, ambígua ou simplesmente textual, não havia mecanismo formal para tratar isso.

A API poderia aceitar dados inconsistentes ou bloquear fluxos de forma pouco planejada. Nenhuma das duas opções era ideal.

A introdução de **LocalidadePendente** resolveu esse problema com uma estratégia intermediária:

- o recurso pode ser salvo internamente;
- a localidade incerta é registrada como pendência técnica;
- o recurso não aparece publicamente;
- o sistema pode tentar reprocessar depois;
- o dono pode visualizar o status em endpoints internos.

Essa decisão foi essencial para o MVP, porque evitou depender totalmente de resolução imediata de localidade e ao mesmo tempo impediu exposição pública de dados geográficos incertos.

4.11 Ausência de paginação e filtros

No estado inicial, as listagens retornavam `List` e não havia `Pageable`, `Page`, `Specification` ou filtros estruturados.

Isso comprometia tanto backend quanto frontend.

Para o backend, listagens sem paginação podem gerar consumo excessivo de memória e respostas grandes demais. Para o frontend, a ausência de filtros obriga processamento client-side, dificulta busca, ordenação e navegação.

A refatoração introduziu o padrão:

- `FilterDTO`;
- `Pageable`;
- `@PageableDefault`;
- `@ParameterObject`;
- `Page<ResponseDTO>`;
- `JpaSpecificationExecutor`;
- `Specification`.

Esse padrão foi aplicado nas listagens principais, especialmente oportunidades, empresas, oportunidades do recrutador, interesses do candidato e endpoints administrativos.

4.12 Tratamento de erros inconsistente

Outro problema importante era a ausência de um tratamento global de erros.

O diagnóstico inicial apontou que não havia `@RestControllerAdvice`, `StandardError`, `ValidationError` ou mapeamento consistente de status HTTP. Os erros apareciam como `RuntimeException`, `IllegalArgumentException`, `EntityNotFoundException`, `ResponseEntity.notFound()`, `badRequest().build()` e até `System.out.println`.

Isso dificultava a integração com o frontend.

Sem um padrão, a pessoa do frontend não sabe se a mensagem virá em `message`, em corpo vazio, em outro campo ou se virá como erro padrão do Spring. Também fica difícil diferenciar erro de validação, regra de negócio, conflito, não encontrado ou falta de permissão.

No estado final, a API passou a usar tratamento padronizado com:

- `StandardError`;
- `ValidationError`;
- `FieldMessage`;
- exceptions específicas;
- status HTTP mais consistentes.

Essa mudança melhorou previsibilidade, manutenção e integração.

4.13 Documentação desatualizada ou inexistente

A documentação inicial não acompanhava o estado real do projeto.

O README mencionava Java 17, embora o Maven usasse Java 21. Também dizia que Spring Security seria uma implementação futura, apesar de a API já possuir segurança. Havia menção a Swagger, mas sem configuração real de SpringDoc/OpenAPI.

Não havia documentação real de endpoints, DTOs, erros, testes, Docker ou decisões arquiteturais.

Isso criava uma barreira para qualquer pessoa nova no projeto. Sem documentação, a única forma de entender a API era ler o código diretamente, o que é difícil quando há decisões de domínio importantes, como localidade pendente, visibilidade pública, ownership e endpoints `/me`.

Por isso, a documentação passou a ser uma frente importante do MVP.

4.14 Testes insuficientes

A suíte de testes inicial era muito pequena.

O diagnóstico encontrou apenas poucos testes, incluindo `contextLoads` e um teste simples de `UsuarioService`, com indício de problema no próprio teste por uso de objeto não inicializado. Não havia cobertura relevante para controllers, security, localidade, oportunidades, empresa, interesses, indicações ou paginação.

Isso aumentava o risco de refatoração.

Sem testes, mudanças em regra de negócio poderiam quebrar comportamento sem serem detectadas. Por isso, ao longo da evolução, foram criados ou ajustados testes de services, controllers, security, domínio, repository, OpenAPI, localidade e paginação/filtros.

4.15 Ausência de Docker e dificuldade de reprodução do ambiente

No estado inicial, não havia `Dockerfile`, `docker-compose.yml` nem `.dockerignore`.

Isso dificultava a execução por outras pessoas. Cada desenvolvedor precisava configurar o ambiente manualmente, instalar dependências e entender perfis por conta própria.

Para um MVP que pode ser avaliado ou continuado por outra pessoa, isso é um problema prático. A dockerização foi adicionada depois para facilitar a execução local e tornar o ambiente mais reprodutível.

4.16 Problemas de encoding e textos sem acentuação

Outro problema recorrente era a presença de textos com encoding quebrado ou sem acentuação adequada.

O diagnóstico inicial apontou exemplos de mojibake em mensagens e documentação, como textos com caracteres corrompidos em português.

Além de ser um problema estético, isso afeta a qualidade percebida da API e a clareza das mensagens para usuário, frontend e documentação.

Durante a evolução, foi feita uma revisão textual para corrigir acentuação, mensagens de erro, validações e documentação exposta.

4.17 Síntese dos problemas encontrados

Os problemas encontrados podem ser resumidos em sete grupos principais.

O primeiro grupo era arquitetural: entidades expostas, poucos DTOs, services com pouca regra e ausência de separação clara entre contrato, domínio e persistência.

O segundo grupo era de segurança: risco de IDOR, ausência de ownership, uso excessivo de IDs vindos do cliente e autorização ainda pouco orientada ao domínio.

O terceiro grupo era de modelagem: empresa como texto, localidade como texto livre, recrutador separado do usuário autenticável e oportunidade sem dono claro.

O quarto grupo era de escalabilidade: listagens sem paginação, sem filtros e sem specifications.

O quinto grupo era de integração: erros inconsistentes, ausência de documentação formal e contratos pouco previsíveis para o frontend.

O sexto grupo era de operação: ausência de Docker, configuração pouco clara e README desatualizado.

O sétimo grupo era de qualidade: testes mínimos, problemas de encoding e falta de documentação técnica.

A refatoração foi conduzida como resposta direta a esses problemas. O objetivo não foi apenas adicionar funcionalidades, mas transformar a API em uma base mais segura, documentada, testável e compreensível para continuidade.

5. Objetivos da refatoração

A refatoração da API **Quero Trabalhar Backend** não teve como objetivo apenas “melhorar o código” ou adicionar endpoints. O objetivo principal foi transformar uma base inicial funcional, mas ainda exploratória, em uma API mais segura, compreensível, documentada e preparada para ser usada como MVP.

O estado inicial já provava a ideia central do produto: conectar candidatos, recrutadores e oportunidades de emprego. Porém, a forma como a API estava organizada dificultava manutenção, integração com frontend e evolução de regras de negócio. Havia entidades expostas diretamente, localidade em texto livre, empresa sem entidade própria, risco de IDOR, ausência de paginação, tratamento de erro inconsistente, documentação incompleta e testes mínimos.

Por isso, a refatoração foi guiada por alguns objetivos principais.

5.1 Transformar a API em uma base mais profissional de MVP

O primeiro objetivo foi elevar a API de uma base experimental para uma base mais próxima de um backend real de MVP.

Isso envolveu organizar melhor as responsabilidades, padronizar contratos, reduzir riscos de segurança, melhorar previsibilidade das respostas e preparar o projeto para que outras pessoas pudessem executar, testar, consumir e evoluir a API.

O foco não foi reescrever tudo do zero. A ideia foi aproveitar o que já existia de útil no projeto inicial e evoluir gradualmente os pontos mais frágeis.

A refatoração buscou preservar a intenção original do produto, mas corrigindo a estrutura ao redor dela.

5.2 Separar contrato HTTP de modelo de persistência

Um objetivo central foi parar de expor entidades JPA diretamente nos endpoints.

No estado inicial, muitos controllers recebiam e retornavam entidades. Isso fazia o contrato HTTP depender diretamente da estrutura interna do banco. Essa

dependência é perigosa porque qualquer mudança na entidade pode quebrar o frontend, expor campos indevidos ou gerar problemas de serialização.

A refatoração introduziu DTOs e Java Records para representar requests, responses e filtros.

A decisão foi:

Entidade JPA representa persistência. DTO representa contrato HTTP.

Essa separação tornou possível criar respostas diferentes para contextos diferentes. Por exemplo, uma empresa pública pode retornar apenas dados seguros para qualquer usuário, enquanto uma empresa visualizada internamente pode trazer status de localidade pendente para apoiar o dono ou fluxo administrativo.

Essa decisão também facilita a integração com o frontend, porque o contrato fica mais explícito e menos dependente do banco.

5.3 Reposicionar os services como camada de regra de negócio

Outro objetivo foi fortalecer a camada de services.

No início, muitos services apenas repassavam chamadas ao repository. A refatoração buscou concentrar as decisões de negócio nessa camada.

No estado final, os services passaram a ser responsáveis por regras como:

- buscar o usuário autenticado;
- validar se o recurso pertence ao usuário;
- impedir duplicidades;
- validar vínculos entre recrutador e empresa;
- validar tipos de emprego aprovados;
- decidir quando uma empresa ou vaga pode ser pública;
- acionar o fluxo de resolução de localidade;
- controlar criação, atualização e remoção de recursos;
- lançar exceptions específicas.

A regra arquitetural consolidada foi:

Controller deve ser fino. Service deve decidir regra de negócio.

Essa decisão é importante para manutenção futura. Se uma nova regra for adicionada ao domínio, ela normalmente deve entrar no service, não no controller.

5.4 Reduzir risco de IDOR com endpoints /me

Um dos objetivos mais importantes foi reduzir o risco de IDOR.

No estado inicial, alguns fluxos usavam IDs de usuário no path para ações sensíveis. Isso abre espaço para um usuário autenticado tentar manipular recursos de outro usuário apenas trocando o ID na URL.

A refatoração passou a priorizar endpoints `/me`.

A ideia é simples:

Quando uma operação pertence ao próprio usuário, o backend deve descobrir quem é o usuário pelo token JWT, e não por um ID enviado pelo cliente.

Isso impactou fluxos como:

- consultar dados do usuário autenticado;
- atualizar o próprio usuário;
- alterar senha;
- gerenciar perfil de candidato;
- gerenciar perfil de recrutador;
- listar oportunidades do recrutador autenticado;
- listar vagas de interesse do candidato autenticado;
- gerenciar experiências profissionais do candidato;
- listar indicações dadas e recebidas.

Essa decisão também facilita o trabalho do frontend. Em vez de precisar guardar e enviar `usuarioId` para ações pessoais, o frontend usa endpoints `/me` e envia apenas o token.

5.5 Consolidar autenticação e autorização

A API inicial já possuía JWT, mas a autorização ainda não estava bem ligada às regras de domínio.

A refatoração teve como objetivo consolidar uma divisão clara entre:

- endpoints públicos;
- endpoints autenticados do próprio usuário;
- endpoints administrativos.

A estrutura final ficou baseada em:

- login via `POST /login`;
- token JWT no header `Authorization`;
- endpoints públicos explicitamente permitidos;
- endpoints `/me` baseados no usuário autenticado;
- endpoints `/api/admin/**` protegidos por perfil administrativo;
- respostas 401 e 403 padronizadas.

Essa separação é importante para backend e frontend. O backend mantém regras de acesso mais claras, e o frontend entende melhor quando deve mostrar telas públicas, telas autenticadas ou telas administrativas.

5.6 Transformar Empresa em entidade real do domínio

Um objetivo importante da refatoração foi substituir empresa como texto por uma entidade real.

No estado inicial, empresa aparecia como `String` em recrutador e oportunidade. Isso impedia o sistema de controlar vínculos, dados públicos, localização e oportunidades por empresa.

A refatoração criou um módulo de Empresa mais consistente.

A partir disso, passou a ser possível representar:

- dados públicos da empresa;

- localidade da empresa;
- listagem pública de empresas;
- oportunidades vinculadas a uma empresa;
- recrutadores vinculados;
- solicitação de vínculo;
- aprovação ou recusa de vínculo por admin.

Essa mudança foi essencial para transformar empresa em um conceito real do produto, não apenas em um texto solto.

Ao mesmo tempo, foi tomada uma decisão de escopo: **ownership formal de Empresa ficou para pós-MVP**. O sistema passou a ter vínculo moderado entre recrutador e empresa, mas ainda não define de forma completa dono, administradores ou login empresarial.

5.7 Reestruturar OportunidadeDeEmprego em torno do recrutador autenticado

Outro objetivo central foi dar dono claro para oportunidade.

No estado inicial, a oportunidade não estava vinculada de forma segura ao recrutador autenticado. Isso dificultava regras de edição, exclusão, publicação e associação com empresa.

No estado final, oportunidade passou a nascer no contexto do recrutador autenticado.

A regra consolidada foi:

O cliente não informa `recrutadorId` para criar oportunidade. O backend identifica o recrutador a partir do token.

Essa decisão resolveu um problema de segurança e também tornou o domínio mais coerente. A vaga passa a ter um responsável claro.

Também foi adicionada a possibilidade de publicação em nome de empresa, mas com regra:

`publicarComoEmpresa` só funciona se o recrutador tiver vínculo aprovado com a empresa.

Essa regra protege a credibilidade da plataforma e evita que qualquer recrutador publique vaga usando indevidamente o nome de uma empresa.

5.8 Resolver o problema crítico de Localidade

Localidade foi provavelmente o objetivo mais importante da refatoração.

No estado inicial, localidade era tratada como texto livre. Isso gerava inconsistência, dificuldade de filtro e risco de exposição pública de empresas ou oportunidades com localização errada.

O objetivo foi criar uma estratégia que equilibrasse duas necessidades:

1. Não bloquear completamente o cadastro quando a localidade não pudesse ser resolvida imediatamente.
2. Não exibir publicamente recurso com localização incerta.

A solução foi criar o modelo baseado em:

- `Pais`;
- `Estado`;
- `Cidade`;
- `Localidade`;
- `LocalidadePendente`.

A decisão central foi:

Localidade validada é a fonte oficial. LocalidadePendente é uma fila técnica temporária.

Com isso, quando a localidade é validada, o recurso pode aparecer publicamente. Quando a localidade fica pendente, o recurso é salvo internamente, mas não aparece nas listagens públicas.

Essa decisão protege a qualidade das buscas públicas sem sacrificar totalmente a experiência de cadastro.

5.9 Encapsular Google Maps como detalhe de resolução

A refatoração também buscou evitar que a integração com Google Maps se espalhasse pela aplicação.

O objetivo não era transformar Google Maps em fonte pública direta, nem fazer controllers dependerem dele. O papel do Google Maps ficou restrito ao fluxo técnico de resolução de localidade.

A decisão foi:

Google Maps é fallback encapsulado no fluxo de resolução, não parte do contrato público da API.

Isso significa que:

- **LocalidadeService** atua como catálogo oficial;
- endpoints públicos consultam base validada;
- Google Maps é usado apenas quando necessário para tentar resolver texto livre;
- resultados ambíguos não são escolhidos automaticamente;
- falhas externas não devem quebrar a arquitetura pública da API.

Essa separação reduz acoplamento e facilita manutenção futura.

5.10 Padronizar visibilidade pública dos recursos

Outro objetivo foi definir claramente o que pode ou não aparecer publicamente.

A regra final ficou:

Endpoints públicos só exibem empresas e oportunidades com Localidade validada.

Essa decisão impacta diretamente:

- `GET /api/empresas;`
- `GET /api/empresas/{id};`
- `GET /api/empresas/{id}/oportunidades;`
- `GET /api/oportunidades;`
- `GET /api/oportunidades/{id}.`

Por outro lado, endpoints internos `/me` podem exibir recursos pendentes ao dono, porque o usuário precisa saber o estado do próprio cadastro.

Essa distinção é fundamental para backend e frontend.

Para o backend, ela deve ser preservada em specifications, services e DTOs públicos.

Para o frontend, ela significa que uma vaga criada pode não aparecer imediatamente na listagem pública se a localidade ainda estiver pendente.

5.11 Introduzir paginação e filtros nas listagens principais

A API inicial retornava listas completas, sem filtros estruturados.

O objetivo da refatoração foi preparar as listagens principais para uso real pelo frontend e para crescimento de dados.

Foi adotado o padrão:

- `FilterDTO;`
- `Pageable;`
- `@PageableDefault;`
- `@ParameterObject;`
- `Page<ResponseDTO>;`
- `JpaSpecificationExecutor;`
- `Specification.`

Esse padrão foi aplicado nos endpoints mais importantes:

- oportunidades públicas;
- empresas públicas;
- oportunidades da empresa;
- oportunidades do recrutador autenticado;
- vagas de interesse do candidato autenticado;
- usuários admin;
- experiências admin;
- tipos de emprego não aprovados.

A decisão de escopo foi manter alguns endpoints como `List`, principalmente catálogos/autocomplete e listas pessoais pequenas.

5.12 Padronizar tratamento de erros

Outro objetivo foi tornar os erros previsíveis para frontend e manutenção.

No estado inicial, os erros eram heterogêneos. No estado final, a API passou a usar um tratamento global com modelos padronizados.

O objetivo foi permitir que o frontend saiba onde encontrar mensagens e como reagir a cada status.

A estrutura final passou a incluir:

- `StandardError`;
- `ValidationError`;
- `FieldMessage`;
- exceptions específicas;
- `@RestControllerAdvice`.

Os status foram organizados para diferenciar:

- erro de validação;
- não autenticado;
- sem permissão;
- recurso não encontrado;

- conflito;
- regra de negócio.

Essa padronização melhora a experiência de integração e facilita debug.

5.13 Documentar a API para backend e frontend

Um objetivo forte da fase final foi documentar a API.

A documentação foi dividida em vários níveis:

- Swagger/OpenAPI para contratos;
- JavaDoc técnico para decisões internas;
- README para execução e visão geral;
- docs/testing.md para testes;
- coleção Postman para testes manuais;
- esta documentação histórica para contexto e continuidade.

O objetivo foi evitar que o próximo desenvolvedor precise reconstruir todo o raciocínio apenas lendo o código.

A documentação também foi pensada para o frontend, explicando:

- login;
- JWT;
- endpoints públicos;
- endpoints `/me`;
- endpoints admin;
- paginação;
- filtros;
- erros;
- localidade pendente;
- diferença entre DTO público e interno.

5.14 Melhorar execução e ambiente com Docker

Outro objetivo foi facilitar a execução do projeto.

No estado inicial, não havia Docker. Isso dificultava a reprodução do ambiente por outra pessoa.

A refatoração adicionou:

- `Dockerfile`;
- `docker-compose.yml`;
- `.dockerignore`.

Com isso, o projeto fica mais fácil de executar em ambiente local, demonstrar, testar e futuramente preparar para implantação.

Docker não resolve todas as necessidades de produção, mas melhora muito a experiência de quem recebe o projeto.

5.15 Fortalecer a suíte de testes

A refatoração também teve como objetivo aumentar a confiança no comportamento da API.

A suíte inicial era mínima. Ao longo da evolução, foram adicionados ou ajustados testes para:

- services;
- controllers;
- security;
- OpenAPI;
- domain;
- repository;
- localidade;
- reprocessamento de pendências;
- paginação e filtros;
- regras de negócio;
- endpoints admin;
- endpoints `/me`.

O objetivo não foi atingir cobertura perfeita, mas criar uma base de segurança para os fluxos mais importantes.

Também foi planejada uma documentação específica em [docs/testing.md](#) para explicar a organização da suíte.

5.16 Registrar decisões de MVP e separar o que ficou para depois

Um objetivo importante foi não misturar o que está pronto com o que é futuro.

Durante a refatoração, algumas funcionalidades foram identificadas como importantes, mas não essenciais para o MVP. Em vez de implementá-las de forma apressada, elas foram documentadas como pós-MVP.

Entre elas:

- confirmação manual de localidade sugerida;
- notificação quando recurso ficar oculto por localidade pendente;
- ownership formal de Empresa;
- controle operacional avançado da fila de LocalidadePendente;
- estabilização do contrato JSON de paginação;
- revisão da exposição de dados técnicos de localidade em DTOs internos;
- revogação de token via blacklist;
- possível conta empresarial ou login por CNPJ.

Essa separação é importante para evitar que o próximo desenvolvedor interprete ausência de algumas funcionalidades como esquecimento. Em vários casos, foi uma decisão consciente de escopo.

5.17 Síntese dos objetivos

A refatoração teve como objetivo transformar a API em um MVP mais coerente, seguro e documentado.

De forma resumida, os principais objetivos foram:

- criar separação mais clara entre contrato HTTP, domínio e persistência;

- centralizar regras de negócio nos services;
- reduzir risco de IDOR usando endpoints `/me`;
- consolidar autenticação e autorização com JWT;
- transformar empresa em entidade real;
- vincular oportunidade ao recrutador autenticado;
- criar uma estratégia robusta para localidade;
- impedir exposição pública de recursos com localidade pendente;
- adicionar paginação e filtros nas listagens principais;
- padronizar tratamento de erros;
- documentar a API para backend e frontend;
- facilitar execução com Docker;
- ampliar testes;
- registrar decisões e trade-offs do MVP.

Esses objetivos orientaram a evolução do projeto e explicam por que a API final possui a estrutura atual.

6. Evolução arquitetural

A evolução arquitetural da API **Quero Trabalhar Backend** foi uma das partes mais importantes da refatoração. O projeto começou com uma estrutura básica de camadas, mas ainda muito próxima de um CRUD simples. Ao longo do desenvolvimento, essa estrutura foi amadurecida para separar melhor responsabilidades, proteger contratos HTTP, centralizar regras de negócio, melhorar segurança e tornar a API mais compreensível para manutenção futura.

No estado inicial, já existiam controllers, services e repositories. Isso foi positivo, porque havia uma base organizada o suficiente para evoluir sem precisar reescrever tudo do zero. Porém, essa separação ainda era superficial: muitos controllers recebiam e retornavam entidades JPA diretamente, os services muitas vezes apenas repassavam chamadas para os repositories, não havia tratamento global de erros, não havia specifications, os DTOs eram limitados e a documentação era insuficiente.

No estado final, a API passou a seguir uma arquitetura mais clara em camadas, com controllers finos, services concentrando regras de negócio, repositories cuidando da

persistência, DTOs protegendo os contratos HTTP, specifications centralizando filtros dinâmicos e exceptions padronizando respostas de erro.

6.1 Da API CRUD para uma API em camadas

A primeira grande mudança arquitetural foi sair de uma estrutura baseada em CRUD direto para uma estrutura mais orientada a casos de uso e regras de negócio.

No início, a API tinha endpoints que basicamente recebiam uma entidade, salvavam no banco e devolviam a entidade. Essa abordagem é comum em protótipos, mas cria problemas quando o sistema começa a ter regras reais.

O principal risco era que a API não separava claramente:

- o que é representação interna do banco;
- o que é contrato exposto para o frontend;
- o que é regra de negócio;
- o que é apenas operação de persistência.

Com a refatoração, essa separação ficou mais clara.

A arquitetura final passou a seguir a lógica:

- **Controller** recebe a requisição HTTP, valida entrada e delega.
- **DTO** representa o contrato de entrada ou saída.
- **Service** executa regra de negócio.
- **Repository** acessa o banco.
- **Specification** monta filtros dinâmicos.
- **Exception Handler** padroniza erros.
- **Security** fornece identidade e autorização.

Essa mudança foi importante porque tornou o projeto mais previsível. Uma pessoa nova consegue entender melhor onde cada tipo de lógica deve ficar.

6.2 Controllers finos

No estado final, os controllers foram organizados para serem uma camada de borda HTTP.

A responsabilidade deles é:

- receber parâmetros;
- receber DTOs de request;
- aplicar validação de entrada;
- configurar paginação quando necessário;
- chamar o service correto;
- devolver `ResponseEntity`;
- documentar o endpoint com Swagger/OpenAPI.

O controller não deve decidir regra de negócio relevante.

Essa decisão é importante porque evita que regras fiquem espalhadas entre vários endpoints. Por exemplo, a regra de que uma oportunidade deve ser criada no contexto do recrutador autenticado não deve ficar no controller. O controller apenas chama o service; o service obtém o recrutador autenticado e aplica a regra.

Isso deixa o backend mais fácil de manter e testar.

Quando um novo endpoint for criado, a orientação deve ser:

- criar um contrato claro com DTO;
- manter o controller simples;
- colocar validações de domínio no service;
- não acessar repository diretamente pelo controller;
- não expor entidade JPA como resposta.

6.3 Services como centro da regra de negócio

Uma das decisões mais importantes foi fortalecer a camada de service.

No estado inicial, vários services tinham comportamento muito simples, quase como uma ponte direta para o repository. No estado final, os services passaram a concentrar decisões de domínio.

Essa camada passou a ser responsável por regras como:

- identificar o usuário autenticado;

- buscar perfil de candidato ou recrutador;
- validar se o recurso pertence ao usuário;
- impedir duplicidades;
- validar vínculo entre recrutador e empresa;
- validar se um tipo de emprego está aprovado;
- aplicar regra de publicação como empresa;
- chamar o fluxo de resolução de localidade;
- decidir se um recurso pode ser público;
- lançar exceptions específicas;
- controlar transações.

Essa mudança melhora a clareza da API porque define um lugar natural para novas regras.

Se no futuro for necessário mudar uma regra de negócio, como a forma de publicar oportunidade, aprovar vínculo de empresa ou resolver localidade, a primeira camada a ser analisada deve ser o service correspondente.

6.4 Repositories focados em persistência

Os repositories ficaram responsáveis pelo acesso ao banco.

Eles devem continuar sendo usados para consultas, persistência e operações derivadas do Spring Data JPA. A regra geral é que repository não deve carregar regra de negócio complexa. Quando uma consulta precisa refletir uma regra de negócio mais ampla, essa regra deve ser coordenada pelo service ou expressa de forma clara em uma *Specification*.

No estado final, alguns repositories passaram a ter consultas específicas importantes, especialmente no módulo de LocalidadePendente. Isso acontece porque esse módulo precisa buscar pendências por dono genérico, por recurso, por campo alvo e por recência.

Mesmo nesses casos, a intenção é que o repository continue respondendo à pergunta “como consultar?”, enquanto o service responde “o que fazer com esse resultado?”.

Essa separação é importante para evitar que regras fiquem escondidas em queries.

6.5 Specifications para filtros e visibilidade

A introdução de `Specification` foi uma das melhorias arquiteturais ligadas à paginação e filtros.

No estado inicial, não havia `Pageable`, `Page`, `Specification` ou filtros estruturados. As listagens retornavam `List`, o que dificultava escalabilidade e integração com frontend.

No estado final, as principais listagens passaram a usar:

- `FilterDTO`;
- `Pageable`;
- `Page`;
- `JpaSpecificationExecutor`;
- `Specification`.

As specifications cumprem dois papéis.

O primeiro é montar filtros opcionais, como termo, tipo de emprego, empresa, recrutador, modalidade, país, estado e cidade.

O segundo é aplicar regras de visibilidade. Esse ponto é especialmente importante nos endpoints públicos.

Por exemplo, listagens públicas de empresas e oportunidades devem aplicar a regra de localidade validada. Ou seja, não basta filtrar por nome ou cidade; a consulta também precisa garantir que o recurso pode aparecer publicamente.

Por isso, specifications públicas não são apenas filtros técnicos. Elas também preservam regras de produto.

A regra que deve ser mantida é:

- specification pública deve ocultar recurso com localidade pendente;

- specification interna `/me` pode permitir recurso pendente;
- filtros internos devem respeitar o usuário autenticado;
- filtros públicos não devem expor estados técnicos de pendência.

6.6 DTOs e Java Records como contratos HTTP

A adoção de DTOs foi uma mudança central da arquitetura.

No estado inicial, entidades JPA eram frequentemente usadas como entrada ou saída dos endpoints. Isso deixava o frontend acoplado ao modelo interno da aplicação.

No estado final, os contratos HTTP passaram a ser representados por DTOs, em grande parte usando Java Records.

Essa decisão trouxe algumas vantagens:

- evita expor campos internos das entidades;
- permite respostas diferentes para contextos diferentes;
- facilita validação de entrada;
- melhora documentação Swagger;
- reduz risco de quebra no frontend quando a entidade muda;
- permite separar DTO público de DTO interno.

Essa separação é muito importante em módulos como Empresa e Oportunidade.

Por exemplo, uma empresa pública deve expor apenas dados seguros e visíveis publicamente. Já uma empresa em contexto interno pode expor status de localidade para que o usuário entenda por que o recurso ainda não aparece no público.

A regra consolidada é:

- entidade JPA não é contrato de API;
- request deve usar DTO próprio;
- response deve usar DTO próprio;
- filtros devem usar FilterDTO;
- DTO público não deve vaziar detalhe técnico interno;

- DTO interno pode trazer informações operacionais quando necessário.

6.7 Separação entre DTO público e DTO interno

Uma evolução importante foi diferenciar respostas públicas de respostas internas.

No estado inicial, essa distinção praticamente não existia. Como entidades eram expostas diretamente, o mesmo objeto tendia a servir para todos os contextos.

No estado final, a API passou a diferenciar melhor os contratos.

Os DTOs públicos são usados em endpoints acessíveis sem autenticação ou em listagens públicas. Eles devem expor apenas dados que qualquer usuário pode ver.

Os DTOs internos são usados em endpoints autenticados, especialmente `/me` e `admin`. Eles podem expor informações adicionais úteis para o dono do recurso ou para operação administrativa.

Essa distinção é essencial no caso de `LocalidadePendente`.

Uma oportunidade pública não deve expor motivo de pendência, texto original ou estado técnico de validação. Se ela estiver pendente, nem deve aparecer no endpoint público.

Já no painel do recrutador, a vaga pendente pode aparecer com status de localidade, porque o recrutador precisa entender o que aconteceu com o próprio recurso.

Essa decisão deve ser preservada no frontend e no backend.

6.8 Tratamento global de erros

A criação de um tratamento global de erros também fez parte da evolução arquitetural.

No estado inicial, os erros eram retornados de forma inconsistente: algumas vezes com corpo vazio, outras com exceptions soltas, outras com respostas manuais.

No estado final, a API passou a usar uma estrutura padronizada com:

- `StandardError`;
- `ValidationError`;
- `FieldMessage`;
- exceptions específicas;
- handler global.

Essa mudança melhora a arquitetura porque tira a responsabilidade de formatar erro dos controllers e services.

O service pode lançar uma exception de negócio. O handler decide como essa exception vira uma resposta HTTP.

Essa separação melhora o código e facilita a integração com o frontend. A pessoa do frontend passa a saber onde ler a mensagem de erro e como interpretar erros de validação, autenticação, permissão, conflito ou regra de negócio.

6.9 Segurança integrada à arquitetura

A evolução arquitetural também envolveu integrar melhor segurança e regra de negócio.

No estado inicial, já existia JWT, mas a API ainda dependia muito de IDs vindos do cliente para ações sensíveis. O problema não era apenas autenticar, mas garantir que o usuário autenticado só pudesse operar o que realmente pertence a ele.

No estado final, a arquitetura passou a valorizar endpoints `/me` e services que extraem o usuário a partir do contexto de segurança.

Essa decisão impacta diretamente a arquitetura:

- controllers não recebem `usuarioId` para ações pessoais;
- services usam o usuário autenticado;
- regras de ownership ficam no service;
- endpoints admin ficam separados em `/api/admin`;
- o frontend não precisa manipular IDs sensíveis para ações pessoais.

A regra arquitetural é:

Identidade vem do token. Escopo de recurso é validado no service.

Essa decisão reduz risco de IDOR e deve ser preservada em novas funcionalidades.

6.10 Módulo de Localidade como arquitetura própria

O módulo de Localidade evoluiu para uma arquitetura própria dentro da API.

Isso aconteceu porque o problema era mais complexo do que simplesmente criar campos `cidade` e `estado`.

A solução final separou responsabilidades entre várias classes:

- `LocalidadeService` atua como catálogo oficial;
- `LocalidadeResolucaoService` orquestra o fluxo normal de cadastro;
- `FluxoResolucaoLocalidadeService` executa a tentativa pura de resolução;
- `ResolvedorLocalidadeBaseInterna` tenta resolver pela base interna;
- `ResolvedorLocalidadeGoogleMaps` encapsula a integração externa;
- `ConversorGoogleResultParaLocalidade` converte resultado externo em modelo interno;
- `RegistroLocalidadePendenteService` registra pendências;
- `LocalidadePendenteReprocessamentoService` reprocessa pendências existentes;
- `LocalidadePendenteReprocessamentoScheduler` agenda o reprocessamento.

Essa separação foi necessária para evitar que Localidade virasse um service gigante e acoplado.

A arquitetura do módulo deixa claro que:

- consulta pública de catálogo não chama Google Maps;
- Google Maps é fallback técnico;
- pendência é fila temporária;
- reprocessamento não cria nova pendência;

- ambiguidade não deve ser resolvida automaticamente.

Esse módulo deve ser alterado com cuidado, porque concentra uma das principais decisões do MVP.

6.11 Documentação como parte da arquitetura

Outro ponto importante é que a documentação passou a fazer parte da arquitetura do projeto, não apenas um complemento externo.

O estado final possui Swagger/OpenAPI para contratos, JavaDoc técnico para decisões internas, README para execução, documentação de testes e coleção Postman para exploração manual.

Isso é relevante porque a API possui comportamentos que não são óbvios apenas olhando para o endpoint. Por exemplo:

- por que uma vaga aparece em `/me`, mas não no público;
- por que LocalidadePendente não é erro de cadastro;
- por que Google Maps não aparece como endpoint;
- por que alguns endpoints são `Page` e outros continuam `List`;
- por que `publicarComoEmpresa` depende de vínculo aprovado;
- por que estados futuros de localidade existem mas não estão implementados como fluxo público.

Documentar essas decisões evita que uma pessoa nova interprete escolhas conscientes como bugs.

6.12 Testes como proteção da arquitetura

A evolução arquitetural também foi acompanhada por testes.

No estado inicial, os testes eram mínimos e não cobriam os principais fluxos.

No estado final, a suíte passou a cobrir controllers, services, security, OpenAPI, domínio, repositories, Localidade, LocalidadePendente, paginação, filtros e fluxos administrativos.

Os testes ajudam a proteger decisões arquiteturais, como:

- endpoints públicos não exibem pendências;
- endpoints `/me` usam usuário autenticado;
- filtros funcionam com paginação;
- admin exige permissão;
- reprocessamento de localidade não duplica pendência;
- Swagger documenta contratos esperados.

Mesmo que a cobertura ainda possa evoluir, os testes já passaram a funcionar como uma rede de segurança para os fluxos mais importantes.

6.13 O que não foi feito de propósito

A evolução arquitetural também envolveu decisões de escopo.

Nem tudo foi resolvido no MVP.

Alguns pontos foram conscientemente deixados para depois:

- ownership formal de Empresa;
- confirmação manual de localidade sugerida;
- notificação de recurso oculto por localidade pendente;
- painel operacional completo para LocalidadePendente;
- contrato JSON próprio para paginação;
- revisão completa da exposição de estados internos em DTOs;
- revogação de token por blacklist.

Isso é importante porque o estado final não deve ser interpretado como produto acabado em todos os aspectos. Ele é um MVP organizado, com decisões documentadas e pontos futuros identificados.

6.14 Síntese da evolução arquitetural

A evolução arquitetural da API pode ser resumida como a passagem de uma base CRUD funcional para uma API em camadas, com contratos protegidos, regras de

negócio centralizadas, segurança mais orientada ao usuário autenticado, tratamento de erros padronizado, listagens paginadas e documentação consolidada.

A principal mudança não foi apenas técnica. Foi uma mudança de responsabilidade.

Antes, a API funcionava, mas misturava contrato, entidade, regra e persistência. Depois, cada camada passou a ter um papel mais claro.

A regra que deve orientar a continuidade é:

- controller não deve concentrar regra de negócio;
- service deve proteger o domínio;
- repository deve cuidar de persistência;
- DTO deve proteger o contrato;
- specification deve concentrar filtros;
- handler deve padronizar erros;
- security deve fornecer identidade;
- documentação deve explicar decisões que o código sozinho não revela.

Essa arquitetura não é complexidade por complexidade. Ela foi adotada porque o domínio cresceu e passou a exigir segurança, clareza, visibilidade pública controlada, integração confiável com o frontend e manutenção por outras pessoas.

7. Segurança e autorização

A evolução da segurança foi uma das frentes mais importantes da API **Quero Trabalhar Backend**. No estado inicial, a aplicação já possuía Spring Security e autenticação com JWT, o que era um ponto positivo. Porém, a segurança ainda estava concentrada principalmente em “estar autenticado ou não estar autenticado”. A API ainda não usava de forma consistente a identidade do usuário autenticado para proteger recursos específicos do domínio.

O principal problema não era apenas fazer login. O problema era garantir que, depois de autenticado, o usuário só pudesse acessar, alterar ou remover aquilo que realmente pertencia a ele ou aquilo que seu perfil permitia.

Por isso, a refatoração tratou segurança não apenas como configuração de filtros JWT, mas como parte das regras de negócio da API.

7.1 Segurança no estado inicial

No estado inicial, a API já tinha uma base de autenticação com Spring Security e JWT. O login recebia e-mail e senha, validava o usuário e retornava um token. O usuário implementava `UserDetails`, existiam roles como `ADMIN` e `USER`, e as senhas já eram criptografadas com BCrypt.

Isso mostrava que o projeto já tinha uma fundação importante.

Porém, o diagnóstico inicial apontou riscos relevantes:

- endpoints usavam IDs vindos do cliente para ações sensíveis;
- não havia padrão consolidado de endpoints `/me`;
- a autorização era mais baseada em rota do que em dono do recurso;
- havia risco de IDOR;
- o cadastro podia aceitar role no payload, dependendo do contrato;
- algumas respostas de erro de segurança não eram padronizadas;
- o H2 console estava liberado;
- havia segredo JWT hardcoded;
- endpoints autenticados ainda podiam operar recursos sem validação forte de ownership.

Isso significa que a API sabia autenticar um usuário, mas ainda não protegia de forma suficiente **o que esse usuário podia fazer depois de autenticado**.

7.2 O problema central: autenticação não é autorização

Uma decisão importante da refatoração foi separar mentalmente dois conceitos:

- autenticação;
- autorização.

Autenticação responde à pergunta:

Quem é o usuário?

Autorização responde à pergunta:

Esse usuário pode fazer esta ação sobre este recurso?

No estado inicial, a API já respondia parcialmente à primeira pergunta, mas ainda respondia mal à segunda.

Por exemplo, se um endpoint recebe `usuarioId` no path, o backend precisa garantir que o usuário autenticado é realmente aquele usuário ou que possui permissão administrativa para agir em nome dele. Se essa validação não existe, basta trocar o ID na URL para tentar acessar ou modificar dados de outra pessoa.

Esse tipo de risco é conhecido como IDOR. No contexto da API, ele era especialmente sensível em fluxos de usuário, interesses, experiências, oportunidades e recursos ligados a perfis.

A refatoração buscou reduzir esse risco com uma decisão simples e forte:

Em fluxos pessoais, o backend deve usar a identidade do token JWT, não um ID de usuário enviado pelo cliente.

7.3 Adoção de endpoints `/me`

A principal resposta arquitetural ao risco de IDOR foi a adoção de endpoints `/me`.

Endpoints `/me` representam ações feitas no contexto do usuário autenticado. Isso significa que o cliente não precisa informar qual usuário está sendo manipulado. O backend descobre isso pelo token JWT.

Essa decisão melhora segurança e também simplifica o frontend.

Em vez de o frontend guardar e enviar `usuarioId` em ações pessoais, ele apenas envia o token. A API identifica o usuário e aplica as regras de domínio.

Exemplos de fluxos que passaram a seguir essa lógica:

- consultar dados do usuário autenticado;

- atualizar o próprio usuário;
- alterar a própria senha;
- adicionar ou remover perfil de candidato;
- adicionar ou remover perfil de recrutador;
- consultar perfil do recrutador autenticado;
- listar oportunidades do recrutador autenticado;
- listar vagas de interesse do candidato autenticado;
- gerenciar experiências profissionais do candidato;
- listar indicações dadas e recebidas.

Essa mudança é uma das mais importantes da API final.

A regra para manutenção futura é:

Se a operação pertence ao próprio usuário, prefira endpoint `/me`. Não peça `usuarioId` ao frontend sem necessidade.

7.4 Organização dos tipos de endpoint

No estado final, a API passou a separar melhor os endpoints em três grandes grupos:

- endpoints públicos;
- endpoints autenticados do próprio usuário;
- endpoints administrativos.

Essa separação é essencial tanto para backend quanto para frontend.

7.4.1 Endpoints públicos

Endpoints públicos são aqueles que podem ser acessados sem token.

Eles são usados para consulta de informações que fazem sentido antes do login ou para fluxos abertos do produto.

Exemplos:

- login;

- cadastro de usuário;
- listagem pública de oportunidades;
- listagem pública de empresas;
- consulta pública de empresa;
- consulta pública de oportunidade;
- catálogo de tipos de emprego aprovados;
- catálogo de localidades.

Mesmo sendo públicos, esses endpoints não retornam qualquer coisa. Eles obedecem regras de visibilidade.

A principal regra é:

Endpoints públicos de empresa e oportunidade só retornam recursos com Localidade validada.

Isso significa que uma vaga ou empresa com localidade pendente pode existir internamente, mas não aparecer no público.

Para o frontend, isso é importante: se um recurso não aparece na listagem pública, isso pode estar relacionado à regra de visibilidade, não necessariamente a uma falha de cadastro.

7.4.2 Endpoints **/me**

Endpoints **/me** exigem autenticação e operam sobre o usuário autenticado.

Eles são usados para painéis internos do usuário, candidato ou recrutador.

Exemplos:

- dados do usuário autenticado;
- perfil candidato do usuário autenticado;
- perfil recrutador do usuário autenticado;
- oportunidades criadas pelo recrutador autenticado;

- interesses do candidato autenticado;
- experiências do candidato autenticado.

Esses endpoints podem mostrar informações que não aparecem no público. Por exemplo, uma oportunidade com localidade pendente pode aparecer no painel do recrutador, porque ele precisa saber que a vaga foi salva, mas ainda não está pública.

A regra é:

O público vê apenas recursos prontos para publicação. O dono autenticado pode ver seus próprios recursos em estado interno.

7.4.3 Endpoints admin

Endpoints administrativos ficam agrupados em `/api/admin/**`.

Eles exigem usuário autenticado com perfil administrativo.

Esses endpoints foram criados para operações de gestão, como:

- listar usuários;
- consultar ou atualizar usuários por ID;
- gerenciar perfis de usuários;
- listar experiências globalmente;
- moderar tipos de emprego não aprovados;
- aprovar tipos de emprego;
- aprovar ou recusar vínculo entre recrutador e empresa.

A decisão de concentrar administração em `/api/admin/**` melhora clareza e segurança. Fica mais fácil identificar rotas sensíveis, configurar autorização e orientar o frontend sobre quais telas exigem perfil administrativo.

A regra é:

Endpoint admin não deve ser usado para substituir fluxo `/me`. Admin é para gestão administrativa, não para ações pessoais do usuário comum.

7.5 JWT e fluxo de autenticação

No estado final, o login é feito por `POST /login`.

Quando as credenciais são válidas, a API retorna o token JWT no header `Authorization`, no formato Bearer Token.

O frontend deve capturar esse token e enviá-lo nas próximas requisições protegidas.

A API trabalha de forma stateless. Isso significa que o backend não mantém sessão tradicional no servidor. Cada requisição protegida precisa enviar o token.

Essa decisão é coerente com APIs REST e facilita integração com frontend.

A documentação OpenAPI também passou a registrar esse comportamento. O `/login` precisou ser documentado manualmente porque ele é implementado por filtro de segurança, não por um controller tradicional.

7.6 Respostas 401 e 403

A refatoração também padronizou melhor os erros de segurança.

A distinção entre 401 e 403 é importante:

- 401 significa que o usuário não está autenticado ou o token é inválido/ausente/expirado;
- 403 significa que o usuário está autenticado, mas não tem permissão para executar aquela ação.

Para o frontend, isso muda o comportamento esperado.

Em caso de 401, normalmente o frontend deve redirecionar para login, limpar token inválido ou pedir nova autenticação.

Em caso de 403, o frontend deve mostrar uma mensagem de acesso negado ou ocultar funcionalidades para as quais o usuário não tem perfil.

Essa distinção evita tratar todo erro de segurança como “login inválido”.

7.7 Proteção contra IDOR nos services

A refatoração não depende apenas de anotações de segurança. Muitas proteções acontecem na camada de service.

Isso é importante porque algumas regras não são apenas “tem role ou não tem role”. Elas dependem do domínio.

Exemplos:

- o candidato só pode remover o próprio interesse;
- o candidato só pode manipular as próprias experiências;
- o recrutador só pode editar ou excluir as próprias oportunidades;
- o recrutador só pode publicar como empresa se tiver vínculo aprovado;
- o autor de uma indicação deve ser validado para exclusão;
- endpoints `/me` devem buscar o perfil a partir do usuário autenticado.

Essas regras pertencem ao service porque dependem de dados do banco e do contexto de negócio.

A regra arquitetural é:

SecurityConfig protege a entrada. Service protege o domínio.

As duas camadas são complementares.

7.8 Perfil de candidato e perfil de recrutador

No estado inicial, havia uma separação frágil entre usuário e recrutador. No estado final, `Usuario` passou a ser a base autenticável, e os papéis de candidato e recrutador passaram a ser tratados como perfis associados.

Isso permite que um mesmo usuário possa:

- ser apenas candidato;
- ser apenas recrutador;
- ter os dois perfis.

Essa decisão é importante para o produto, porque uma pessoa pode procurar oportunidades e também publicar oportunidades, dependendo do contexto.

Do ponto de vista de segurança, isso significa que não basta estar autenticado. Alguns fluxos exigem perfil específico.

Por exemplo:

- demonstrar interesse em vaga exige perfil candidato;
- gerenciar oportunidades exige perfil recrutador;
- aprovar vínculos ou moderar tipos exige perfil admin.

Quando o perfil necessário não existe, a API deve responder com erro de regra de negócio ou permissão conforme o caso.

7.9 Segurança no fluxo de oportunidade

A oportunidade de emprego é um dos pontos mais importantes de autorização.

No estado final, oportunidade sempre nasce no contexto do recrutador autenticado. O cliente não envia `recrutadorId` para criar a vaga.

Essa decisão evita que um usuário tente criar oportunidade em nome de outro recrutador.

Além disso, a atualização e exclusão de oportunidade precisam respeitar o dono da vaga.

Outra regra importante é `publicarComoEmpresa`. O recrutador só pode publicar em nome da empresa se tiver vínculo aprovado. Isso impede que um recrutador use indevidamente o nome de uma empresa.

A regra consolidada é:

Recrutador autenticado pode gerenciar suas oportunidades. Publicação em nome de empresa depende de vínculo aprovado.

7.10 Segurança no fluxo de empresa

Empresa também possui regras sensíveis.

No MVP final, empresa existe como entidade pública, mas ainda não possui ownership formal completo. O sistema permite vínculo entre recrutador e empresa, com aprovação administrativa.

Isso significa que:

- recrutador pode solicitar vínculo com empresa;
- admin pode aprovar ou recusar;
- somente vínculos aprovados devem aparecer publicamente;
- publicar oportunidade como empresa depende desse vínculo aprovado.

A ausência de ownership formal é uma limitação conhecida do MVP. O próximo desenvolvedor não deve interpretar isso como esquecimento, mas como decisão de escopo.

No futuro, será necessário decidir se empresa terá dono, administradores, conta própria, login por CNPJ ou outro modelo de governança.

7.11 Segurança no fluxo de candidato

O candidato possui fluxos pessoais:

- interesses em vagas;
- experiências profissionais;
- indicações recebidas/dadas;
- perfil candidato.

Esses fluxos devem sempre ser protegidos pelo usuário autenticado.

A principal regra é:

O candidato não deve manipular recursos de outro candidato.

Por isso, endpoints de interesse e experiências usam `/me` e devem buscar o perfil candidato a partir do token.

No caso de interesses, a API também bloqueia duplicidade e trata remoção de interesse inexistente como regra de negócio.

7.12 Segurança no fluxo admin

O fluxo admin foi organizado em namespace próprio.

Isso facilita:

- configurar autorização;
- documentar endpoints;
- orientar o frontend;
- separar telas administrativas de telas comuns.

Endpoints admin são usados para operações globais, como gestão de usuários, moderação de tipos de emprego e aprovação de vínculos entre recrutador e empresa.

A regra é:

O frontend só deve exibir telas admin para usuários com permissão administrativa.

Mesmo assim, o backend nunca deve depender apenas do frontend. A API precisa continuar bloqueando acesso sem `ROLE_ADMIN`.

7.13 Swagger e segurança

A documentação Swagger/OpenAPI também foi ajustada para refletir a segurança real.

No estado final:

- Bearer JWT aparece no Swagger;
- endpoints protegidos herdam autenticação;
- endpoints públicos aparecem sem exigência de token;
- `/login` foi documentado manualmente;
- Swagger UI fica habilitado no ambiente local;

- Swagger fica desabilitado em test/prod.

Essa decisão ajuda o frontend e novos desenvolvedores a testarem a API com clareza, mas evita expor documentação interativa em ambientes onde ela não deve estar disponível.

7.14 O que o frontend precisa entender sobre segurança

Para o frontend, a lógica principal é:

- endpoints públicos podem ser consumidos sem token;
- endpoints `/me` exigem token;
- endpoints admin exigem token de usuário admin;
- 401 indica problema de autenticação;
- 403 indica falta de permissão;
- o token vem no header `Authorization` após login;
- o token deve ser reenviado como Bearer Token;
- o frontend não deve montar fluxos sensíveis usando `usuarioId` quando existir endpoint `/me`.

Também é importante entender que uma tela pública e uma tela interna podem ver conjuntos diferentes de dados. Isso acontece por regra de segurança e visibilidade, especialmente por causa de LocalidadePendente.

7.15 Cuidados para manutenção futura

Ao evoluir segurança e autorização, alguns cuidados devem ser preservados.

Primeiro, não criar novos endpoints pessoais com `usuarioId` no path se a ação puder ser feita por `/me`.

Segundo, não confiar apenas no frontend para esconder ações. O backend precisa validar permissão.

Terceiro, não misturar fluxo admin com fluxo do usuário comum. Admin deve continuar em `/api/admin/**`.

Quarto, ao criar novas ações sobre Empresa ou Oportunidade, validar se o usuário tem direito sobre aquele recurso.

Quinto, se futuramente for implementado ownership formal de Empresa, revisar cuidadosamente as regras de publicação, vínculo de recrutador, administração e permissões.

Sexto, se for implementada revogação de token ou blacklist, garantir que o fluxo continue compatível com a autenticação stateless atual ou documentar claramente a mudança.

7.16 Síntese

A evolução de segurança da API foi da autenticação básica para uma autorização mais orientada ao domínio.

No início, a API já tinha JWT, mas ainda havia risco de manipulação de recursos por ID. No estado final, a API passou a usar endpoints `/me`, escopo pelo usuário autenticado, endpoints admin separados, validações de ownership nos services e respostas de erro padronizadas.

A regra mais importante é:

Autenticação identifica o usuário. Services e regras de domínio determinam o que ele pode fazer.

Essa separação deve orientar qualquer evolução futura da API.

8. Evolução do domínio

A evolução do domínio foi um dos pontos centrais da refatoração da API **Quero Trabalhar Backend**. No estado inicial, os conceitos principais do produto já existiam, mas ainda estavam modelados de forma simples e, em alguns casos, frágil. Usuário, recrutador, oportunidade, tipo de emprego, experiência, interesse e indicação já apareciam no código, mas ainda faltava uma organização mais clara das responsabilidades, dos relacionamentos e das regras de negócio.

A refatoração não criou um domínio totalmente novo. Ela reorganizou e amadureceu os conceitos que já existiam, transformando uma base inicial de CRUDs em um modelo mais coerente para o MVP.

O principal movimento foi sair de uma modelagem baseada em entidades isoladas e campos textuais para uma modelagem mais orientada a papéis, vínculos, visibilidade pública, localidade validada e ações do usuário autenticado.

8.1 Visão geral da evolução do domínio

No estado inicial, o domínio era formado principalmente por:

- `Usuario;`
- `UsuarioRecrutador;`
- `OportunidadeDeEmprego;`
- `TipoDeEmprego;`
- `ExperienciaProfissional;`
- `Indicacoes;`
- `InteresseEmOportunidades;`
- `InteresseEmEmprego.`

Esses elementos já representavam a ideia geral do produto, mas ainda havia lacunas importantes. Empresa não existia como entidade própria, recrutador era separado do usuário autenticável, oportunidade não possuía dono claro, localidade era texto livre e os interesses do candidato ainda dependiam de IDs enviados pelo cliente.

No estado final, o domínio passou a ser organizado em torno dos seguintes módulos principais:

- usuários;
- perfis de candidato e recrutador;
- empresas;
- oportunidades de emprego;
- tipos de emprego;

- experiências profissionais;
- interesses em vagas;
- indicações;
- localidades;
- administração.

Essa evolução permitiu que cada parte do domínio tivesse um papel mais claro dentro da API.

8.2 Usuario como base autenticável

Uma das decisões mais importantes foi consolidar **Usuario** como a base autenticável do sistema.

No estado inicial, **Usuario** representava principalmente o usuário/candidato, enquanto **UsuarioRecrutador** existia como entidade separada. Isso gerava uma divisão frágil, porque o recrutador não estava naturalmente integrado à identidade autenticada.

No estado final, **Usuario** passou a ser o centro da identidade do sistema. A autenticação acontece sobre o usuário, e os papéis de candidato e recrutador são representados por perfis associados.

Essa decisão permite que uma mesma pessoa possa:

- atuar como candidato;
- atuar como recrutador;
- possuir os dois perfis ao mesmo tempo.

Essa flexibilidade é importante para o domínio do produto. Uma pessoa pode estar procurando trabalho e, em outro contexto, também publicar oportunidades ou atuar como recrutadora.

A regra consolidada é:

Usuario representa a identidade autenticável. Perfis representam papéis de negócio.

8.3 PerfilCandidato

O **PerfilCandidato** foi introduzido para separar melhor o papel de candidato da identidade do usuário.

No estado inicial, o candidato era basicamente o próprio **Usuario**. Isso funcionava para fluxos simples, mas dificultava a evolução quando o mesmo usuário poderia ter outros papéis.

No estado final, o perfil de candidato concentra os comportamentos ligados à pessoa que busca oportunidades.

Entre os principais fluxos relacionados ao candidato estão:

- demonstrar interesse em vagas;
- remover interesse em vagas;
- listar vagas de interesse;
- gerenciar experiências profissionais;
- manter preferências de empregabilidade como base de evolução;
- receber e dar indicações por meio do usuário autenticado.

A principal regra de negócio é:

O candidato só deve manipular recursos próprios em endpoints **/me**.

Isso significa que o frontend não deve enviar **usuarioId** para manipular interesses ou experiências pessoais. O backend deve descobrir o usuário pelo token JWT e buscar o perfil de candidato correspondente.

8.4 PerfilRecrutador

O **PerfilRecrutador** foi introduzido para representar o papel do usuário que publica oportunidades.

No estado inicial, o recrutador era uma entidade separada chamada **UsuarioRecrutador**, sem integração robusta com o usuário autenticado. Além disso, a empresa era apenas um campo textual.

No estado final, o recrutador passou a ser um perfil associado ao **Usuario**.

Esse perfil é responsável por:

- publicar oportunidades;
- listar suas próprias oportunidades;
- solicitar vínculo com empresa;
- publicar em nome da empresa quando o vínculo estiver aprovado;
- consultar dados internos do próprio perfil.

A regra principal é:

Oportunidades são criadas no contexto do recrutador autenticado.

Isso evita que o cliente informe **recrutadorId** e tente criar vaga em nome de outro usuário.

Essa decisão reforça a segurança e deixa o domínio mais coerente: uma oportunidade sempre tem um responsável claro.

8.5 Usuário com múltiplos papéis

Uma decisão importante do domínio foi permitir que um usuário tenha mais de um papel.

No estado final, um usuário pode ter perfil de candidato e perfil de recrutador ao mesmo tempo. Isso evita duplicar contas e reflete melhor a realidade do produto.

Essa decisão também influencia o frontend. O mesmo usuário pode acessar telas de candidato e telas de recrutador, desde que possua os perfis correspondentes.

Do ponto de vista de backend, isso exige cuidado para não confundir autenticação com perfil de negócio.

Estar autenticado significa que existe um usuário válido. Porém, algumas ações exigem um perfil específico.

Exemplos:

- demonstrar interesse em vaga exige perfil de candidato;
- publicar oportunidade exige perfil de recrutador;
- aprovar vínculo empresa-recrutador exige perfil admin.

A regra consolidada é:

Autenticação identifica o usuário. Perfis definem capacidades de negócio.

8.6 Empresa como entidade de domínio

A criação de **Empresa** como entidade foi uma das evoluções mais importantes.

No estado inicial, empresa aparecia como texto em recrutador e oportunidade. Isso impedia qualquer controle real sobre empresas, vínculos ou publicação em nome de empresa.

No estado final, empresa passou a ter identidade própria e dados públicos.

A entidade empresa passou a representar:

- nome;
- descrição;
- contato público;
- site;
- localidade;
- status ativo;
- relação com recrutadores vinculados.

Com isso, a API passou a permitir:

- cadastro de empresas;
- listagem pública de empresas;
- consulta pública de empresa;
- listagem de oportunidades da empresa;
- listagem de recrutadores vinculados;
- solicitação de vínculo por recrutador;
- aprovação ou recusa de vínculo por admin.

A principal regra de visibilidade é:

Empresa só aparece publicamente se possuir Localidade validada.

Se a empresa tiver localidade pendente, ela pode existir internamente, mas não deve aparecer nos endpoints públicos.

Essa regra foi importante para manter qualidade das listagens públicas.

8.7 Vínculo entre recrutador e empresa

Com a criação da entidade **Empresa**, surgiu a necessidade de controlar a relação entre recrutadores e empresas.

No MVP, a decisão foi criar um vínculo moderado entre recrutador e empresa. O recrutador pode solicitar vínculo, e um admin pode aprovar ou recusar.

Essa solução resolve um problema importante: impede que qualquer recrutador publique em nome de qualquer empresa sem validação.

A regra consolidada é:

Recrutador só pode publicar em nome da empresa se tiver vínculo aprovado.

Essa regra afeta diretamente **publicarComoEmpresa** no fluxo de oportunidade.

Ainda assim, foi tomada uma decisão de escopo: ownership formal de empresa ficou para pós-MVP. Ou seja, o sistema possui vínculo aprovado entre recrutador e empresa, mas ainda não possui uma modelagem completa de dono da empresa, administradores da empresa, conta empresarial ou login por CNPJ.

Essa limitação deve ser preservada como decisão consciente de MVP, não interpretada como esquecimento.

8.8 OportunidadeDeEmprego

A evolução de **OportunidadeDeEmprego** foi central para o domínio.

No estado inicial, oportunidade tinha campos como descrição, tipo de emprego, modalidade, cidade, estado e empresa textual. Não havia vínculo consistente com recrutador autenticado nem regra forte de ownership.

No estado final, oportunidade passou a ser uma entidade de negócio mais consistente.

As principais regras passaram a ser:

- oportunidade nasce no contexto do recrutador autenticado;
- cliente não informa `recrutadorId`;
- tipo de emprego precisa existir e estar aprovado;
- modalidade deve ser uma das opções permitidas;
- localidade é obrigatória por IDs estruturados ou texto livre;
- oportunidade pública exige Localidade validada;
- oportunidade com LocalidadePendente fica oculta do público;
- endpoint `/me` do recrutador pode mostrar oportunidades pendentes;
- `publicarComoEmpresa` depende de vínculo aprovado com empresa.

Essa evolução resolveu o problema de oportunidade sem dono claro e melhorou a segurança da API.

8.9 Publicação como pessoa ou como empresa

Uma decisão de negócio importante foi permitir que o recrutador publique uma oportunidade em dois contextos:

- em nome próprio;
- em nome de uma empresa vinculada e aprovada.

Essa distinção é controlada pelo campo `publicarComoEmpresa`.

A regra é:

`publicarComoEmpresa=true` apenas solicita publicação em nome da empresa. A publicação só será associada à empresa se o recrutador tiver vínculo aprovado.

Isso evita uso indevido do nome de uma empresa.

Também foi decidido que a atualização da oportunidade não deve transformar livremente o contexto original de publicação. Essa cautela evita que uma vaga pessoal seja convertida sem controle em vaga de empresa, ou o contrário, sem regra explícita.

Essa é uma área que pode evoluir no futuro, principalmente quando ownership formal de empresa for implementado.

8.10 TipoDeEmprego como catálogo moderado

TipoDeEmprego já existia no estado inicial, com título e descrição. Porém, no estado final, ele passou a ter uma função mais clara: servir como catálogo moderado de áreas/cargos/tipos de trabalho.

A decisão foi separar tipos aprovados de tipos sugeridos.

As regras consolidadas são:

- tipos criados por admin podem nascer aprovados;
- tipos sugeridos por usuários entram como não aprovados;
- catálogo público exibe apenas aprovados;
- oportunidade só pode usar tipo aprovado;
- admin pode listar tipos não aprovados;
- admin pode aprovar individualmente;
- admin pode aprovar em lote;
- exclusão respeita integridade quando há uso em oportunidades ou experiências.

Essa decisão evita que o catálogo público fique poluído com sugestões duplicadas, mal escritas ou inadequadas.

Para o frontend, isso significa que campos de seleção pública devem usar o catálogo de aprovados.

8.11 ExperienciaProfissional

A experiência profissional passou a ser tratada como recurso do perfil de candidato.

No estado inicial, ela já existia como entidade ligada ao usuário e ao tipo de emprego, mas ainda sem uma separação forte de ownership e contrato.

No estado final, a experiência pertence ao candidato e pode ser gerenciada em fluxos `/me`.

As regras principais são:

- candidato gerencia as próprias experiências;
- experiência se relaciona a um tipo de emprego;
- datas de início e fim são usadas para representar histórico;
- experiência em andamento pode ser representada pela ausência de data final;
- admin pode consultar e gerenciar experiências globalmente.

Essa separação permite duas visões:

- visão pessoal do candidato;
- visão administrativa.

No frontend, telas pessoais devem usar endpoints `/me`, enquanto telas administrativas devem usar endpoints admin.

8.12 Interesse em vagas

O interesse do candidato em oportunidades foi reorganizado para ficar mais seguro e mais alinhado ao usuário autenticado.

No estado inicial, o interesse era feito por endpoint com `usuarioId`, o que criava risco de manipulação de interesse de outro usuário.

No estado final, o candidato autenticado pode:

- demonstrar interesse em uma vaga;

- remover interesse;
- listar vagas de interesse;
- filtrar e paginar a listagem de interesses.

As regras consolidadas são:

- duplicidade de interesse é bloqueada;
- remover interesse inexistente gera regra de negócio;
- listagem de interesses pertence ao candidato autenticado;
- endpoint `/me` pode mostrar vagas pendentes quando fizer sentido;
- filtro `statusLocalidade` pode ser usado em contexto interno.

Essa mudança reduziu risco de IDOR e melhorou a integração com frontend.

8.13 Indicação

O módulo de indicação evoluiu para representar recomendações entre usuários.

No estado inicial, indicação era uma entidade simples, sem uma modelagem clara de autor, destinatário ou ownership.

No estado final, indicação passou a funcionar como ação de usuário autenticado.

As regras principais são:

- usuário autenticado pode indicar outro usuário;
- autoindicação é proibida;
- usuário pode listar indicações dadas;
- usuário pode listar indicações recebidas;
- exclusão só deve ser permitida ao autor;
- listas de indicações continuam como `List`, por serem fluxos pessoais menores no MVP.

Essa modelagem é suficiente para o MVP e pode evoluir futuramente com campos adicionais, ordenação por data, paginação ou status.

8.14 Preferencia

Preferencia aparece no domínio final como base para interesses de empregabilidade do candidato, mas não foi consolidada como fluxo HTTP completo no MVP.

Isso significa que a entidade existe como preparação para evolução futura, mas a API final priorizou outros fluxos mais críticos, como:

- interesses em vagas;
- experiências profissionais;
- oportunidades;
- empresas;
- localidade;
- indicações.

Essa decisão é importante para o próximo desenvolvedor: a presença da entidade não significa que o fluxo de preferências esteja finalizado como produto. Ela deve ser tratada como base de domínio para evolução posterior.

8.15 Localidade como parte transversal do domínio

Localidade deixou de ser apenas um detalhe de endereço e passou a ser uma regra transversal do domínio.

Ela afeta diretamente:

- empresas;
- oportunidades;
- listagens públicas;
- filtros;
- status interno em endpoints **/me**;
- reprocessamento de pendências;
- experiência do frontend;
- qualidade do catálogo público.

A decisão final foi:

Localidade validada define visibilidade pública.

Isso significa que localidade não é apenas um campo informativo. Ela participa da regra de publicação de recursos.

Esse é um dos pontos mais importantes da evolução do domínio e será detalhado em seção própria.

8.16 Administração como parte do domínio

O domínio também evoluiu para incluir fluxos administrativos.

No estado inicial, havia role **ADMIN**, mas pouca estrutura real de administração. No estado final, endpoints admin passaram a cuidar de operações globais, como:

- gestão de usuários;
- gestão de experiências;
- moderação de tipos de emprego;
- aprovação de vínculos entre recrutador e empresa;
- consulta por IDs em contexto administrativo.

A decisão foi agrupar esses fluxos em **/api/admin/****.

Isso separa claramente ações administrativas de ações pessoais do usuário e facilita controle de segurança, documentação e integração com frontend.

8.17 DTO público, DTO interno e domínio

Uma consequência importante da evolução do domínio foi a necessidade de diferentes representações para o mesmo recurso.

Por exemplo, uma oportunidade pode ser vista:

- publicamente por qualquer usuário;
- internamente pelo recrutador dono;
- administrativamente por um admin.

Essas visões não devem necessariamente expor os mesmos dados.

Por isso, foram criados DTOs diferentes para contextos diferentes.

A regra geral é:

- DTO público mostra apenas informação segura e publicável;
- DTO interno pode mostrar informações operacionais do dono;
- DTO admin pode mostrar dados necessários à administração;
- entidade JPA não deve ser exposta diretamente.

Essa separação é fundamental para preservar regras de visibilidade e evitar vazamento de detalhes técnicos.

8.18 Síntese da evolução do domínio

A evolução do domínio transformou uma estrutura inicial baseada em entidades simples e campos textuais em um modelo mais adequado ao MVP.

As principais mudanças foram:

- **Usuario** virou base autenticável central;
- candidato e recrutador passaram a ser perfis;
- empresa deixou de ser texto e virou entidade;
- oportunidade passou a pertencer ao recrutador autenticado;
- publicação como empresa passou a depender de vínculo aprovado;
- tipo de emprego virou catálogo moderado;
- experiência passou a pertencer ao perfil candidato;
- interesse em vaga passou a ser fluxo **/me**;
- indicação passou a ser ação do usuário autenticado;
- localidade passou a controlar visibilidade pública;
- administração passou a ter namespace e regras próprias.

A principal ideia por trás dessa evolução foi tornar o domínio mais próximo das regras reais do produto.

O projeto deixou de tratar os dados como CRUDs independentes e passou a organizar relações de negócio: quem é o usuário, qual papel ele exerce, que recurso

pertence a quem, o que pode aparecer publicamente, o que precisa de moderação e o que deve ficar pendente até validação.

9. Localidade e LocalidadePendente

O módulo de **Localidade** foi a parte mais importante e mais sensível da evolução da API. Ele não representa apenas uma melhoria de modelagem; ele define uma regra central de visibilidade pública do sistema.

No estado inicial, localização era tratada como texto livre. Oportunidades tinham **cidade** e **estado** como **String**, preferências armazenavam cidades, estados e países também como listas de texto, e não existiam entidades como **País**, **Estado**, **Cidade**, **Localidade** ou **LocalidadePendente**. Também não havia integração com Google Maps, validação de localidade ou fluxo formal para tratar localidades ambíguas ou não encontradas.

No estado final, a API passou a tratar **Localidade validada** como fonte oficial e **LocalidadePendente** como uma fila técnica temporária. Essa decisão foi tomada para equilibrar duas necessidades: permitir que o usuário conclua cadastros mesmo quando a localidade não puder ser resolvida imediatamente, mas impedir que empresas ou oportunidades com localização incerta apareçam publicamente.

A regra central é:

Recursos públicos só devem aparecer quando possuem Localidade validada. **LocalidadePendente** permite salvar o recurso internamente, mas não autoriza sua exposição pública.

9.1 O problema inicial de localização

No início, a API não tinha um módulo de localidade. Cidade, estado e país eram apenas textos livres ou listas de textos.

Isso causava vários problemas práticos:

- dificuldade para filtrar por cidade, estado ou país;

- dados inconsistentes por acento, abreviação, caixa ou erro de digitação;
- falta de normalização entre candidato, oportunidade e empresa;
- impossibilidade de saber se uma localidade realmente existia;
- possibilidade de publicar uma vaga com localização textual inválida;
- ausência de fluxo para lidar com localidade não encontrada ou ambígua;
- dificuldade futura para busca geográfica confiável.

O diagnóstico inicial foi claro: sem um modelo de localidade, empresas e oportunidades não se beneficiavam de uma referência geográfica comum, e a API não conseguia garantir a qualidade da informação exibida publicamente.

Por isso, localidade se tornou uma frente crítica da refatoração.

9.2 Decisão principal: Localidade validada como fonte oficial

A decisão central do estado final foi considerar **Localidade** como a fonte oficial de localização.

No modelo final, localidade é composta por uma estrutura relacional envolvendo:

- país;
- estado;
- cidade;
- objeto de localidade usado pelos recursos.

Essa estrutura permite que a API trabalhe com referências consistentes, em vez de depender de textos soltos.

Na prática, isso significa que empresas e oportunidades públicas só entram em listagens se possuírem localidade validada. O diagnóstico final registra essa regra de forma direta: **Localidade** é a fonte oficial, formada pela composição relacional de **País**, **Estado** e **Cidade**, e empresas e oportunidades públicas só entram em listagens quando possuem localidade válida.

Essa regra passou a orientar várias partes da API:

- listagens públicas de oportunidades;

- listagens públicas de empresas;
- consulta pública de oportunidade por ID;
- consulta pública de empresa por ID;
- filtros por país, estado e cidade;
- separação entre DTO público e DTO interno;
- comportamento dos endpoints [/me](#).

Localidade deixou de ser apenas um campo informativo e passou a ser uma regra de publicação.

9.3 Por que **LocalidadePendente** existe

Criar apenas uma regra rígida de validação resolveria parte do problema, mas criaria outro: o usuário poderia ser impedido de concluir um cadastro sempre que a API não conseguisse validar a localidade imediatamente.

Isso poderia acontecer por vários motivos:

- localidade digitada com variação;
- base interna ainda incompleta;
- resposta ambígua do Google Maps;
- falha externa temporária;
- localidade válida, mas ainda não cadastrada internamente;
- texto insuficiente para decisão segura.

A solução foi criar **LocalidadePendente**.

LocalidadePendente existe para registrar uma tentativa de localização que ainda não foi validada com segurança. Ela permite que o recurso seja salvo internamente, mas sem ficar visível publicamente.

A decisão foi:

Não bloquear necessariamente o cadastro, mas também não publicar recurso com localização incerta.

Essa decisão é essencial para o MVP. Ela preserva a experiência do usuário sem comprometer a confiabilidade das listagens públicas.

9.4 LocalidadePendente não é estado público normal

Um ponto fundamental é que `LocalidadePendente` não deve ser entendida como um estado público comum do produto.

Ela é uma fila técnica temporária de resolução.

Isso significa que ela não deve ser tratada como uma localidade alternativa, nem como algo que o usuário público deve navegar, filtrar ou consultar.

A regra é:

- usuário público não vê recurso pendente;
- listagem pública não retorna recurso pendente;
- DTO público não expõe dados de pendência;
- endpoints `/me` podem mostrar pendência ao dono;
- serviços internos podem reprocessar a pendência;
- no futuro, admin pode ganhar visão operacional se fizer sentido.

Essa distinção é uma das decisões mais importantes da API.

Se um recurso tem `LocalidadePendente`, ele existe internamente, mas ainda não está apto para publicação pública.

9.5 Público, interno e admin no contexto de localidade

A regra de localidade muda conforme o contexto do endpoint.

9.5.1 Endpoints públicos

Endpoints públicos devem retornar apenas recursos com localidade validada.

Isso vale principalmente para:

- empresas públicas;

- oportunidades públicas;
- oportunidades de uma empresa pública;
- recrutadores públicos vinculados à empresa, quando aplicável ao contexto.

A pessoa usuária que está navegando sem autenticação não deve encontrar empresas ou vagas cuja localidade ainda não foi validada.

9.5.2 Endpoints `/me`

Endpoints `/me` são diferentes.

Eles representam o painel do próprio usuário autenticado. Por isso, podem retornar recursos pendentes pertencentes ao dono.

Por exemplo, uma oportunidade criada por um recrutador pode não aparecer em `GET /api/oportunidades`, mas aparecer em `GET /api/recrutadores/me/oportunidades` com status de localidade pendente.

Isso não é inconsistência. É uma regra de visibilidade.

O público vê apenas o que está validado. O dono vê também o que ainda está em processamento.

9.5.3 Endpoints admin

No MVP, a visão administrativa de localidade pendente ainda não foi tratada como um painel completo. A ideia de controle operacional avançado da fila ficou para evolução futura.

Mesmo assim, a documentação e o desenho atual já preparam o caminho para que, futuramente, um admin possa consultar ou agir sobre pendências, sem transformar `LocalidadePendente` em estado público normal.

9.6 Fluxo geral de resolução de localidade

O fluxo de resolução foi desenhado para aceitar tanto dados estruturados quanto texto livre.

Quando o cliente informa IDs estruturados de país, estado e cidade, a API tenta montar a localidade diretamente a partir da base oficial. Essa é a forma mais confiável.

Quando o cliente informa texto livre, a API tenta resolver esse texto em etapas.

O fluxo conceitual é:

1. receber dados de localidade no cadastro ou atualização;
2. priorizar IDs estruturados quando fornecidos;
3. tentar resolver pela base interna oficial;
4. se a base interna não resolver, tentar fallback externo;
5. se houver resultado seguro, criar ou reutilizar Localidade validada;
6. se houver ambiguidade, ausência ou falha, registrar LocalidadePendente;
7. manter o recurso oculto dos endpoints públicos;
8. permitir que endpoints internos mostrem a pendência ao dono;
9. reprocessar a pendência em segundo plano;
10. promover para Localidade validada quando possível.

Essa lógica reduz a chance de publicar dados geográficos incorretos e evita bloquear todos os cadastros em caso de falha externa.

9.7 Responsabilidades das classes do módulo

O módulo de localidade foi separado em componentes para evitar acoplamento excessivo.

LocalidadeService atua como catálogo oficial. Ele expõe países, estados e cidades existentes na base e não deve chamar Google Maps nem consultar pendências. Esse ponto é importante: o catálogo oficial é uma coisa; o fluxo de resolução de texto livre é outra.

LocalidadeResolucaoService orquestra a resolução no fluxo normal de cadastro. É ele que coordena a tentativa de transformar IDs ou texto livre em uma localidade validada ou pendente.

`FluxoResolucaoLocalidadeService` executa a tentativa pura de resolução, sem efeito colateral de criação de pendência. Ele separa o ato de tentar resolver do ato de registrar uma pendência.

`ResolvedorLocalidadeBaseInterna` tenta resolver primeiro usando a base oficial existente.

`ResolvedorLocalidadeGoogleMaps` encapsula a integração externa. O Google Maps entra como fallback, não como fonte pública.

`ConversorGoogleResultParaLocalidade` traduz o resultado externo para o modelo interno da aplicação.

`RegistroLocalidadePendenteService` registra ou associa uma pendência quando a localidade não pode ser validada com segurança.

`LocalidadePendenteReprocessamentoService` tenta resolver pendências existentes em segundo plano e promover recursos para localidade validada.

`LocalidadePendenteReprocessamentoScheduler` apenas dispara periodicamente o reprocessamento conforme configuração. Ele não deve conter regra de negócio.

Essa separação evita que o módulo vire um único service grande e difícil de manter.

9.8 Google Maps como fallback encapsulado

A integração com Google Maps foi pensada como um detalhe de infraestrutura do fluxo de resolução, não como parte do contrato público da API.

Isso significa que:

- controller não chama Google Maps;
- endpoint público não depende diretamente de Google Maps;
- `LocalidadeService` não consulta Google Maps;
- Google Maps não é exposto como endpoint;
- falhas externas não devem vazar como detalhe do contrato público;

- resultados externos passam por validação antes de virar Localidade.

Essa decisão reduz acoplamento e evita que a API pública dependa diretamente de uma fonte externa em tempo de listagem.

A documentação final reforça que Google Maps não vira fonte pública; ele apenas auxilia a resolução.

9.9 Ambiguidade não deve ser resolvida automaticamente

Uma decisão crítica foi não escolher automaticamente resultados ambíguos.

Se uma busca por texto livre retorna mais de uma localidade possível, a API não deve simplesmente escolher a primeira. Fazer isso poderia publicar uma empresa ou vaga na cidade errada.

Esse risco é especialmente grave porque localidade controla visibilidade pública. Uma escolha errada poderia fazer uma vaga aparecer em filtros de cidade/estado incorretos.

Por isso, a regra consolidada foi:

Resultado ambíguo não deve ser escolhido automaticamente.

Quando há ambiguidade, a API mantém ou cria pendência. A confirmação manual de uma sugestão ficou para pós-MVP.

Essa decisão privilegia segurança e consistência em vez de conveniência imediata.

9.10 Dono genérico da pendência

Inicialmente, foi discutido que uma pendência poderia ficar referenciada diretamente em entidades como Empresa ou Oportunidade. Porém, essa solução aumentaria o acoplamento entre entidades de negócio e uma fila técnica temporária.

A decisão final foi modelar `LocalidadePendente` com dono genérico:

- tipo do recurso;
- ID do recurso;

- campo alvo.

Essa estrutura permite que a pendência aponte para o recurso afetado sem exigir que `Empresa` ou `OportunidadeDeEmprego` carreguem uma relação JPA direta com `LocalidadePendente`.

A decisão arquitetural é:

Entidades de negócio não devem depender diretamente da fila técnica de pendência.

Essa escolha preserva a ideia de que `LocalidadePendente` é mecanismo temporário, não parte estrutural permanente do domínio público.

O diagnóstico final registra exatamente isso: a pendência não é relação direta nas entidades de negócio; o dono é genérico por `tipoRecurso + recursoId + campoAlvo`.

9.11 Reprocessamento de pendências

Outra parte importante do desenho final é o reprocessamento.

Quando uma pendência é criada, ela não deve ficar parada indefinidamente como estado normal. A API possui serviço e scheduler para tentar resolver pendências em segundo plano.

O objetivo do reprocessamento é:

- buscar pendências existentes;
- tentar resolver novamente;
- não criar nova pendência duplicada;
- aplicar Localidade validada ao recurso quando resolver;
- remover a pendência resolvida.

Essa regra é importante:

Reprocessamento trabalha sobre a pendência existente. Ele não deve duplicar `LocalidadePendente`.

O diagnóstico final indica que o scheduler roda em intervalo configurável e que o serviço de reprocessamento tenta promover pendências para localidade validada, removendo a pendência quando resolve.

Essa abordagem mantém **LocalidadePendente** como fila técnica temporária.

9.12 Localidade em Empresa

Empresa passou a depender de localidade.

No estado final, o cadastro de empresa pode usar IDs válidos ou texto livre resolvível/pendente. Quando a localidade é validada, a empresa pode aparecer publicamente. Quando fica pendente, a empresa é persistida internamente, mas não aparece no público.

Essa regra é importante para o frontend.

Se a pessoa criou uma empresa e ela não aparece na listagem pública, isso pode acontecer porque a localidade ainda está pendente. O frontend deve tratar isso em telas internas, não como falha geral da API.

DTOs públicos de empresa não devem mostrar pendência. DTOs internos podem trazer status e motivo para apoiar o usuário ou fluxo administrativo.

9.13 Localidade em OportunidadeDeEmprego

Oportunidade também depende de localidade.

A regra final é:

- oportunidade precisa de localidade;
- localidade pode ser informada por IDs ou texto livre;
- se for validada, a vaga pode aparecer publicamente;
- se ficar pendente, a vaga é salva, mas fica oculta do público;
- o recrutador pode visualizar a vaga pendente em **/me**.

O diagnóstico final registra que o público só vê vagas com localidade validada e que **/me** do recrutador pode ver vagas pendentes e detalhes técnicos.

Essa regra deve ser preservada nas specifications, nos services e nos DTOs.

9.14 Localidade em filtros públicos

Como Localidade passou a ser estruturada, os filtros públicos puderam se tornar mais confiáveis.

Endereços textuais livres não são bons para filtro. Com **País**, **Estado** e **Cidade**, os filtros podem usar IDs estruturados.

Isso impacta endpoints como:

- listagem pública de empresas;
- listagem pública de oportunidades;
- oportunidades de uma empresa;
- listagens internas que aceitam filtro por localidade.

A regra é que filtros públicos por localidade operam sobre localidade validada. Eles não devem buscar em texto de pendência.

Se a pessoa filtra oportunidades por cidade, a API deve retornar apenas oportunidades publicamente válidas naquela cidade, não oportunidades pendentes com texto original parecido.

9.15 Status de localidade em endpoints internos

Em endpoints internos, a API pode expor status de localidade para ajudar o dono do recurso.

Isso aparece principalmente em fluxos como:

- oportunidades do recrutador autenticado;
- vagas de interesse do candidato autenticado;
- dados internos de empresa, quando aplicável.

Campos internos podem indicar:

- se a localidade está validada;

- se está pendente;
- qual foi o texto original informado;
- qual é o motivo da pendência;
- qual é o status técnico da validação.

Essa exposição foi aceita no MVP para dar feedback ao usuário autenticado. Porém, foi reconhecido que, futuramente, esses DTOs internos podem ser refinados para esconder detalhes técnicos demais e oferecer mensagens mais orientadas ao produto.

A regra atual é:

DTO público não expõe pendência. DTO interno pode expor status de localidade quando isso ajuda o dono do recurso.

9.16 StatusLocalidadeFiltro

Durante a evolução, foi discutida a existência do enum `StatusLocalidadeFiltro`.

A decisão foi manter esse filtro apenas em contextos internos, especialmente endpoints `/me`.

A regra é:

- endpoint público não deve filtrar por status de localidade;
- público já retorna apenas validado;
- endpoint interno pode permitir filtrar entre `VALIDADA` e `PENDENTE`;
- admin pode usar esse conceito futuramente se houver tela operacional.

Essa distinção evita transformar pendência em recurso público pesquisável.

Para o frontend, isso significa que `statusLocalidade` pode fazer sentido em telas internas, como painel do recrutador, mas não em listagens públicas.

9.17 O que acontece quando a localidade não é resolvida

Quando uma localidade não é resolvida com segurança, a API não deve tratar isso necessariamente como erro final de cadastro.

Em vez disso, pode criar uma pendência.

O comportamento esperado é:

- recurso é salvo internamente;
- pendência registra texto original e motivo;
- recurso fica fora das listagens públicas;
- dono pode ver o status internamente;
- reproprocessamento pode tentar resolver depois.

Esse desenho reduz fricção no cadastro, mas mantém qualidade pública.

Para o frontend, a mensagem correta não deve ser “erro ao cadastrar”. A mensagem mais adequada é algo como: o recurso foi salvo, mas ainda aguarda validação da localidade antes de aparecer publicamente.

9.18 Limites do MVP no módulo de Localidade

Algumas funcionalidades foram conscientemente deixadas para depois.

As principais são:

- confirmação manual de localidade sugerida pelo usuário;
- notificação ao usuário quando recurso ficar oculto por localidade pendente;
- painel administrativo avançado da fila de pendências;
- ownership formal de empresa relacionado ao fluxo de validação;
- revisão da exposição de estados técnicos em DTOs internos;
- eventuais estados finais auditáveis para pendência resolvida.

Esses pontos não foram esquecidos. Eles foram tratados como evolução pós-MVP para evitar ampliar demais o escopo antes da entrega principal.

O diagnóstico final registra esses limites: sem confirmação manual da sugestão, sem notificação ao usuário, sem painel operacional completo da fila e com estados internos ainda parcialmente expostos em DTOs internos.

9.19 O que o próximo backend deve preservar

Ao mexer nesse módulo, o próximo desenvolvedor deve preservar algumas regras fundamentais.

A primeira é que `LocalidadeService` deve continuar sendo catálogo oficial. Ele não deve virar service de integração externa nem chamar Google Maps diretamente.

A segunda é que Google Maps deve continuar encapsulado no fluxo de resolução.

A terceira é que recurso pendente não deve aparecer publicamente.

A quarta é que ambiguidade não deve ser resolvida automaticamente.

A quinta é que `LocalidadePendente` não deve voltar a ser relacionamento direto em Empresa ou Oportunidade.

A sexta é que reprocessamento não deve criar pendências duplicadas.

A sétima é que mudanças em DTOs públicos devem ser feitas com cuidado para não vazar detalhes de pendência.

Essas regras são mais importantes do que a implementação exata de uma classe específica.

9.20 O que o frontend deve entender sobre LocalidadePendente

Para a pessoa responsável pelo frontend, a regra principal é:

`LocalidadePendente` não é erro de cadastro; é um estado interno de validação.

Em telas públicas, o frontend não precisa lidar com recursos pendentes, porque eles não devem aparecer.

Em telas internas, como painel do recrutador, o frontend pode receber informações indicando que a vaga ou empresa está pendente por localidade. Nesse caso, a interface deve comunicar que o recurso foi salvo, mas ainda não está disponível publicamente.

O frontend não deve prometer confirmação manual de localidade se esse fluxo ainda não estiver implementado.

Também não deve tentar consumir Google Maps diretamente para substituir a regra da API, a menos que isso seja uma decisão futura bem definida. A API é a fonte de verdade sobre se uma empresa ou vaga está publicamente válida.

9.21 Síntese

A evolução de Localidade foi uma das principais transformações da API.

O sistema saiu de um modelo com cidade e estado em texto livre para uma estratégia baseada em localidade oficial, pendência técnica, fallback externo encapsulado e reprocessamento em segundo plano.

Essa mudança resolveu um problema central: como permitir cadastros flexíveis sem comprometer a qualidade das listagens públicas.

A resposta arquitetural foi:

- validar quando possível;
- registrar pendência quando necessário;
- ocultar pendente do público;
- permitir visibilidade interna ao dono;
- reprocessar depois;
- evitar escolha automática em ambiguidade;
- manter Google Maps encapsulado;
- preservar LocalidadePendente como fila técnica temporária.

Essa é uma das decisões mais importantes do projeto e deve ser entendida antes de qualquer alteração em empresas, oportunidades, filtros públicos, endpoints `/me` ou integração com frontend.

10. Empresa e recrutador

A evolução dos módulos de **Empresa** e **Recrutador** foi uma das partes mais importantes da refatoração da API. No estado inicial, esses conceitos já existiam de forma indireta, mas ainda não estavam bem modelados. O recrutador aparecia como uma entidade separada do usuário autenticável, e empresa era apenas um texto associado ao recrutador ou à oportunidade.

Essa modelagem era suficiente para uma primeira versão exploratória, mas não permitia responder perguntas importantes do produto, como: quais empresas existem no sistema, quais recrutadores estão vinculados a uma empresa, quem pode publicar em nome de uma empresa e quais oportunidades pertencem a uma empresa real.

No estado final, **Empresa** passou a ser uma entidade própria, **PerfilRecrutador** passou a representar o papel de recrutador de um usuário autenticado, e o vínculo entre recrutador e empresa passou a ter um fluxo moderado por administração.

10.1 Problema inicial: empresa como texto

No estado inicial, empresa não existia como agregado próprio. Ela aparecia como campo textual em partes diferentes do sistema, especialmente em recrutador e oportunidade.

Isso gerava vários problemas:

- não havia cadastro real de empresas;
- não havia dados públicos estruturados da empresa;
- não havia localidade da empresa;
- não havia relacionamento confiável entre empresa e recrutadores;
- não havia como aprovar ou recusar vínculo;
- não havia como listar empresas publicamente;
- não havia como consultar oportunidades de uma empresa real;
- não havia regra segura para publicar vaga em nome de empresa.

Na prática, o sistema não tinha como diferenciar uma empresa real cadastrada de um simples texto digitado em uma vaga ou em um perfil de recrutador.

Essa limitação foi uma das principais motivações para transformar **Empresa** em entidade de domínio.

10.2 Criação de Empresa como entidade

No estado final, **Empresa** passou a representar uma organização cadastrada na plataforma.

Ela deixou de ser apenas uma **String** e passou a ter identidade própria, dados públicos e relação com localidade.

A empresa passou a concentrar informações como:

- nome;
- descrição;
- site;
- e-mail público;
- telefone público;
- status ativo;
- localidade;
- recrutadores vinculados;
- oportunidades associadas, quando publicadas como empresa.

Essa mudança tornou possível criar endpoints públicos de empresas, oportunidades por empresa e recrutadores vinculados à empresa.

A decisão foi importante porque empresa passou a ser um recurso navegável do produto, não apenas uma informação textual.

10.3 Empresa e Localidade

A relação entre empresa e localidade é uma regra central.

No estado final, uma empresa pode ser cadastrada com localidade validada ou ficar temporariamente associada a uma pendência de localidade. Porém, empresas com localidade pendente não aparecem publicamente.

A regra consolidada é:

Empresa só aparece nos endpoints públicos quando possui Localidade validada.

Isso significa que uma empresa pode existir internamente após o cadastro, mas ainda não estar disponível em listagens públicas caso sua localidade não tenha sido resolvida com segurança.

Essa decisão protege a qualidade do catálogo público de empresas. O sistema evita exibir uma empresa com cidade, estado ou país incertos.

Para o frontend, isso significa que uma empresa recém-cadastrada pode não aparecer imediatamente na tela pública. Isso não deve ser tratado automaticamente como erro. Pode significar que a localidade ainda está pendente.

10.4 Empresa pública e empresa interna

A refatoração também consolidou a diferença entre representação pública e representação interna da empresa.

A empresa pública é aquela retornada em endpoints abertos, como listagem e consulta pública. Ela deve expor apenas dados seguros para qualquer usuário e não deve mostrar detalhes técnicos de **LocalidadePendente**.

A empresa interna pode trazer informações adicionais, especialmente quando consultada em contexto autenticado ou administrativo. Nesse caso, pode fazer sentido expor status de localidade, texto original informado ou motivo da pendência.

A regra é:

DTO público de empresa não expõe pendência. DTO interno pode expor status de localidade quando isso ajuda o usuário ou o fluxo administrativo.

Essa separação evita vazar detalhes técnicos em telas públicas e, ao mesmo tempo, permite que o dono ou operador entenda por que a empresa ainda não está visível publicamente.

10.5 PerfilRecrutador como papel do usuário

No estado inicial, recrutador era uma entidade separada chamada `UsuarioRecrutador`, sem integração robusta com o usuário autenticado.

No estado final, a API passou a tratar recrutador como um perfil associado ao `Usuario`.

Essa decisão é importante porque `Usuario` representa a identidade autenticável, enquanto `PerfilRecrutador` representa o papel de negócio de quem pode publicar oportunidades e se vincular a empresas.

Com isso, um mesmo usuário pode:

- ter apenas perfil de candidato;
- ter apenas perfil de recrutador;
- ter perfil de candidato e recrutador ao mesmo tempo.

Essa flexibilidade é importante para o produto, porque uma pessoa pode procurar vagas e também atuar como recrutadora em outro contexto.

A regra consolidada é:

O usuário autentica. O perfil define quais capacidades de negócio ele possui.

10.6 Recrutador autenticado

As ações de recrutador passaram a ser baseadas no usuário autenticado.

Isso significa que, em fluxos de recrutador, o cliente não deve informar manualmente qual recrutador está agindo. A API deve descobrir isso a partir do token JWT.

Essa decisão reduz risco de IDOR e impede que um usuário tente agir como outro recrutador apenas alterando um ID na requisição.

Exemplos de fluxos baseados no recrutador autenticado:

- consultar dados do próprio perfil recrutador;
- listar oportunidades próprias;
- criar oportunidade;
- solicitar vínculo com empresa;
- publicar oportunidade como empresa, quando permitido.

A regra é:

Em fluxos `/me`, o recrutador vem do token, não do request.

Essa regra deve ser preservada em qualquer nova funcionalidade envolvendo recrutador.

10.7 Vínculo entre recrutador e empresa

Com a criação da entidade `Empresa`, surgiu a necessidade de controlar quais recrutadores podem atuar em nome dela.

A solução implementada no MVP foi um fluxo de vínculo moderado.

O recrutador pode solicitar vínculo com uma empresa. Depois, um usuário administrador pode aprovar ou recusar esse vínculo.

Essa decisão evita que qualquer recrutador publique vaga usando o nome de qualquer empresa.

A regra consolidada é:

Recrutador só pode publicar em nome de empresa se o vínculo com essa empresa estiver aprovado.

Esse vínculo aprovado passa a ser uma permissão de negócio. Não basta o recrutador estar autenticado; ele precisa estar autorizado a representar aquela empresa.

10.8 Aprovação e recusa de vínculo

A aprovação ou recusa de vínculo entre recrutador e empresa é uma ação administrativa.

Essa decisão foi tomada porque, no MVP, ainda não existe ownership formal de empresa. Como não há dono da empresa gerenciando seus próprios recrutadores, a moderação fica no fluxo admin.

O admin pode:

- aprovar solicitação de vínculo;
- recusar solicitação de vínculo;
- controlar quais recrutadores aparecem publicamente vinculados à empresa;
- permitir ou impedir publicação de vagas em nome da empresa.

Essa regra protege a credibilidade do sistema.

Sem aprovação, um recrutador não deve conseguir se apresentar publicamente como representante de uma empresa.

10.9 Recrutadores públicos da empresa

A listagem pública de recrutadores de uma empresa deve mostrar apenas vínculos aprovados.

Isso significa que recrutadores com solicitação pendente ou recusada não devem aparecer como representantes públicos da empresa.

Essa regra é importante para o frontend.

Quando uma tela pública exibe uma empresa e seus recrutadores, ela deve mostrar apenas relações confirmadas. O usuário público não precisa ver pendências administrativas.

A regra é:

Público vê apenas recrutadores com vínculo aprovado.

10.10 Publicação de oportunidade como empresa

A publicação de oportunidade em nome de empresa depende diretamente do vínculo aprovado.

No fluxo de oportunidade, existe a ideia de `publicarComoEmpresa`. Esse campo indica que o recrutador deseja publicar a vaga em nome da empresa vinculada.

Porém, esse campo não dá permissão sozinho.

A regra é:

`publicarComoEmpresa=true` só pode associar a vaga à empresa se o recrutador tiver vínculo aprovado com essa empresa.

Essa regra impede que um recrutador publique usando indevidamente o nome de uma empresa.

Se o recrutador não tiver vínculo aprovado, a publicação como empresa deve ser bloqueada ou tratada conforme a regra de negócio atual do service.

Essa decisão conecta três partes do domínio:

- recrutador autenticado;
- empresa;
- oportunidade.

10.11 Empresa e oportunidades públicas

Com empresa como entidade, passou a ser possível consultar oportunidades de uma empresa.

O endpoint público de oportunidades da empresa deve respeitar duas regras:

- a empresa precisa estar publicamente visível;

- as oportunidades também precisam estar publicamente visíveis.

Isso significa que empresa e oportunidade precisam ter localidade validada.

Não basta a oportunidade estar associada a uma empresa. Se a empresa ou a vaga estiver pendente por localidade, ela não deve aparecer em listagens públicas.

Essa regra mantém consistência na experiência pública.

10.12 Empresa e LocalidadePendente

Empresa pode ficar temporariamente pendente por localidade.

Isso acontece quando a localidade informada no cadastro ou atualização não pode ser resolvida com segurança. Nesse caso, a empresa é salva internamente, uma pendência é registrada, e o recurso fica oculto publicamente.

Essa decisão evita dois extremos ruins:

- bloquear todo cadastro por falha ou ambiguidade de localidade;
- publicar empresa com localização incerta.

O comportamento esperado é:

- a empresa existe no sistema;
- a localidade fica pendente;
- o público não vê a empresa;
- fluxos internos podem mostrar status ao usuário;
- o reproprocessamento pode resolver a pendência posteriormente.

Para o frontend, esse fluxo precisa ser comunicado com cuidado em telas internas.

10.13 Ownership formal de Empresa

Uma decisão importante do MVP foi não implementar ainda ownership formal de empresa.

Hoje, a API possui vínculo entre recrutador e empresa, com aprovação administrativa. Porém, ainda não existe uma modelagem completa de dono da empresa, administradores da empresa, conta empresarial ou login por CNPJ.

Isso foi deixado para pós-MVP porque envolve decisões maiores de produto e segurança.

Algumas perguntas ainda precisam ser respondidas futuramente:

- quem cria oficialmente uma empresa?
- quem é dono da empresa?
- uma empresa pode ter administradores?
- recrutadores aprovados podem gerenciar dados da empresa?
- haverá conta empresarial separada de usuário pessoa física?
- login por CNPJ faz sentido?
- quem aprova ou remove recrutadores vinculados?

No MVP, a solução adotada foi suficiente para permitir vínculo moderado e publicação em nome da empresa, sem abrir um modelo empresarial completo.

A regra para manutenção é:

Não assumir que recrutador vinculado é automaticamente dono da empresa.

10.14 Diferença entre vínculo e ownership

É importante separar dois conceitos:

- vínculo;
- ownership.

Vínculo significa que um recrutador foi aprovado para representar uma empresa em determinados fluxos, como publicação de oportunidades.

Ownership significaria que um usuário ou conta possui autoridade administrativa sobre a empresa, podendo gerenciar dados, aprovar outros usuários, alterar informações sensíveis ou controlar permissões.

No MVP, existe vínculo aprovado. Não existe ownership formal completo.

Essa diferença deve ser respeitada tanto no backend quanto no frontend.

O frontend não deve apresentar recrutador vinculado como “dono” da empresa, a menos que essa regra seja implementada futuramente.

10.15 Admin no fluxo de empresa

Como o ownership formal ainda não foi implementado, o admin exerce papel importante no MVP.

O admin é responsável por aprovar ou recusar vínculos entre recrutadores e empresas.

Isso mantém controle mínimo sobre a representação empresarial.

A decisão de deixar esse fluxo em `/api/admin/**` ajuda a separar claramente operações administrativas de operações do usuário comum.

Essa separação também facilita a interface do frontend:

- usuário recrutador solicita vínculo;
- admin modera vínculo;
- público vê apenas vínculos aprovados.

10.16 Cuidados para o próximo backend

Ao evoluir empresa e recrutador, o próximo desenvolvedor deve tomar alguns cuidados.

Primeiro, não tratar empresa como simples texto novamente. Empresa agora é entidade de domínio.

Segundo, não permitir publicação em nome de empresa sem validar vínculo aprovado.

Terceiro, não expor empresas com localidade pendente em endpoints públicos.

Quarto, não confundir vínculo aprovado com ownership formal.

Quinto, se for implementar ownership, revisar cuidadosamente os endpoints de empresa, vínculo, oportunidade e admin.

Sexto, se for permitir alteração de dados da empresa por recrutadores, definir antes quais perfis têm permissão.

Sétimo, se for criar login empresarial ou conta por CNPJ, tratar isso como uma nova decisão de arquitetura, não como simples campo adicional.

10.17 O que o frontend deve entender

Para a pessoa responsável pelo frontend, os pontos principais são:

- empresas públicas são apenas empresas com localidade validada;
- empresa pode existir internamente e ainda não aparecer publicamente;
- recrutador pode solicitar vínculo com empresa;
- vínculo precisa ser aprovado por admin;
- apenas vínculos aprovados devem ser mostrados publicamente;
- `publicarComoEmpresa` depende de vínculo aprovado;
- recrutador vinculado não é necessariamente dono da empresa;
- telas admin são necessárias para aprovar ou recusar vínculos;
- tela interna pode precisar mostrar status de localidade da empresa.

Também é importante que o frontend não trate ausência de empresa na listagem pública como erro automático após cadastro. Se a localidade estiver pendente, a empresa pode não aparecer publicamente até resolução.

10.18 Síntese

A evolução de Empresa e Recrutador transformou um modelo textual e pouco controlado em uma estrutura mais coerente com o produto.

Empresa deixou de ser uma `String` e passou a ser entidade. Recrutador deixou de ser uma entidade desconectada da autenticação e passou a ser perfil de usuário. O vínculo entre recrutador e empresa passou a ter aprovação administrativa. A

publicação de oportunidade como empresa passou a depender desse vínculo aprovado. A visibilidade pública da empresa passou a depender de localidade validada.

A principal regra é:

Empresa é um recurso real do domínio, recrutador é um perfil do usuário autenticado, e a atuação em nome da empresa depende de vínculo aprovado.

Essa regra deve orientar qualquer manutenção futura nos módulos de empresa, recrutador e oportunidade.

11. Oportunidade de emprego

O módulo de **Oportunidade de Emprego** é um dos principais módulos da API **Quero Trabalhar Backend**, porque representa o recurso central que conecta recrutadores, empresas e candidatos.

No estado inicial, oportunidade já existia como entidade, mas ainda era tratada de forma simples. A vaga tinha descrição, tipo de emprego, modalidade, cidade, estado e empresa como texto, mas não possuía um dono de negócio bem definido. Também não havia vínculo consistente com recrutador autenticado, nem regra clara para publicação em nome de empresa, nem controle de visibilidade pública por localidade validada.

No estado final, **OportunidadeDeEmprego** passou a ser um recurso muito mais estruturado. A oportunidade nasce no contexto do recrutador autenticado, pode ou não estar associada a uma empresa, depende de tipo de emprego aprovado, possui modalidade, exige localidade e obedece à regra central de visibilidade pública: **só aparece publicamente quando possui Localidade validada**.

11.1 Problema inicial da oportunidade

No estado inicial, o módulo de oportunidades era próximo de um CRUD simples.

A entidade `OportunidadeDeEmprego` possuía campos básicos, como descrição, tipo de emprego, modalidade, cidade, estado e empresa. Porém, empresa e localidade eram informações textuais, e a oportunidade não estava associada de forma segura a um recrutador autenticado.

Isso gerava problemas importantes:

- não havia dono claro da oportunidade;
- não havia garantia de que apenas o recrutador responsável poderia editar ou excluir a vaga;
- empresa era apenas texto;
- cidade e estado eram apenas texto;
- não havia validação confiável de localidade;
- não havia publicação controlada em nome de empresa;
- não havia filtros e paginação;
- não havia distinção clara entre listagem pública e listagem interna do recrutador;
- não havia proteção suficiente contra manipulação indevida por ID.

Esse modelo era suficiente para uma primeira versão exploratória, mas não era adequado para um MVP mais consistente.

11.2 Oportunidade no estado final

No estado final, a oportunidade passou a ter regras de negócio mais claras.

A principal mudança foi que a oportunidade passou a ser criada no contexto do recrutador autenticado. Isso significa que o cliente não informa qual recrutador está criando a vaga. O backend identifica o recrutador a partir do token JWT.

A regra consolidada é:

Toda oportunidade criada pela API deve estar associada ao recrutador autenticado.

Essa decisão resolve o problema de ownership da vaga. A oportunidade passa a ter um responsável claro dentro do domínio.

Além disso, a oportunidade passou a se relacionar com outros conceitos importantes:

- `PerfilRecrutador`;
- `Empresa`, quando publicada em nome de empresa;
- `TipoDeEmprego`;
- `Modalidade`;
- `Localidade`;
- candidatos interessados.

Com isso, oportunidade deixou de ser apenas uma entidade de vaga e passou a representar um fluxo de publicação, visibilidade e interação entre usuários.

11.3 Recrutador autenticado como dono da oportunidade

Uma das decisões mais importantes foi impedir que o cliente controle diretamente o `recrutadorId`.

No estado final, o recrutador é obtido a partir do usuário autenticado. Isso evita que alguém tente criar, editar ou remover oportunidade em nome de outro recrutador.

Essa decisão segue o mesmo princípio usado nos endpoints `/me`:

A identidade sensível vem do token, não do corpo da requisição nem de um ID enviado pelo cliente.

Essa regra deve ser preservada em qualquer evolução do módulo.

Se futuramente forem criados novos endpoints para duplicar vaga, arquivar vaga, republicar vaga, pausar vaga ou alterar status, eles também devem validar o recrutador autenticado e o ownership da oportunidade.

11.4 Criação de oportunidade

A criação de oportunidade envolve várias regras de negócio.

O recrutador precisa estar autenticado e possuir perfil de recrutador. A vaga precisa ter uma descrição, um tipo de emprego válido, uma modalidade e uma localidade. Dependendo da forma como a localidade é informada, ela pode ser validada imediatamente ou ficar pendente.

No estado final, o tipo de emprego também passou a ter regra de curadoria. Isso significa que a oportunidade não deve usar um tipo de emprego que ainda não foi aprovado.

A regra é:

Oportunidade deve usar TipoDeEmprego existente e aprovado.

Essa decisão impede que vagas sejam publicadas com categorias ainda pendentes de moderação.

Além disso, a criação de oportunidade pode envolver a empresa, quando o recrutador deseja publicar em nome de uma empresa vinculada. Esse comportamento é controlado pela regra de `publicarComoEmpresa`.

11.5 Publicar como empresa

O campo `publicarComoEmpresa` representa uma decisão de negócio importante.

Ele indica que o recrutador deseja publicar a oportunidade em nome da empresa à qual está vinculado. Porém, esse campo não é uma permissão automática.

A regra consolidada é:

`publicarComoEmpresa=true` só pode associar a vaga à empresa se o recrutador tiver vínculo aprovado com essa empresa.

Essa regra existe para evitar que um recrutador publique vagas usando indevidamente o nome de uma empresa.

No MVP, o vínculo entre recrutador e empresa é moderado por admin. Portanto, para publicar como empresa, não basta o recrutador informar a empresa. Ele precisa ter vínculo aprovado.

Se o recrutador não tiver vínculo aprovado, a API deve impedir ou tratar conforme a regra de negócio definida no service.

Essa decisão conecta os módulos de oportunidade, recrutador e empresa.

11.6 Oportunidade pessoal e oportunidade de empresa

No domínio final, uma oportunidade pode ser entendida em dois contextos:

- vaga publicada pelo recrutador em nome próprio;
- vaga publicada pelo recrutador em nome de uma empresa vinculada.

Essa distinção é importante para o frontend e para o backend.

Uma vaga pessoal pertence ao recrutador e não aparece como vaga de empresa. Já uma vaga publicada como empresa pode aparecer nas listagens relacionadas à empresa, desde que a empresa e a vaga estejam publicamente válidas.

A regra principal é:

Toda oportunidade tem recrutador responsável. Algumas oportunidades também possuem empresa associada.

Isso significa que mesmo uma vaga publicada como empresa não deixa de ter recrutador responsável. A empresa representa o contexto de publicação; o recrutador continua sendo o usuário que realizou a ação.

11.7 Atualização de oportunidade

A atualização de oportunidade deve preservar as regras de ownership.

O recrutador só deve conseguir atualizar oportunidades que pertencem a ele. Esse controle não deve depender do frontend, mas da validação no backend.

Outro ponto importante é que a atualização não deve alterar de forma descuidada o contexto original de publicação.

Durante a refatoração, foi registrado que a atualização preserva o contexto original de publicação, não convertendo livremente uma vaga pessoal em vaga de empresa ou uma vaga de empresa em vaga pessoal sem regra explícita.

Essa cautela existe porque mudar o contexto de publicação envolve permissões diferentes. Uma vaga em nome de empresa depende de vínculo aprovado; logo, essa mudança não pode ser tratada como simples alteração de campo.

11.8 Exclusão de oportunidade

A exclusão de oportunidade também precisa respeitar o dono da vaga.

A regra é:

Apenas o recrutador responsável deve conseguir excluir sua própria oportunidade, salvo fluxo administrativo específico.

Além disso, a exclusão precisa lidar com relações associadas, especialmente interesses de candidatos na vaga.

O diagnóstico final registra que a remoção valida o dono e limpa interesses associados.

Isso é importante porque uma vaga pode estar relacionada a candidatos interessados. Remover a oportunidade sem considerar essas relações poderia deixar inconsistências.

11.9 Oportunidade e TipoDeEmprego

Toda oportunidade pertence a um tipo de emprego.

No estado inicial, **TipoDeEmprego** já existia, mas no estado final ele passou a funcionar como catálogo moderado.

Isso impacta diretamente oportunidade.

A regra é:

Oportunidade só deve usar TipoDeEmprego aprovado.

Essa regra evita que sugestões ainda não revisadas apareçam como categorias públicas de vagas.

Para o frontend, isso significa que o formulário de criação de oportunidade deve consumir o catálogo público de tipos aprovados. Tipos sugeridos por usuários passam por fluxo de moderação antes de serem usados em vagas públicas.

11.10 Oportunidade e Modalidade

A oportunidade possui uma modalidade de trabalho.

No domínio final, as modalidades consideradas são:

- remoto;
- presencial;
- híbrido.

A modalidade é importante para filtro, busca e apresentação no frontend.

Ela aparece em listagens públicas e internas, e pode ser usada como filtro em endpoints paginados.

A regra de manutenção é simples: se novas modalidades forem adicionadas no futuro, isso deve ser tratado como alteração de contrato, documentação, filtros e frontend. Não deve ser apenas uma inclusão isolada no enum sem revisar os impactos.

11.11 Oportunidade e Localidade

A relação entre oportunidade e localidade é central.

No estado inicial, oportunidade tinha cidade e estado como texto. No estado final, ela depende do fluxo de Localidade.

A oportunidade pode receber localidade por IDs estruturados ou por texto livre. Quando a localidade é validada, a vaga pode aparecer publicamente. Quando a localidade fica pendente, a vaga é salva internamente, mas não aparece em listagens públicas.

A regra consolidada é:

Oportunidade pública exige Localidade validada.

Essa regra afeta:

- listagem pública de oportunidades;
- consulta pública de oportunidade por ID;
- listagem pública de oportunidades por empresa;
- filtros públicos por país, estado e cidade.

Se a oportunidade estiver com **LocalidadePendente**, ela não deve aparecer nesses endpoints públicos.

11.12 Oportunidade com LocalidadePendente

Quando a localidade da oportunidade não pode ser resolvida com segurança, a API pode criar uma **LocalidadePendente**.

Isso não significa que a criação da vaga falhou completamente.

Significa que a vaga foi salva internamente, mas ainda não está apta para publicação pública.

O comportamento esperado é:

- a vaga existe internamente;
- a pendência registra o texto original e o motivo;
- a vaga não aparece em endpoints públicos;
- o recrutador pode visualizá-la em **/me**;
- o reprocessamento pode tentar resolver a localidade depois;
- quando resolvida, a vaga pode passar a aparecer publicamente.

Essa decisão equilibra experiência de cadastro e qualidade pública.

Para o frontend, isso é essencial. Se o recrutador cria uma vaga e ela não aparece na página pública, a interface deve considerar a possibilidade de localidade pendente.

11.13 DTO público e DTO interno de oportunidade

A oportunidade possui representações diferentes dependendo do contexto.

O DTO público é usado em endpoints acessíveis ao público. Ele deve mostrar apenas dados publicáveis e não deve expor detalhes técnicos de pendência.

O DTO interno é usado em endpoints autenticados, como painel do recrutador ou interesses do candidato. Esse DTO pode expor informações adicionais, incluindo status de localidade, texto original e motivo da pendência.

A regra é:

DTO público não expõe LocalidadePendente. DTO interno pode expor status de localidade para apoiar o dono do recurso.

Essa separação é importante para evitar vazamento de detalhes técnicos em telas públicas e, ao mesmo tempo, permitir que o usuário autenticado entenda o estado do próprio recurso.

11.14 Listagem pública de oportunidades

A listagem pública de oportunidades é um dos endpoints mais importantes para o frontend.

Ela deve retornar apenas oportunidades publicamente válidas.

Isso significa que a vaga precisa:

- existir;
- estar associada a tipo de emprego aprovado;
- ter localidade validada;
- passar pelos filtros informados;
- respeitar paginação.

A listagem pública não deve retornar vagas com localidade pendente.

Essa regra deve estar protegida em service/specification, não apenas no frontend.

Para o frontend, a listagem pública deve ser tratada como catálogo de vagas disponíveis para navegação geral.

11.15 Consulta pública de oportunidade por ID

A consulta pública de uma oportunidade também deve respeitar visibilidade.

Mesmo que o ID exista no banco, a API não deve necessariamente retornar a oportunidade no endpoint público se ela ainda não estiver publicamente válida.

Isso é importante para evitar que alguém acesse uma vaga pendente diretamente por URL.

A regra é:

Existir no banco não significa estar disponível publicamente.

Essa lógica vale para oportunidade e também para empresa.

11.16 Oportunidades da empresa

O endpoint de oportunidades de uma empresa também precisa respeitar regras duplas.

A empresa precisa estar pública e a oportunidade também precisa estar pública.

Isso significa que:

- a empresa deve ter Localidade validada;
- a oportunidade deve ter Localidade validada;
- a oportunidade deve pertencer à empresa do path;
- o cliente não controla `empresaId` por query param;
- filtros adicionais podem ser aplicados, mas sem escapar do escopo da empresa.

Durante a implementação da paginação, foi criado um DTO específico de filtro para oportunidades da empresa sem `empresaId`. Isso evita que o cliente tente manipular o escopo da empresa por query param.

A regra é:

Em `/api/empresas/{id}/oportunidades`, o `id` da empresa vem do path e prevalece sobre qualquer tentativa de filtro externo.

11.17 Oportunidades do recrutador autenticado

O endpoint de oportunidades do recrutador autenticado é diferente da listagem pública.

Ele pertence ao contexto `/me`.

Por isso, deve retornar oportunidades do recrutador autenticado, inclusive aquelas que ainda estão pendentes por localidade.

A regra é:

O recrutador deve conseguir ver suas próprias oportunidades, mesmo que ainda não estejam públicas.

Esse endpoint também pode permitir filtro por `statusLocalidade`, porque esse filtro faz sentido no painel interno.

Possíveis interpretações:

- **VALIDADA**: vaga já pode aparecer publicamente;
- **PENDENTE**: vaga existe, mas ainda depende de resolução de localidade.

Essa distinção ajuda o frontend a construir uma tela de acompanhamento para o recrutador.

11.18 Oportunidades de interesse do candidato

O candidato também pode ver oportunidades em que demonstrou interesse.

Esse endpoint foi migrado para paginação e filtros. Ele opera no contexto do candidato autenticado.

A regra é:

O candidato só lista as vagas de interesse do próprio perfil.

Essa listagem pode incluir informações internas quando fizer sentido, incluindo status de localidade. Isso acontece porque o interesse foi registrado em contexto autenticado, e a API precisa permitir que o usuário veja sua própria relação com a vaga.

Ao mesmo tempo, isso não muda a regra pública. Uma vaga pendente não passa a aparecer na listagem pública apenas porque um candidato demonstrou interesse nela.

11.19 Interesse em oportunidade

O interesse em oportunidade conecta candidato e vaga.

As regras principais são:

- candidato autenticado pode demonstrar interesse;
- duplicidade de interesse é bloqueada;
- candidato pode remover interesse;
- remoção de interesse inexistente gera regra de negócio;
- a ação é baseada no perfil candidato do usuário autenticado.

Essa modelagem evita depender de `usuarioId` vindo do cliente.

Para o frontend, o fluxo deve ser simples: o usuário autenticado clica em demonstrar interesse em uma vaga; a API usa o token para identificar o candidato.

11.20 Paginação e filtros em oportunidade

O módulo de oportunidade foi um dos principais beneficiados pela paginação e filtros.

Listagens importantes passaram a usar `Page`, `Pageable`, `FilterDTO` e `Specification`.

Os filtros podem incluir, conforme o endpoint:

- termo;
- tipo de emprego;
- empresa;
- recrutador;
- país;
- estado;
- cidade;
- modalidade;
- status de localidade em contextos internos.

A diferença importante é que filtros públicos operam apenas sobre recursos publicamente válidos. Filtros internos podem trabalhar com status de localidade.

A regra para o backend é:

Não fazer paginação em memória quando a consulta pode ser feita no banco.

A regra para o frontend é:

Usar query params de paginação e filtros, em vez de buscar tudo e filtrar no cliente.

11.21 Oportunidade e frontend

Para o frontend, o módulo de oportunidade deve ser entendido em três visões.

A primeira é a visão pública. Ela mostra oportunidades disponíveis para qualquer pessoa, apenas com localidade validada.

A segunda é a visão do recrutador. Ela mostra oportunidades criadas pelo recrutador autenticado, inclusive pendentes.

A terceira é a visão do candidato. Ela mostra oportunidades públicas e também as vagas em que o candidato demonstrou interesse, conforme os endpoints disponíveis.

O frontend precisa tratar corretamente a diferença entre essas visões.

Uma vaga que aparece no painel do recrutador pode não aparecer no público. Isso não é necessariamente erro. Pode ser regra de localidade pendente.

11.22 Cuidados para o próximo backend

Ao evoluir oportunidade, o próximo desenvolvedor deve preservar algumas regras.

Primeiro, não permitir que o cliente controle `recrutadorId`.

Segundo, manter validação de ownership em atualização e exclusão.

Terceiro, não expor oportunidades com localidade pendente em endpoints públicos.

Quarto, preservar separação entre DTO público e DTO interno.

Quinto, validar tipo de emprego aprovado.

Sexto, manter publicação como empresa dependente de vínculo aprovado.

Sétimo, não permitir que filtros de empresa por query escapem do escopo do path em `/api/empresas/{id}/oportunidades`.

O módulo de oportunidade é central para o produto, então mudanças nele devem ser feitas com cuidado.

11.23 Síntese

A evolução de oportunidade transformou um CRUD inicial de vagas em um módulo de negócio mais consistente.

A oportunidade passou a ter dono, contexto de publicação, vínculo opcional com empresa, tipo de emprego aprovado, modalidade, localidade validada ou pendente, visibilidade pública controlada, filtros, paginação e relação com candidatos interessados.

A regra mais importante é:

Oportunidade nasce no contexto do recrutador autenticado e só aparece publicamente quando possui Localidade validada.

Essa regra conecta segurança, domínio, localidade e frontend. Qualquer evolução futura do módulo deve preservá-la.

12. Candidato, experiências, interesses e indicações

Os módulos de **candidato**, **experiências profissionais**, **interesses em vagas** e **indicações** representam a parte da API voltada à pessoa que busca oportunidades de trabalho e à construção de reputação dentro da plataforma.

No estado inicial, esses conceitos já existiam de alguma forma, mas ainda estavam pouco organizados. O candidato era representado diretamente por **Usuario**, experiências profissionais tinham CRUD parcial, interesses em oportunidades dependiam de IDs enviados pelo cliente, e indicações ainda eram simples demais, sem regras fortes de autoria, destinatário e ownership.

No estado final, esses fluxos foram reorganizados em torno do usuário autenticado, principalmente por meio de endpoints **/me**. Essa decisão foi importante para reduzir risco de IDOR, simplificar o frontend e garantir que cada usuário manipule apenas seus próprios dados.

A regra geral desses módulos é:

Fluxos pessoais do candidato devem ser executados no contexto do usuário autenticado, e não a partir de IDs de usuário enviados pelo cliente.

12.1 Evolução do candidato

No estado inicial, o candidato era basicamente o próprio **Usuario**. Isso funcionava para uma primeira versão, mas limitava a evolução do domínio. Como o sistema também precisava permitir que um usuário pudesse atuar como recrutador, manter todas as responsabilidades diretamente em **Usuario** deixaria a entidade crescer demais e misturaria papéis diferentes.

No estado final, o papel de candidato passou a ser representado por **PerfilCandidato**.

Essa mudança separa identidade de papel de negócio.

Usuario representa a pessoa autenticável. **PerfilCandidato** representa o papel dessa pessoa enquanto candidata a oportunidades.

Essa decisão permite que o mesmo usuário tenha perfil de candidato, perfil de recrutador ou ambos. Isso é importante porque uma pessoa pode procurar oportunidades e, em outro contexto, também publicar vagas ou atuar como recrutadora.

A regra consolidada é:

Usuário autentica. PerfilCandidato define as capacidades de candidato.

12.2 PerfilCandidato como centro dos fluxos pessoais

O **PerfilCandidato** passou a concentrar os fluxos relacionados à pessoa que busca trabalho.

Entre esses fluxos estão:

- demonstrar interesse em vagas;
- remover interesse em vagas;
- listar vagas de interesse;
- cadastrar experiências profissionais;
- atualizar experiências profissionais;
- remover experiências profissionais;
- manter base para preferências de empregabilidade;
- participar de fluxos de indicação.

Essa separação deixa o domínio mais claro. Nem tudo que pertence a **Usuario** é necessariamente parte do papel de candidato. Dados de autenticação, senha, e-mail e telefone pertencem ao usuário. Experiências e interesses pertencem ao perfil de candidato.

Essa diferença deve ser preservada em futuras evoluções.

12.3 Endpoints `/me` nos fluxos de candidato

A principal mudança arquitetural nesses fluxos foi o uso de endpoints `/me`.

No estado inicial, algumas ações dependiam de `usuarioId` no path. Isso criava risco de um usuário tentar manipular dados de outro usuário apenas alterando o ID na URL.

No estado final, os fluxos pessoais foram reorganizados para derivar o usuário a partir do token JWT.

Isso significa que o frontend não deve informar qual candidato está executando uma ação pessoal. Ele deve apenas enviar o token, e o backend deve identificar o usuário autenticado.

Essa regra vale para:

- interesses em vagas;
- experiências profissionais;
- perfil candidato;
- dados pessoais do usuário;
- indicações dadas e recebidas.

A regra de manutenção é:

Se o recurso pertence ao próprio candidato, o endpoint deve usar `/me` sempre que possível.

12.4 Interesses em vagas no estado inicial

No estado inicial, o interesse em vaga existia, mas ainda de forma frágil.

O diagnóstico inicial apontou um fluxo em que o usuário demonstrava interesse em uma oportunidade usando um endpoint com `usuarioId` e `oportunidadeId`. Esse desenho criava risco de IDOR, porque um usuário autenticado poderia tentar

informar o ID de outro usuário no path. Também havia inconsistência em um endpoint de remoção, com divergência entre o nome da variável declarada na rota e o nome usado no método.

Além disso, o interesse era mais próximo de uma relação técnica do que de um fluxo de produto bem protegido.

Por isso, esse módulo precisou ser reorganizado.

12.5 Interesses em vagas no estado final

No estado final, o interesse em vagas passou a ser uma ação do candidato autenticado.

As regras principais são:

- candidato autenticado pode demonstrar interesse em uma vaga;
- candidato autenticado pode remover interesse;
- candidato pode listar as vagas em que demonstrou interesse;
- interesse duplicado é bloqueado;
- remoção de interesse inexistente gera erro de regra de negócio;
- a listagem pertence ao candidato autenticado;
- o backend não confia em `usuarioId` enviado pelo cliente.

Essa evolução tornou o fluxo mais seguro e mais simples para o frontend.

A pessoa do frontend não precisa saber o ID do usuário para demonstrar interesse. Ela precisa apenas chamar o endpoint correto com o token do usuário logado.

A regra consolidada é:

Interesse em vaga pertence ao candidato autenticado.

12.6 Listagem de vagas de interesse

A listagem de vagas de interesse foi uma das listagens migradas para paginação e filtros.

Isso foi importante porque um candidato pode demonstrar interesse em várias vagas ao longo do tempo. Retornar tudo como `List` poderia funcionar no início, mas não seria ideal para uma tela real de frontend.

No estado final, a listagem usa paginação, filtros e escopo pelo candidato autenticado.

Ela pode permitir filtros como:

- termo;
- tipo de emprego;
- empresa;
- recrutador;
- país;
- estado;
- cidade;
- modalidade;
- status de localidade, quando aplicável ao contexto interno.

O ponto mais importante é que a listagem não busca interesses de qualquer usuário. Ela busca interesses do candidato autenticado.

Essa regra reduz risco de IDOR e torna a integração mais previsível.

12.7 Status de localidade em interesses

A listagem de vagas de interesse pode lidar com status de localidade em contexto interno.

Isso precisa ser entendido com cuidado.

Uma oportunidade com localidade pendente não aparece na listagem pública de vagas. Porém, em alguns contextos internos, o usuário autenticado pode precisar enxergar recursos relacionados ao seu próprio fluxo.

No caso do candidato, se uma vaga em que ele demonstrou interesse estiver em um estado interno específico, a API pode expor status de localidade conforme o DTO interno utilizado.

A regra geral continua sendo:

- público não vê pendente;
- `/me` pode expor informações internas quando fizer sentido;
- DTO público não expõe pendência;
- DTO interno pode trazer status para apoiar a experiência do usuário autenticado.

Para o frontend, isso significa que telas internas precisam estar preparadas para exibir estados diferentes das telas públicas.

12.8 Experiências profissionais no estado inicial

Experiência profissional já existia no estado inicial da API.

Ela era vinculada a usuário e tipo de emprego, possuía descrição e datas, mas ainda tinha limitações importantes:

- CRUD parcial;
- ausência de DTOs próprios;
- ausência de validação consistente de ownership;
- exposição direta ou próxima de entidade;
- pouca cobertura de testes;
- falta de clareza entre fluxo pessoal e fluxo administrativo.

Isso significava que a entidade já era útil, mas precisava ser reposicionada dentro do domínio.

12.9 Experiências profissionais no estado final

No estado final, experiência profissional passou a ser tratada como recurso do perfil candidato.

A regra principal é:

Experiência profissional pertence ao candidato.

Isso significa que o candidato autenticado deve conseguir gerenciar suas próprias experiências por endpoints `/me`.

As experiências são usadas para representar histórico profissional, associando:

- candidato;
- tipo de emprego;
- descrição;
- data de início;
- data de fim, quando existir.

A ausência de data final pode indicar experiência em andamento, dependendo do fluxo e dos filtros administrativos.

Essa modelagem é mais coerente porque experiência não é apenas dado solto do usuário. Ela faz parte do perfil profissional do candidato.

12.10 Gestão pessoal de experiências

O candidato deve conseguir gerenciar as próprias experiências.

Isso inclui:

- criar experiência;
- listar experiências;
- atualizar experiência;
- remover experiência.

Essas operações devem sempre usar o usuário autenticado como base.

A regra é:

Candidato não deve manipular experiência de outro candidato.

O frontend não deve construir tela de experiência pessoal baseada em `usuarioId` informado manualmente. O fluxo deve ser autenticado e baseado em `/me`.

Essa regra segue o mesmo princípio aplicado em interesses: ações pessoais pertencem ao usuário autenticado.

12.11 Gestão administrativa de experiências

Além da visão pessoal do candidato, existe também visão administrativa de experiências.

No estado final, admin pode listar, consultar, atualizar ou remover experiências globalmente conforme os endpoints disponíveis.

Isso cria duas visões diferentes:

- visão do candidato sobre suas próprias experiências;
- visão administrativa sobre experiências do sistema.

Essas visões não devem ser confundidas.

A visão do candidato é para uso pessoal. A visão admin é para gestão, suporte ou moderação.

Para o frontend, isso significa que telas de candidato e telas admin devem consumir endpoints diferentes.

Para o backend, significa que regras de ownership e regras administrativas precisam continuar separadas.

12.12 Filtros administrativos de experiências

A listagem administrativa de experiências foi migrada para paginação e filtros.

Isso permite que um admin consulte experiências de forma mais eficiente.

Os filtros incluem critérios como:

- termo;

- tipo de emprego;
- experiência em andamento;
- data de início;
- data de fim.

Essa listagem é diferente da listagem pessoal do candidato. Ela tem finalidade administrativa e pode abranger dados de vários usuários.

A regra é:

Listagem admin é global e protegida por perfil administrativo. Listagem `/me` é pessoal e protegida pelo usuário autenticado.

12.13 Indicações no estado inicial

No estado inicial, indicações existiam como uma entidade simples.

Porém, o diagnóstico apontou que a modelagem ainda não tinha clareza suficiente sobre autor, usuário indicado, vínculo com experiência e ownership. As indicações funcionavam mais como CRUD simples do que como fluxo de negócio completo.

Isso era suficiente para registrar uma mensagem, mas não para garantir regras importantes, como impedir autoindicação ou restringir exclusão ao autor.

12.14 Indicações no estado final

No estado final, indicação passou a ser tratada como ação de um usuário autenticado.

As regras principais são:

- usuário autenticado pode indicar outro usuário;
- autoindicação é proibida;
- usuário pode listar indicações dadas;
- usuário pode listar indicações recebidas;
- exclusão deve respeitar autoria;
- listas de indicações continuam como `List` no MVP.

Essa modelagem organiza melhor o fluxo.

A indicação não é apenas uma entidade isolada. Ela representa uma relação entre quem indica e quem recebe a indicação.

A regra central é:

Indicação tem autor e destinatário; o autor deve ser respeitado nas ações sensíveis.

12.15 Por que indicações continuam como List

Diferente de oportunidades, empresas e listagens admin, as listagens de indicações foram mantidas como **List** no MVP.

Essa foi uma decisão de escopo.

As indicações dadas e recebidas são listas pessoais, com volume esperado menor do que listagens públicas de vagas ou empresas. Por isso, a paginação não foi priorizada nessa primeira onda.

Isso não significa que paginação nunca será necessária. Significa apenas que, no MVP, ela não era crítica.

Se futuramente o volume crescer, esses endpoints podem ser migrados para **Page**, seguindo o mesmo padrão já adotado em outros módulos.

12.16 Autoindicação

A regra de autoindicação é importante para manter coerência do domínio.

Um usuário não deve indicar a si mesmo.

Essa validação evita registros sem valor real e preserva o sentido da funcionalidade.

A regra é:

Autor e destinatário da indicação não podem ser o mesmo usuário.

Esse tipo de regra deve ficar no service, não apenas no frontend. O frontend pode prevenir a ação na interface, mas o backend deve continuar validando.

12.17 Exclusão de indicação

A exclusão de indicação deve respeitar autoria.

A regra é:

Apenas o autor da indicação deve poder excluí-la, salvo fluxo administrativo futuro.

Isso impede que um usuário remova indicações recebidas apenas porque não gostou delas ou que manipule indicações dadas por outra pessoa.

Se futuramente for criado um fluxo de moderação admin para indicações, ele deve ser tratado como regra separada, provavelmente em namespace administrativo.

12.18 Preferências do candidato

A entidade de preferência aparece no domínio como base para interesses de empregabilidade.

Ela representa informações como tipos de trabalho de interesse, disponibilidade remota e localidades de interesse.

No estado inicial, já havia uma entidade relacionada a preferências, mas sem controller claro.

No estado final, preferências continuam como base de domínio, mas não foram consolidadas como fluxo HTTP completo do MVP.

Isso deve ser interpretado como decisão de escopo.

O MVP priorizou:

- oportunidades;
- empresas;
- localidade;

- experiências;
- interesses em vagas;
- indicações;
- perfis;
- autenticação;
- paginação;
- documentação.

Preferências podem evoluir futuramente para melhorar recomendação, busca, match entre candidato e vaga ou filtros personalizados.

12.19 Relação com TipoDeEmprego

Experiências e preferências se relacionam com **TipoDeEmprego**.

Essa relação é importante porque o tipo de emprego funciona como catálogo moderado.

No caso das experiências, associar experiência a um tipo de emprego permite organizar o histórico profissional do candidato em categorias mais consistentes.

No caso de preferências, a relação com tipo de emprego permite indicar áreas de interesse do candidato.

A regra de manutenção é que o catálogo de tipos de emprego precisa continuar controlado, porque ele afeta vários módulos ao mesmo tempo: oportunidades, experiências, preferências e filtros.

12.20 Relação com Localidade

Candidato, interesses e preferências também se relacionam com localidade, mesmo que de forma diferente de empresa e oportunidade.

Para empresas e oportunidades, localidade controla visibilidade pública.

Para candidato e preferências, localidade pode ser usada futuramente para indicar onde o candidato tem interesse em trabalhar.

No estado final, o módulo de Localidade já cria a base necessária para essa evolução. Em vez de preferências por cidade/estado como texto livre, o sistema pode caminhar para referências estruturadas.

Essa evolução deve respeitar a regra geral:

Localidade validada deve ser a fonte oficial para filtros e buscas confiáveis.

12.21 O que o frontend deve entender

Para o frontend, esses módulos devem ser entendidos como fluxos pessoais do usuário autenticado.

O painel do candidato deve consumir endpoints `/me` para:

- dados do usuário;
- perfil candidato;
- experiências;
- vagas de interesse;
- indicações dadas;
- indicações recebidas.

O frontend não deve exigir que o usuário informe ou manipule seu próprio ID para essas ações.

Também deve tratar erros de regra de negócio, como:

- interesse duplicado;
- tentativa de remover interesse inexistente;
- autoindicação;
- exclusão de indicação sem autoria;
- ausência de perfil candidato quando a ação exigir esse perfil.

As mensagens do backend devem ser usadas para orientar o usuário sempre que possível.

12.22 Cuidados para o próximo backend

Ao evoluir esses módulos, o próximo desenvolvedor deve preservar alguns cuidados.

Primeiro, manter os fluxos pessoais baseados em `/me`.

Segundo, não permitir que candidato manipule experiências, interesses ou indicações de outro usuário.

Terceiro, manter validações de duplicidade e ownership nos services.

Quarto, não transformar listas pessoais em endpoints admin sem separar contexto.

Quinto, se indicações forem paginadas no futuro, seguir o padrão de `FilterDTO`, `Pageable` e `Specification` quando fizer sentido.

Sexto, se preferências forem expostas como fluxo HTTP, usar localidade validada e tipos de emprego aprovados sempre que possível.

Sétimo, manter DTOs específicos para request e response, evitando expor entidades.

12.23 Síntese

A evolução dos módulos de candidato, experiências, interesses e indicações teve como foco organizar fluxos pessoais em torno do usuário autenticado.

O candidato deixou de ser apenas o próprio `Usuario` e passou a ser representado por um perfil específico. Interesses em vagas passaram a ser ações do candidato autenticado. Experiências profissionais passaram a pertencer ao perfil candidato. Indicações passaram a respeitar autoria, destinatário e proibição de autoindicação.

A regra mais importante é:

Fluxos pessoais do candidato devem usar o usuário autenticado como escopo, não IDs de usuário enviados pelo cliente.

Essa regra conecta segurança, domínio e integração com frontend. Ela deve ser preservada em qualquer evolução futura desses módulos.

13. Tipo de emprego e curadoria

O módulo de **Tipo de Emprego** representa o catálogo de áreas, cargos ou categorias de trabalho que podem ser usadas em oportunidades, experiências profissionais, preferências e filtros da plataforma.

No estado inicial, **TipoDeEmprego** já existia como entidade, com título e descrição. Esse foi um ponto positivo, porque o projeto já reconhecia que oportunidades e experiências precisavam estar associadas a algum tipo/categoria de trabalho. Porém, ainda faltava uma regra mais clara de curadoria, aprovação e controle do catálogo.

No estado final, **TipoDeEmprego** passou a funcionar como um **catálogo moderado**. Isso significa que nem todo tipo sugerido entra automaticamente no catálogo público. A API passou a diferenciar tipos aprovados, que podem ser usados publicamente, de tipos sugeridos ou pendentes, que precisam passar por moderação administrativa.

A regra central é:

Tipos de emprego aprovados formam o catálogo público. Tipos sugeridos por usuários entram como pendentes de aprovação.

13.1 Problema inicial do catálogo

No estado inicial, o tipo de emprego existia, mas ainda era tratado de forma mais simples. Ele tinha título único e descrição, o que já ajudava a organizar oportunidades e experiências.

Porém, sem uma curadoria clara, o catálogo poderia crescer de forma desorganizada.

Em um sistema real, se qualquer usuário puder criar livremente tipos de emprego que passam a ser usados publicamente, alguns problemas aparecem rapidamente:

- categorias duplicadas;
- nomes muito parecidos;
- erros de digitação;
- termos inadequados;
- categorias genéricas demais;
- categorias específicas demais;
- dificuldade de filtro no frontend;
- perda de qualidade no catálogo público.

Por exemplo, sem curadoria, poderiam existir ao mesmo tempo “Desenvolvedor Java”, “Dev Java”, “Programador Java”, “Java Backend” e “Backend Java” como tipos separados, mesmo que representem categorias muito próximas.

A curadoria foi criada para evitar esse tipo de fragmentação.

13.2 TipoDeEmprego como catálogo moderado

No estado final, **TipoDeEmprego** passou a ser tratado como um catálogo moderado.

Isso significa que existem dois grandes estados de uso:

- tipos aprovados;
- tipos não aprovados.

Tipos aprovados fazem parte do catálogo público. Eles podem aparecer para o frontend em seletores, filtros e formulários de criação de oportunidade ou experiência.

Tipos não aprovados representam sugestões ou itens pendentes de curadoria. Eles ainda não devem ser usados como parte do catálogo público normal.

Essa separação ajuda a manter consistência no produto.

A regra é:

O frontend público deve consumir apenas tipos de emprego aprovados.

13.3 Tipos criados por administrador

O administrador tem permissão para criar tipos de emprego diretamente como parte do catálogo oficial.

Essa regra existe porque o admin representa o papel de curadoria do sistema. Quando o admin cria um tipo, a API entende que aquele tipo já foi validado como adequado para o catálogo.

Esse fluxo é útil para popular ou corrigir o catálogo com categorias confiáveis.

Exemplos de uso administrativo:

- criar um novo tipo de emprego aprovado;
- ajustar título e descrição;
- remover tipo não utilizado;
- aprovar sugestões de usuários;
- organizar categorias antes de serem exibidas publicamente.

A regra é:

Tipos criados por admin podem entrar diretamente como aprovados, conforme o fluxo administrativo definido pela API.

13.4 Tipos sugeridos por usuários

Usuários também podem sugerir novos tipos de emprego.

Essa funcionalidade existe porque o catálogo pode não cobrir todas as possibilidades do mundo real. Em vez de bloquear completamente o usuário quando uma categoria não existe, a API permite registrar uma sugestão.

Porém, essa sugestão não entra automaticamente no catálogo público.

A regra é:

Tipo sugerido pelo usuário nasce como não aprovado.

Essa decisão equilibra flexibilidade e controle.

O usuário consegue sugerir algo que está faltando, mas o sistema preserva a qualidade do catálogo ao exigir aprovação antes de tornar o tipo disponível publicamente.

13.5 Catálogo público de tipos aprovados

O endpoint público de tipos aprovados representa a lista que o frontend deve usar em telas comuns.

Esse catálogo pode ser usado em:

- filtros de oportunidades;
- formulário de criação de oportunidade;
- formulário de experiência profissional;
- telas de busca;
- campos de seleção relacionados a áreas de trabalho.

Como esse endpoint é público, ele deve retornar apenas tipos aprovados.

A regra é:

O catálogo público não deve retornar tipos não aprovados.

Isso evita que sugestões pendentes apareçam para usuários antes da validação administrativa.

Para o frontend, essa regra simplifica a integração. A tela que precisa de opções confiáveis deve usar o endpoint de aprovados.

13.6 Tipos não aprovados e moderação administrativa

Tipos não aprovados pertencem ao fluxo administrativo.

Eles representam itens que ainda precisam ser revisados antes de entrar no catálogo público.

No estado final, a API possui endpoint administrativo para listar tipos não aprovados com paginação e filtro textual. Isso permite que um admin revise sugestões de forma organizada.

A listagem administrativa de não aprovados segue a mesma lógica das outras listagens críticas do MVP:

- retorna `Page`;
- aceita filtro textual;
- possui ordenação padrão;
- exige permissão administrativa;
- força a regra `aprovado=false` no backend.

Um ponto importante é que o cliente não controla o filtro `aprovado`. O endpoint de não aprovados sempre busca não aprovados.

A regra é:

Em `/api/admin/tipos-emprego/nao-aprovados`, o backend impõe `aprovado=false`. O frontend não deve tentar controlar esse estado por query param.

13.7 Aprovação individual

O administrador pode aprovar um tipo de emprego individualmente.

Esse fluxo permite revisar uma sugestão e torná-la parte do catálogo oficial.

Durante a aprovação, pode fazer sentido corrigir título ou descrição, dependendo do comportamento atual do endpoint. Isso é importante porque uma sugestão enviada pelo usuário pode conter erro, nome pouco claro ou descrição incompleta.

A aprovação individual representa a decisão administrativa de que aquele tipo agora é adequado para uso no sistema.

A regra é:

Apenas tipos aprovados devem ficar disponíveis para uso público.

13.8 Aprovação em lote

Além da aprovação individual, a API também possui aprovação em lote.

Esse fluxo permite aprovar vários tipos de emprego de uma vez, a partir de uma lista simples de IDs.

A documentação final deixou claro que o request body desse endpoint é um array JSON simples de IDs numéricos, não uma lista de objetos.

Essa decisão mantém o contrato simples.

A aprovação em lote é útil quando o admin revisa várias sugestões e decide promovê-las para o catálogo oficial.

A regra é:

Aprovação em lote altera o estado de vários tipos pendentes para aprovados, sem mudar o formato do contrato para objetos complexos.

13.9 Tipo de emprego em oportunidades

O tipo de emprego é uma informação obrigatória na oportunidade.

No estado final, oportunidade só deve usar tipo de emprego existente e aprovado.

Essa regra é importante porque a oportunidade é um recurso público. Se uma vaga pudesse usar tipo ainda não aprovado, o catálogo público perderia o sentido.

A regra consolidada é:

Oportunidade deve usar TipoDeEmprego aprovado.

Para o frontend, isso significa que o formulário de criação de oportunidade deve consumir o catálogo de tipos aprovados, e não a lista administrativa de pendentes.

13.10 Tipo de emprego em experiências profissionais

Experiências profissionais também se relacionam com tipo de emprego.

Essa associação ajuda a organizar o histórico profissional do candidato. Em vez de deixar a experiência totalmente solta em texto livre, a API consegue relacioná-la a uma categoria do catálogo.

Isso permite futuras evoluções, como:

- filtros por área;
- recomendações de vagas;
- comparação entre experiência do candidato e tipo da oportunidade;
- preferências de trabalho;
- relatórios administrativos.

A regra recomendada é manter a experiência ligada a tipos consistentes, preferencialmente aprovados.

13.11 Tipo de emprego em preferências

A entidade de preferência também pode se relacionar com tipos de emprego.

Embora o fluxo de preferências não tenha sido consolidado como endpoint completo no MVP, o domínio já indica que tipos de emprego podem ser usados para expressar áreas de interesse do candidato.

Essa é uma base importante para evolução futura.

No futuro, preferências podem ser usadas para:

- recomendar vagas;
- filtrar oportunidades mais relevantes;
- criar alertas;
- melhorar match entre candidato e oportunidade.

Para isso funcionar bem, o catálogo de tipos precisa continuar curado.

13.12 Exclusão e integridade

A exclusão de tipos de emprego precisa respeitar integridade.

Se um tipo já está associado a oportunidades, experiências ou outros recursos, removê-lo pode quebrar o histórico ou deixar registros inconsistentes.

Por isso, o estado final prevê tratamento de integridade e bloqueio de exclusão quando há uso associado.

A regra é:

Tipo de emprego em uso não deve ser removido de forma que quebre relações existentes.

Se no futuro for necessário “desativar” tipos sem remover, isso pode ser uma evolução interessante. Em alguns sistemas, desativar é mais seguro do que excluir, porque preserva histórico.

13.13 TipoDeEmprego e frontend

Para o frontend, a regra principal é simples:

Telas públicas e formulários comuns devem usar apenas tipos aprovados.

Isso vale para:

- filtros de vagas;
- criação de oportunidade;
- cadastro de experiência;
- seleção de área de interesse, se existir futuramente.

A lista de não aprovados deve ser usada apenas em telas administrativas.

Também é importante que o frontend não permita ao usuário comum escolher diretamente um tipo pendente como se ele já fosse oficial.

Quando o usuário quiser sugerir um novo tipo, deve usar o fluxo de sugestão. Depois, a aprovação depende do admin.

13.14 TipoDeEmprego e administração

O módulo de TipoDeEmprego possui uma divisão clara entre público e admin.

O público consome o catálogo aprovado.

O admin gerencia a curadoria.

Essa separação é importante para segurança e organização do produto.

As responsabilidades administrativas incluem:

- criar tipos aprovados;
- listar tipos não aprovados;
- aprovar sugestões;
- aprovar em lote;
- excluir quando permitido;
- manter o catálogo limpo e útil.

A regra é:

Curadoria do catálogo pertence ao fluxo administrativo.

13.15 Cuidados para o próximo backend

Ao evoluir esse módulo, o próximo desenvolvedor deve preservar alguns cuidados.

Primeiro, não misturar tipos aprovados e não aprovados no catálogo público.

Segundo, não permitir que oportunidades usem tipos pendentes.

Terceiro, manter a listagem de não aprovados restrita a admin.

Quarto, preservar a regra de que `aprovado=false` no endpoint de não aprovados é imposta pelo backend.

Quinto, ter cuidado ao excluir tipos em uso.

Sexto, se adicionar novos campos ou status ao tipo de emprego, revisar endpoints públicos, endpoints admin, filtros, Swagger e frontend.

Sétimo, se quiser melhorar curadoria, considerar recursos como desativação, merge de tipos duplicados ou histórico de aprovação, mas tratar isso como evolução futura.

13.16 Possíveis evoluções futuras

O módulo de `TipoDeEmprego` já é suficiente para o MVP, mas pode evoluir.

Algumas possibilidades futuras são:

- desativar tipo em vez de excluir;
- mesclar tipos duplicados;
- registrar quem aprovou uma sugestão;
- registrar data de aprovação;
- permitir motivo de recusa;
- criar fluxo de recusa explícita;
- melhorar busca textual;
- agrupar tipos por área;
- criar hierarquia de categorias;
- usar tipos em recomendações automáticas.

Essas evoluções podem tornar o catálogo mais robusto, mas não eram essenciais para a entrega do MVP.

13.17 Síntese

A evolução de `TipoDeEmprego` transformou um cadastro simples de categorias em um catálogo moderado.

A decisão principal foi separar tipos aprovados de tipos sugeridos. Com isso, o sistema preserva flexibilidade para receber sugestões de usuários, mas mantém controle sobre o que aparece publicamente e o que pode ser usado em oportunidades.

A regra mais importante é:

O catálogo público de tipos de emprego contém apenas tipos aprovados, e oportunidades devem usar tipos aprovados.

Essa regra deve ser preservada porque impacta oportunidades, experiências, preferências, filtros, frontend e qualidade geral da plataforma.

14. Paginação e filtros

A implementação de **paginação e filtros** foi uma das etapas mais importantes para transformar a API em uma base mais próxima de um MVP real. No estado inicial, as principais listagens retornavam `List`, sem paginação, sem filtros estruturados e sem uso de `Specification`. Isso funcionava para uma base pequena, mas não era adequado para crescimento da aplicação nem para integração eficiente com o frontend.

No estado final, as listagens principais passaram a seguir um padrão consistente com `Page`, `Pageable`, `FilterDTO` e `Specification`. Essa decisão melhorou a performance, a organização das consultas e a experiência de integração com o frontend.

A regra geral consolidada foi:

Listagens principais, públicas, internas ou administrativas, devem ser paginadas e filtráveis quando houver expectativa de crescimento ou necessidade real de busca. Catálogos simples, autocompletes e listas pessoais pequenas podem continuar retornando `List` no MVP.

14.1 Problema inicial das listagens

No estado inicial, as listagens eram simples e retornavam coleções completas.

Isso acontecia em fluxos como:

- usuários;
- recrutadores;
- oportunidades;
- tipos de emprego;
- experiências;
- indicações;

- interesses.

Não havia uso de `Pageable`, `Page`, `Specification` ou filtros dinâmicos estruturados.

Esse modelo tinha algumas limitações importantes.

Primeiro, ele não escala bem. Conforme a base cresce, retornar todos os registros de uma vez pode gerar respostas grandes demais, lentidão e consumo desnecessário de memória.

Segundo, ele dificulta o frontend. Sem paginação, a tela precisa lidar com uma lista potencialmente grande. Sem filtros no backend, o frontend pode acabar tentando filtrar dados localmente, o que não é adequado para bases maiores.

Terceiro, ele reduz a qualidade da navegação. Em um sistema de empregabilidade, o usuário precisa buscar vagas, filtrar por tipo, empresa, localidade, modalidade e outros critérios. Sem filtros, a listagem se torna pouco útil.

Por isso, a paginação e os filtros foram tratados como parte importante da preparação para o MVP.

14.2 Objetivo da mudança

O objetivo da refatoração nessa frente foi padronizar as listagens mais importantes da API.

A ideia não foi paginar tudo de forma indiscriminada. A decisão foi identificar quais endpoints realmente precisavam de paginação e filtros no MVP.

Foram priorizadas listagens com maior chance de crescimento ou maior importância para telas do frontend:

- oportunidades públicas;
- empresas públicas;
- oportunidades de uma empresa;
- oportunidades do recrutador autenticado;
- vagas de interesse do candidato autenticado;

- usuários no admin;
- experiências no admin;
- tipos de emprego não aprovados no admin.

Essas listagens passaram a usar um padrão comum, facilitando manutenção e integração.

14.3 Padrão adotado

O padrão final adotado foi baseado em:

- `FilterDTO`;
- `Pageable`;
- `@PageableDefault`;
- `@ParameterObject`;
- `Page<ResponseDTO>`;
- `JpaSpecificationExecutor`;
- `Specification`.

Cada parte tem um papel.

O `FilterDTO` agrupa os filtros opcionais do endpoint. Isso evita muitos parâmetros soltos no controller e organiza melhor o contrato.

O `Pageable` recebe paginação e ordenação vindas da query string, como `page`, `size` e `sort`.

O `@PageableDefault` define um comportamento padrão quando o cliente não envia paginação ou ordenação.

O `@ParameterObject` ajuda o Swagger/OpenAPI a documentar corretamente parâmetros de filtro e paginação.

O `Page<ResponseDTO>` retorna os dados paginados junto com metadados úteis para o frontend.

O `JpaSpecificationExecutor` permite executar consultas dinâmicas.

A `Specification` centraliza filtros opcionais e, em alguns casos, regras de visibilidade.

A regra arquitetural é:

Controller recebe filtros e paginação. Service coordena a consulta. Specification monta os predicados. Repository executa. DTO define a resposta.

14.4 Por que usar Specification

`Specification` foi usada porque os filtros são opcionais e combináveis.

Em uma listagem de oportunidades, por exemplo, o usuário pode filtrar apenas por termo, ou por tipo de emprego e modalidade, ou por localidade, ou por vários critérios ao mesmo tempo.

Criar um método no repository para cada combinação seria inviável.

Com `Specification`, os filtros são montados dinamicamente de acordo com os parâmetros enviados.

Isso permite combinar critérios como:

- termo textual;
- tipo de emprego;
- empresa;
- recrutador;
- país;
- estado;
- cidade;
- modalidade;
- status de localidade em fluxos internos;
- datas;
- andamento;

- perfis;
- texto de busca.

Além disso, as specifications também foram usadas para preservar regras de negócio, não apenas filtros técnicos.

O exemplo mais importante é a regra de visibilidade pública por localidade validada.

14.5 Specification como proteção de visibilidade pública

Em endpoints públicos, a **Specification** não serve apenas para filtrar. Ela também protege a regra de que recursos pendentes não devem aparecer publicamente.

Isso vale principalmente para:

- empresas públicas;
- oportunidades públicas;
- oportunidades públicas de uma empresa.

A regra é:

Endpoints públicos devem restringir a consulta a recursos com Localidade validada.

Isso evita que uma empresa ou vaga com **LocalidadePendente** apareça apenas porque o filtro textual ou de cidade bateu.

Essa regra precisa estar no backend, não no frontend. O frontend não deve receber recursos pendentes em endpoints públicos e depois escondê-los. A API deve garantir que eles não sejam retornados.

Essa decisão é uma das ligações mais importantes entre paginação, filtros e Localidade.

14.6 Diferença entre filtros públicos e filtros internos

A API passou a diferenciar filtros públicos de filtros internos.

Filtros públicos são usados por qualquer usuário, sem autenticação, em listagens abertas. Eles devem retornar apenas recursos publicamente válidos.

Filtros internos são usados em endpoints autenticados, especialmente `/me`. Eles podem retornar recursos do próprio usuário em estados que não aparecem publicamente, como localidade pendente.

Por exemplo, a listagem pública de oportunidades não expõe `statusLocalidade`, porque público só vê oportunidades validadas. Já a listagem de oportunidades do recrutador autenticado pode aceitar `statusLocalidade`, porque o recrutador precisa filtrar as próprias vagas entre validadas e pendentes.

A regra é:

Público filtra apenas recursos publicáveis. `/me` pode filtrar recursos internos do próprio usuário.

Essa distinção deve ser preservada em novas listagens.

14.7 Endpoints paginados no estado final

No estado final, os principais endpoints paginados são:

- `GET /api/oportunidades;`
- `GET /api/empresas;`
- `GET /api/empresas/{id}/oportunidades;`
- `GET /api/recrutadores/me/oportunidades;`
- `GET /api/candidatos/me/interesses/vagas;`
- `GET /api/admin/usuarios;`
- `GET /api/admin/experiencias;`
- `GET /api/admin/tipos-emprego/nao-aprovados.`

Esses endpoints foram escolhidos porque representam listagens com maior potencial de crescimento ou com necessidade clara de busca e navegação.

O diagnóstico final confirma que esses endpoints passaram a usar [Page](#), filtros, ordenação padrão e [Specification](#), respeitando o contexto público, [/me](#) ou [admin](#).

14.8 Oportunidades públicas

A listagem pública de oportunidades é uma das mais importantes da API.

Ela permite ao frontend exibir vagas disponíveis para navegação geral.

Essa listagem é paginada e filtrável.

Os filtros incluem, conforme o contrato atual:

- termo;
- tipo de emprego;
- empresa;
- recrutador;
- país;
- estado;
- cidade;
- modalidade.

A regra mais importante é que oportunidades com localidade pendente não aparecem.

Ou seja, a consulta pública combina filtros de busca com regra de visibilidade.

A listagem pública não é apenas “todas as oportunidades”. Ela é “todas as oportunidades publicamente válidas”.

14.9 Empresas públicas

A listagem pública de empresas também foi migrada para paginação e filtros.

Ela permite buscar empresas por critérios como:

- termo;

- país;
- estado;
- cidade.

Assim como oportunidades, empresas públicas precisam ter Localidade validada.

Empresas com localidade pendente podem existir internamente, mas não devem aparecer na listagem pública.

Essa regra protege a qualidade do catálogo público de empresas.

Para o frontend, isso significa que o endpoint de empresas públicas já retorna apenas empresas prontas para exibição pública. A tela não precisa remover pendentes manualmente.

14.10 Oportunidades de uma empresa

O endpoint de oportunidades de uma empresa também foi paginado e filtrado.

Esse endpoint possui uma regra especial: o `empresaId` vem do path.

Isso significa que o cliente não deve conseguir alterar o escopo da empresa por query param.

A decisão foi criar um filtro específico para esse endpoint, sem `empresaId`, garantindo que o escopo venha da rota e seja imposto pelo backend.

A regra é:

Em `GET /api/empresas/{id}/oportunidades`, a empresa é definida pelo path. Query params não devem alterar esse escopo.

Além disso, tanto a empresa quanto as oportunidades precisam estar publicamente válidas.

Essa regra evita inconsistência e manipulação de escopo.

14.11 Oportunidades do recrutador autenticado

A listagem de oportunidades do recrutador autenticado usa paginação e filtros em contexto interno.

Ela é diferente da listagem pública.

Como pertence ao recrutador autenticado, pode retornar vagas próprias que ainda estão pendentes por localidade.

Os filtros incluem critérios como:

- termo;
- tipo de emprego;
- empresa;
- país;
- estado;
- cidade;
- modalidade;
- status de localidade.

O ponto mais importante é que o escopo do recrutador não vem do cliente. Ele vem do token.

A regra é:

O recrutador autenticado só lista suas próprias oportunidades, e essa listagem pode incluir oportunidades pendentes.

Essa listagem é importante para o painel do recrutador no frontend.

14.12 Vagas de interesse do candidato

A listagem de vagas de interesse do candidato também foi paginada e filtrada.

No estado inicial, interesses eram mais frágeis e dependiam de IDs de usuário no path. No estado final, a listagem pertence ao candidato autenticado.

Essa listagem pode combinar filtros de oportunidade com o escopo do candidato.

A regra é:

O candidato autenticado só lista as vagas em que ele demonstrou interesse.

Esse endpoint também pode usar `statusLocalidade` em contexto interno, quando fizer sentido, porque não é uma listagem pública geral.

Para o frontend, esse endpoint deve ser usado para a área do candidato, não para a página pública de vagas.

14.13 Usuários no admin

A listagem administrativa de usuários foi paginada e filtrada.

Ela é protegida por perfil admin.

Os filtros permitem buscar usuários por critérios como:

- termo;
- nome;
- e-mail;
- CPF;
- existência de perfil candidato;
- existência de perfil recrutador.

Esse endpoint é global e administrativo. Ele não deve ser confundido com `/api/usuarios/me`.

A regra é:

Usuário comum acessa seus próprios dados por `/me`. Admin consulta usuários globalmente por endpoint administrativo.

Essa separação mantém clareza de segurança e contrato.

14.14 Experiências no admin

A listagem administrativa de experiências também foi migrada para paginação e filtros.

Os filtros incluem:

- termo;
- tipo de emprego;
- experiência em andamento;
- data de início;
- data de fim.

Esse endpoint é útil para visão global administrativa.

Ele é diferente da listagem pessoal de experiências do candidato.

A regra é:

Experiências pessoais pertencem ao candidato autenticado. Experiências admin são visão global protegida por ADMIN.

14.15 Tipos de emprego não aprovados

A listagem de tipos de emprego não aprovados foi paginada e filtrada.

Esse endpoint pertence à curadoria administrativa.

O filtro textual permite buscar sugestões pendentes por título ou descrição.

A regra mais importante é que `aprovado=false` é imposto pelo backend.

O cliente não deve controlar esse estado por query param.

A regra é:

`GET /api/admin/tipos-emprego/nao-aprovados` sempre retorna tipos não aprovados, independentemente de qualquer tentativa do cliente de alterar esse escopo.

Essa decisão evita que o endpoint administrativo de pendências vire uma listagem genérica de tipos.

14.16 Endpoints que continuaram retornando List

Nem todos os endpoints foram paginados no MVP.

Alguns continuaram retornando **List** por decisão de escopo e por natureza do dado.

Exemplos:

- catálogos de localidade;
- tipos de emprego aprovados;
- experiências pessoais do candidato;
- indicações dadas e recebidas;
- recrutadores públicos de uma empresa;
- algumas listas subordinadas ou pequenas.

Essa decisão foi consciente.

Nem toda lista precisa virar **Page** imediatamente. Catálogos simples, autocompletes e listas pessoais pequenas podem continuar como **List** enquanto o volume esperado for controlado.

A regra é:

Paginar onde há risco de crescimento ou necessidade real de navegação. Manter **List** onde o retorno é pequeno, controlado ou funciona como catálogo simples.

No futuro, se o volume crescer, esses endpoints podem migrar para **Page** seguindo o padrão já adotado.

14.17 Sort padrão

Cada endpoint paginado recebeu um sort padrão coerente com o caso de uso.

Alguns exemplos:

- oportunidades geralmente usam ordenação por `id` descendente;
- empresas usam ordenação por nome;
- listagens administrativas podem usar `id` descendente ou outro critério adequado;
- tipos pendentes usam ordenação estável para revisão.

O sort padrão é importante porque garante comportamento previsível quando o cliente não envia parâmetro `sort`.

Para o frontend, isso reduz a necessidade de sempre definir ordenação manualmente.

Para o backend, ajuda a evitar respostas inconsistentes entre chamadas.

14.18 Como o frontend deve consumir Page

Endpoints paginados retornam uma estrutura de página.

O frontend deve ler principalmente:

- `content`;
- `number`;
- `size`;
- `totalElements`;
- `totalPages`;
- `first`;
- `last`;
- informações de ordenação, quando necessário.

O array de dados fica em `content`.

A navegação deve ser feita com query params como:

- `page`;

- `size`;
- `sort`.

Filtros também devem ser enviados como query params.

A orientação para o frontend é:

Não buscar tudo para filtrar no cliente quando o backend já oferece filtro e paginação.

Essa regra melhora performance e mantém a fonte da consulta no backend.

14.19 PageImpl e contrato futuro

No MVP, a API retorna `Page` diretamente, usando a serialização padrão do Spring.

Isso é aceitável para o momento, mas existe um ponto futuro: estabilizar o contrato JSON de paginação.

O próprio Spring costuma emitir aviso sobre serialização direta de `PageImpl`, indicando que o formato pode não ser ideal como contrato público de longo prazo.

Por isso, foi identificado um possível ponto pós-MVP:

Criar um contrato próprio de resposta paginada para estabilizar o JSON de paginação.

Isso não bloqueia o MVP, mas é importante para evolução futura.

Se essa mudança for feita no futuro, ela deve ser planejada com cuidado porque impacta o frontend.

14.20 Paginação e LocalidadePendente

A relação entre paginação e `LocalidadePendente` é muito importante.

Não basta paginar. É preciso paginar o conjunto correto.

Em endpoints públicos, a paginação deve acontecer sobre recursos publicamente válidos. Ou seja, recursos pendentes devem ser excluídos antes da paginação, na consulta ao banco.

Se o backend buscasse todos os registros, paginasse e depois removesse pendentes em memória, a paginação ficaria errada. Poderia retornar páginas vazias ou totais incorretos.

Por isso, a regra de localidade validada precisa estar na *Specification* ou na query, não no pós-processamento.

A regra é:

Filtrar pendências no banco antes de paginar.

Essa decisão garante que *totalElements*, *totalPages* e *content* representem corretamente o conjunto público.

14.21 Não paginar em memória

Durante a evolução, também ficou claro que algumas listagens não deveriam ser paginadas em memória.

A regra geral é:

Quando for possível consultar com paginação no banco, não buscar tudo para paginar depois.

Paginar em memória pode parecer mais simples, mas gera problemas:

- carrega dados desnecessários;
- prejudica performance;
- distorce filtros;
- dificulta contagem correta;
- não escala.

Por isso, as listagens principais foram migradas para *repository + specification + pageable*.

Essa decisão deve ser preservada em novos endpoints.

14.22 Como documentar filtros

A documentação Swagger/OpenAPI passou a refletir filtros e paginação.

Os filtros foram documentados nos `FilterDTOs`, e os controllers usam `@ParameterObject` para que o Swagger consiga exibir os parâmetros corretamente.

Isso é importante para o frontend.

A pessoa que implementa uma tela pode consultar o Swagger para entender quais filtros existem, quais tipos eles aceitam e quais endpoints retornam `Page`.

A regra de manutenção é:

Sempre que adicionar filtro novo, atualizar `FilterDTO`, `Specification`, testes e documentação Swagger.

Não basta adicionar o campo no DTO sem implementar na `Specification`. Também não basta implementar na `Specification` sem documentar.

14.23 Testes de paginação e filtros

A implementação de paginação e filtros foi acompanhada por testes.

Os testes cobrem cenários como:

- retorno paginado;
- sort padrão;
- filtros por termo;
- filtros por tipo;
- filtros por localidade;
- filtros por modalidade;
- escopo do usuário autenticado;
- escopo de empresa pelo path;
- exclusão de recursos pendentes em endpoints públicos;

- inclusão de pendentes em endpoints internos quando aplicável;
- segurança em endpoints admin.

Esses testes são importantes porque paginação e filtros podem quebrar facilmente com mudanças pequenas na query ou na Specification.

A regra para manutenção é:

Alterou filtro ou specification? Atualize ou adicione teste correspondente.

14.24 Cuidados para o próximo backend

Ao criar ou alterar endpoints paginados, o próximo desenvolvedor deve observar alguns cuidados.

Primeiro, não retornar `List` em uma listagem que pode crescer bastante.

Segundo, não paginar em memória.

Terceiro, não deixar filtros públicos acessarem pendências de localidade.

Quarto, não permitir que query param altere escopo que deveria vir do path ou do token.

Quinto, manter filtros opcionais realmente opcionais.

Sexto, ignorar strings em branco quando fizer sentido.

Sétimo, manter `@PageableDefault` coerente.

Oitavo, atualizar Swagger e testes.

Nono, preservar a separação entre filtros públicos, filtros `/me` e filtros admin.

14.25 O que o frontend deve entender

Para o frontend, as regras principais são:

- endpoints paginados retornam dados em `content`;

- metadados de paginação devem ser usados para navegação;
- filtros são enviados como query params;
- `page` começa em zero;
- `sort` segue o padrão do Spring, como `campo,direcao`;
- endpoints públicos não retornam pendentes;
- endpoints `/me` podem retornar pendentes;
- filtros de status de localidade fazem sentido apenas em telas internas;
- não é recomendado filtrar grandes listas no client-side se o backend já oferece filtro;
- ausência de recurso em listagem pública pode estar relacionada à regra de visibilidade.

Essa compreensão é essencial para evitar telas inconsistentes.

14.26 Síntese

A implementação de paginação e filtros transformou as principais listagens da API em contratos mais adequados para uso real.

A API saiu de listagens simples com `List` para um padrão mais consistente com `Page`, `Pageable`, `FilterDTO`, `Specification` e DTOs de resposta.

Mais importante do que a técnica foi a regra de domínio embutida nas consultas: endpoints públicos paginam apenas recursos publicamente válidos, enquanto endpoints internos podem exibir estados próprios do usuário autenticado.

A regra mais importante é:

Paginação e filtros não são apenas recursos de consulta; eles também preservam escopo, ownership e visibilidade pública.

Essa regra deve guiar qualquer evolução futura em listagens da API.

15. Tratamento de erros

O tratamento de erros foi uma das frentes importantes da refatoração porque afeta diretamente a previsibilidade da API, a experiência do frontend e a manutenção do backend.

No estado inicial, a API não possuía um padrão consolidado para respostas de erro. Algumas situações retornavam `ResponseEntity.notFound()`, outras retornavam `badRequest().build()`, outras lançavam exceções genéricas como `RuntimeException`, `IllegalArgumentException` ou `EntityNotFoundException`. Também havia casos em que situações de negócio eram tratadas apenas com mensagens soltas ou sem uma estrutura comum de resposta.

No estado final, a API passou a usar um tratamento global de erros, com modelos padronizados e exceptions mais específicas. Essa mudança tornou os erros mais previsíveis para o frontend e mais fáceis de manter no backend.

A regra central é:

Erros devem ter estrutura previsível, status HTTP coerente e mensagem clara para quem consome a API.

15.1 Problema inicial dos erros

No estado inicial, a API não tinha um `@RestControllerAdvice` responsável por centralizar o tratamento de exceções. Isso fazia com que cada controller ou fluxo lidasse com erros de forma própria.

Esse modelo gerava alguns problemas.

Primeiro, o frontend não tinha um contrato confiável para interpretar erros. Em alguns casos, poderia vir corpo vazio. Em outros, uma mensagem padrão do Spring. Em outros, uma exception não tratada.

Segundo, o backend ficava mais difícil de manter. Cada novo endpoint poderia criar um padrão diferente de erro, aumentando inconsistência.

Terceiro, status HTTP diferentes poderiam ser usados para situações parecidas. Por exemplo, uma regra de negócio poderia ser retornada como `400` em um fluxo, como exception genérica em outro, ou até resultar em erro interno se não fosse tratada.

Esse cenário dificultava integração, debugging e evolução da API.

15.2 Objetivo da padronização

O objetivo da refatoração foi criar um padrão único para respostas de erro.

A ideia foi separar duas responsabilidades:

- o service identifica o problema e lança uma exception adequada;
- o handler global transforma essa exception em uma resposta HTTP padronizada.

Essa separação evita que controllers e services precisem montar respostas manualmente o tempo todo.

No estado final, a API passou a ter modelos como:

- `StandardError`;
- `ValidationError`;
- `FieldMessage`.

Esses modelos permitem que o frontend tenha uma estrutura previsível para ler mensagens, status e detalhes de validação.

15.3 StandardError

`StandardError` representa o formato básico de erro da API.

Ele deve ser usado para erros gerais, como recurso não encontrado, regra de negócio, conflito, não autorizado ou acesso negado.

A estrutura geralmente inclui informações como:

- timestamp;

- status;
- error;
- message;
- path.

A função desse modelo é garantir que, mesmo quando erros diferentes acontecem, a resposta tenha um formato comum.

Para o frontend, o campo mais importante costuma ser `message`, porque ele traz a explicação que pode ser exibida ao usuário ou usada em toast, modal ou feedback de tela.

A regra é:

Quando a API retornar erro, o frontend deve procurar primeiro uma mensagem padronizada no corpo da resposta.

15.4 `ValidationError` e `FieldMessage`

Erros de validação precisam de tratamento especial.

Quando o problema está nos campos enviados pelo cliente, não basta retornar uma mensagem genérica. O frontend precisa saber quais campos estão inválidos e por quê.

Para isso, o estado final da API usa `ValidationError` e `FieldMessage`.

`ValidationError` representa um erro de validação com lista de campos problemáticos.

`FieldMessage` representa cada campo inválido, normalmente contendo:

- nome do campo;
- mensagem de validação.

Essa estrutura permite que o frontend destaque campos específicos do formulário.

Por exemplo, se o e-mail é inválido ou um campo obrigatório não foi preenchido, o frontend pode associar a mensagem ao campo correto.

A regra é:

Erros de validação devem permitir que o frontend identifique campo e mensagem.

15.5 Exceptions específicas

A refatoração também organizou melhor as exceptions de domínio.

Em vez de depender apenas de exceptions genéricas, a API passou a usar exceptions mais específicas para representar situações diferentes.

Exemplos de categorias importantes:

- recurso não encontrado;
- regra de negócio violada;
- duplicidade;
- recurso em uso;
- integridade customizada;
- acesso negado ou não autorizado, quando aplicável ao fluxo.

Essa distinção é importante porque cada tipo de erro deve mapear para um status HTTP coerente.

Uma duplicidade, por exemplo, não deve ser tratada da mesma forma que um recurso inexistente. Uma regra de negócio violada também não deve ser necessariamente tratada como erro de validação de campo.

A regra é:

Exceptions devem expressar a natureza do problema, não apenas interromper o fluxo.

15.6 Status HTTP utilizados

No estado final, a API passou a usar status HTTP de forma mais organizada.

Os principais status são:

400 — Bad Request

Usado para erros de validação, payload inválido, parâmetros inválidos ou problemas de bind.

Esse status indica que a requisição está mal formada ou possui dados inválidos do ponto de vista do contrato.

Para o frontend, normalmente deve resultar em feedback de formulário.

401 — Unauthorized

Usado quando o usuário não está autenticado ou quando o token está ausente, inválido ou expirado.

Esse status indica problema de autenticação.

Para o frontend, normalmente significa que o usuário deve fazer login novamente, ou que o token salvo deve ser removido.

403 — Forbidden

Usado quando o usuário está autenticado, mas não tem permissão para acessar aquele recurso ou executar aquela ação.

Esse status indica problema de autorização.

Para o frontend, a resposta adequada pode ser exibir “acesso negado” ou ocultar telas e botões para perfis sem permissão.

404 — Not Found

Usado quando o recurso solicitado não existe ou não está acessível naquele contexto.

Em alguns endpoints públicos, um recurso existente internamente, mas indisponível publicamente, também pode ser tratado como não encontrado. Isso evita expor detalhes de recursos que ainda não deveriam ser visíveis.

409 — Conflict

Usado para conflitos, duplicidades ou recursos em uso.

Exemplos possíveis incluem tentativa de cadastrar dados duplicados ou remover algo que está associado a outros recursos.

Esse status ajuda o frontend a diferenciar conflito de validação simples.

422 — Unprocessable Entity

Usado para regras de negócio violadas.

Esse status representa situações em que a requisição é sintaticamente válida, mas não pode ser processada por uma regra do domínio.

Exemplos de regras de negócio:

- tentar demonstrar interesse duplicado;
- tentar remover interesse inexistente;
- tentar fazer autoindicação;
- tentar publicar como empresa sem vínculo aprovado;
- tentar usar tipo de emprego não aprovado;
- tentar executar ação sem possuir o perfil necessário.

A regra é:

400 é problema de formato/validação da requisição; 422 é problema de regra de negócio.

15.7 Diferença entre erro de validação e regra de negócio

Essa distinção é importante para backend e frontend.

Erro de validação acontece quando o payload não atende ao contrato esperado. Por exemplo: campo obrigatório ausente, e-mail inválido, valor nulo onde não pode, formato de enum inválido ou tipo incorreto.

Regra de negócio acontece quando os dados até podem estar bem formados, mas a ação não é permitida pelo domínio.

Exemplo: um candidato tenta demonstrar interesse em uma vaga na qual já demonstrou interesse. O payload pode estar correto, mas a regra de negócio impede a operação.

Essa diferença ajuda o frontend a decidir como exibir o erro.

Validação geralmente aparece perto do campo.

Regra de negócio geralmente aparece como mensagem geral da ação.

15.8 Erros de segurança

Erros de segurança foram padronizados para melhorar a experiência de consumo da API.

Antes, um token inválido ou uma tentativa de acesso sem permissão poderia gerar respostas menos previsíveis. No estado final, 401 e 403 passaram a ter comportamento mais claro.

A diferença é:

- 401: o usuário não está autenticado corretamente;
- 403: o usuário está autenticado, mas não tem permissão.

Essa distinção é especialmente importante no frontend.

Se a API retorna 401, o frontend pode redirecionar para login ou limpar a sessão.

Se a API retorna 403, o frontend deve informar que o usuário não tem permissão para aquela ação.

15.9 Erros em endpoints /me

Nos endpoints /me, erros geralmente indicam problemas ligados ao usuário autenticado ou ao perfil necessário.

Exemplos:

- usuário autenticado não possui perfil de candidato;
- usuário autenticado não possui perfil de recrutador;
- recurso não pertence ao usuário;
- experiência não pertence ao candidato;
- oportunidade não pertence ao recrutador;
- tentativa de ação duplicada ou inválida.

Esses erros precisam ser claros porque o frontend usa endpoints /me para telas pessoais.

A regra é:

Endpoints /me devem evitar depender de IDs sensíveis do cliente e devem retornar erros claros quando o perfil ou recurso não for compatível com o usuário autenticado.

15.10 Erros em endpoints admin

Endpoints admin podem retornar erros de autenticação, permissão, recurso inexistente, conflito e regra de negócio.

Um usuário sem token recebe 401.

Um usuário autenticado sem perfil admin recebe 403.

Um ID inexistente retorna 404.

Uma operação administrativa inválida pode retornar 422 ou 409, conforme o caso.

Essa separação ajuda o frontend administrativo a exibir mensagens corretas.

A regra é:

Endpoint admin deve continuar protegido no backend, mesmo que o frontend esconda a tela de usuários comuns.

15.11 Erros de Localidade

O módulo de Localidade tem comportamento especial.

Nem toda falha de resolução de localidade deve virar erro final para o usuário.

Em alguns casos, se a localidade não puder ser resolvida com segurança, a API pode criar uma `LocalidadePendente`, salvar o recurso internamente e ocultá-lo do público até reprocessamento.

Isso significa que localidade pendente não deve ser tratada sempre como erro.

A regra é:

`LocalidadePendente` é estado técnico temporário, não erro fatal de cadastro.

Porém, existem casos em que a localidade deve gerar erro, por exemplo quando IDs estruturados são inconsistentes ou inválidos. Nesse caso, a API deve retornar erro de validação ou regra de negócio conforme a situação.

Essa distinção é importante para o frontend. Quando o backend cria uma pendência, a resposta pode ser de sucesso com status interno de localidade. Quando a entrada é inválida, a resposta deve ser erro.

15.12 Mensagens em português e acentuação

Durante a evolução da API, houve cuidado especial com mensagens em português brasileiro e acentuação.

No estado inicial, havia textos com encoding quebrado ou sem acentuação adequada. Isso prejudicava a qualidade da API e a clareza das mensagens.

Na fase final, mensagens expostas, documentação Swagger, JavaDoc e README foram revisados para evitar problemas como:

- `usuario` em vez de `usuário`;
- `validacao` em vez de `validação`;
- `correcao` em vez de `correção`;
- mojibake como `NÃ£o`, `invÃlido` ou `descriÃ§Ã£o`.

Esse cuidado é importante porque mensagens de erro podem aparecer diretamente para usuário, avaliadores ou desenvolvedores de frontend.

A regra é:

Textos humanos expostos pela API devem estar em português correto, com acentuação real em UTF-8.

15.13 Como o frontend deve tratar erros

Para o frontend, a regra principal é consumir a estrutura padronizada da API.

Quando houver erro simples, o frontend deve buscar a mensagem geral da resposta.

Quando houver erro de validação, deve verificar se existe lista de campos inválidos.

A interpretação recomendada é:

- 400: destacar campos inválidos ou informar erro de entrada;
- 401: redirecionar para login ou limpar sessão;
- 403: mostrar acesso negado;
- 404: mostrar recurso não encontrado;
- 409: mostrar conflito ou duplicidade;
- 422: mostrar mensagem de regra de negócio retornada pelo backend.

O frontend não deve depender de texto fixo exato para tomar decisões importantes quando o status HTTP já expressa o tipo do problema. A mensagem deve ser usada para feedback ao usuário.

15.14 Por que o backend não deve delegar erro ao frontend

Algumas validações podem ser feitas no frontend para melhorar experiência. Por exemplo, impedir envio de campo obrigatório vazio antes de chamar a API.

Mas isso não substitui validação no backend.

O backend precisa continuar validando porque:

- o cliente pode ser manipulado;
- outro consumidor pode usar a API;
- requisições podem ser feitas fora do frontend;
- regras de negócio pertencem ao domínio;
- segurança não pode depender da interface.

A regra é:

Frontend pode prevenir erro. Backend deve garantir regra.

Essa decisão vale para validações simples e para regras de negócio.

15.15 Tratamento de erros e documentação Swagger

Os modelos de erro também foram documentados no Swagger/OpenAPI.

Isso ajuda a pessoa do frontend a entender o formato esperado das respostas de erro e facilita testes manuais.

A documentação inclui modelos como `StandardError`, `ValidationError` e `FieldMessage`, além de respostas esperadas nos endpoints.

Essa documentação é importante porque erros fazem parte do contrato da API. Não basta documentar apenas respostas de sucesso.

A regra de manutenção é:

Ao criar novo padrão de erro ou novo status relevante, atualizar Swagger e testes quando necessário.

15.16 Testes relacionados a erros

A cobertura de erros evoluiu junto com a API, mas ainda pode melhorar.

O estado final já possui testes para segurança, controllers, services e regras de negócio importantes. O documento de testes também reconhece lacunas, como ampliar testes HTTP específicos para corpos de erro 400, 404, 409 e 422.

Isso significa que a base atual é muito melhor que a inicial, mas ainda existe espaço para fortalecer a suíte.

Recomendação futura:

Para fluxos críticos, testar não apenas o status, mas também o corpo do erro retornado.

Isso ajuda a proteger o contrato com o frontend.

15.17 Cuidados para o próximo backend

Ao criar ou alterar regras de erro, o próximo desenvolvedor deve observar alguns cuidados.

Primeiro, não lançar `RuntimeException` genérica para regra de negócio esperada.

Segundo, usar exceptions específicas quando o erro representar uma situação conhecida do domínio.

Terceiro, preservar a diferença entre validação, conflito, não encontrado e regra de negócio.

Quarto, evitar montar erro manualmente em cada controller.

Quinto, manter mensagens humanas em português claro e com acentuação correta.

Sexto, ao alterar formato de erro, avisar que isso impacta o frontend.

Sétimo, adicionar ou ajustar testes quando um erro fizer parte importante do contrato.

15.18 Síntese

A evolução do tratamento de erros tornou a API mais previsível, mais integrável e mais fácil de manter.

No início, erros eram tratados de forma dispersa e inconsistente. No estado final, a API passou a ter modelos padronizados, handler global, exceptions específicas e status HTTP mais coerentes.

A regra mais importante é:

Erro também é contrato da API.

Essa regra deve orientar novas funcionalidades. Uma API bem documentada e previsível não trata apenas sucesso de forma organizada; ela também trata falhas de forma clara, consistente e útil para o frontend.

16. Documentação, README, testes, Docker e Postman

A etapa final da API **Quero Trabalhar Backend** não se limitou à implementação de regras de negócio. Também foi necessário preparar o projeto para ser compreendido, executado, testado e integrado por outras pessoas.

No estado inicial, a documentação era limitada e desatualizada. O README mencionava Java 17, embora o Maven estivesse configurado com Java 21; citava Spring Security como algo futuro, mesmo a API já tendo segurança; e mencionava Swagger, embora não houvesse configuração real de OpenAPI/SpringDoc. Também não havia Docker, documentação de testes, JavaDoc técnico relevante ou coleção Postman organizada.

No estado final, a API passou a contar com documentação em diferentes níveis: Swagger/OpenAPI para contrato HTTP, JavaDoc para decisões técnicas próximas ao código, README para execução e visão geral, documentação de testes, Docker para execução em container e coleção Postman para validação manual dos fluxos.

A regra geral desta etapa foi:

Uma API de MVP não deve apenas funcionar. Ela precisa ser compreensível, executável, testável e integrável por outras pessoas.

16.1 Por que a documentação virou parte da entrega

Durante a evolução da API, algumas regras ficaram complexas o suficiente para não dependerem apenas da leitura do código.

Exemplos importantes:

- por que uma empresa pode existir internamente, mas não aparecer publicamente;
- por que uma oportunidade aparece no painel do recrutador, mas não aparece em `GET /api/oportunidades`;
- por que `LocalidadePendente` não deve ser tratada como erro fatal;
- por que endpoints `/me` são preferidos em ações pessoais;
- por que tipos de emprego precisam de aprovação;
- por que algumas listagens são paginadas e outras continuam como `List`;
- por que Google Maps não aparece como endpoint público;
- por que empresa ainda não possui ownership formal.

Essas decisões são regras de arquitetura e produto. Se não forem documentadas, uma pessoa nova pode interpretá-las como bugs, inconsistências ou omissões.

Por isso, a documentação foi tratada como parte essencial da entrega do MVP.

16.2 Níveis de documentação criados

A documentação final foi organizada em diferentes níveis, cada um com uma finalidade específica.

O Swagger/OpenAPI documenta o contrato formal da API: endpoints, parâmetros, requests, responses, modelos de erro, autenticação e filtros.

O JavaDoc documenta decisões técnicas internas próximas ao código, especialmente nos módulos mais sensíveis.

O README documenta como executar o projeto, quais tecnologias são usadas, quais módulos existem e como acessar recursos como Swagger e Docker.

O `docs/testing.md` documenta a suíte de testes automatizados, sua estrutura, comandos de execução, tipos de teste e lacunas conhecidas.

A coleção Postman ajuda na validação manual dos fluxos, principalmente autenticação, endpoints protegidos, endpoints públicos e fluxos administrativos.

Esta documentação histórica complementa todos esses materiais explicando a evolução, o raciocínio e as decisões tomadas.

Cada uma dessas documentações cumpre um papel diferente. Nenhuma substitui totalmente a outra.

16.3 Swagger/OpenAPI

A documentação Swagger/OpenAPI foi uma das entregas centrais da etapa final.

No estado final, a API passou a ter uma configuração base de OpenAPI com título, descrição, versão e autenticação Bearer JWT. Os endpoints foram documentados com `@Tag`, `@Operation` e `@ApiResponse`s, enquanto DTOs de request, response, filtros e modelos de erro foram documentados com `@Schema` ou estruturas equivalentes.

O Swagger passou a documentar:

- endpoints públicos;
- endpoints autenticados;
- endpoints `/me`;
- endpoints admin;
- paginação;
- filtros;
- modelos de erro;
- autenticação JWT;
- DTOs principais;

- contrato do login;
- comportamento de endpoints específicos, como aprovação em lote.

Essa documentação é especialmente útil para o frontend, porque permite consultar rapidamente quais endpoints existem, quais parâmetros aceitam e quais respostas retornam.

16.4 Documentação manual do login

Um detalhe importante foi o endpoint de login.

O login da API é feito em `POST /login`, mas esse endpoint não nasce de um controller tradicional. Ele é implementado pelo filtro de autenticação do Spring Security.

Por isso, o Swagger não conseguiria documentá-lo automaticamente da mesma forma que documenta controllers comuns.

A solução foi documentar o login manualmente na configuração OpenAPI.

Isso foi importante porque o login é o primeiro ponto de integração do frontend. A documentação precisava deixar claro que:

- o usuário envia credenciais;
- a autenticação retorna token JWT;
- o token vem no header `Authorization`;
- as próximas requisições protegidas usam Bearer Token.

Essa decisão melhora muito a experiência de quem vai testar ou integrar a API.

16.5 Swagger restrito ao ambiente local

Outra decisão importante foi restringir o Swagger ao ambiente adequado.

No estado final, Swagger UI e `/v3/api-docs` ficam disponíveis em ambiente local e são desabilitados em ambientes como test/prod.

Essa decisão equilibra produtividade e segurança.

Durante desenvolvimento, o Swagger ajuda a testar e entender a API. Em ambientes mais restritos, expor a documentação interativa pode não ser desejável.

A regra é:

Swagger é ferramenta de desenvolvimento e integração local, não necessariamente recurso público de produção.

16.6 JavaDoc técnico

Além do Swagger, foi adicionado JavaDoc técnico em partes críticas do código.

O objetivo do JavaDoc não foi repetir o que o método já mostra, mas explicar decisões que não são óbvias apenas pela implementação.

O foco principal foi em módulos como:

- Localidade;
- LocalidadePendente;
- reprocessamento de pendências;
- integração Google Maps;
- Empresa;
- Oportunidade;
- Specifications;
- serviços de resolução e visibilidade.

Esse JavaDoc explica, por exemplo:

- por que **LocalidadePendente** é uma fila técnica;
- por que ela não deve ser exposta como recurso público;
- por que Empresa e Oportunidade não devem depender diretamente da pendência;
- por que ambiguidade não deve ser resolvida automaticamente;
- por que oportunidade pública exige Localidade validada;
- por que endpoints internos podem exibir status de pendência.

Esse tipo de documentação ajuda muito o próximo backend, porque registra o raciocínio por trás da arquitetura.

16.7 Cuidado com acentuação e encoding

Durante a documentação, houve uma preocupação especial com textos em português e encoding.

No estado inicial, havia textos sem acentuação ou com caracteres quebrados. Isso afetava mensagens de erro, README e documentação.

Na etapa final, textos expostos foram revisados para português brasileiro com acentuação correta.

Também foi necessário evitar duas armadilhas.

A primeira era o mojibake, como `NÃ£o`, `invÃ;lido` ou `descriÃ§Ã£o`.

A segunda era trocar acentos reais por Unicode escapes em comentários e JavaDoc, como `resolu\u00e7\u00e3o`. Embora isso compile, prejudica a legibilidade do código.

A decisão final foi manter textos humanos em UTF-8 real, com acentos corretos.

A regra é:

Texto humano deve ser legível no próprio arquivo. Não usar Unicode escapes para esconder problema de terminal.

16.8 README

O README foi tratado como guia de entrada do projeto.

Ele deve servir para alguém que acabou de receber a API e precisa entender rapidamente:

- o que é o projeto;
- qual o objetivo da API;
- qual stack é usada;

- como configurar ambiente;
- como executar localmente;
- como executar com Docker;
- como acessar Swagger;
- como funciona autenticação JWT;
- quais são os módulos principais;
- como funcionam Localidade e LocalidadePendente;
- como rodar testes;
- quais melhorias ficaram para o futuro;
- quem são os responsáveis pelo projeto.

O README não deve substituir a documentação detalhada, mas precisa dar visão suficiente para a pessoa começar.

Também foi importante atualizar informações antigas, como a divergência entre Java 17 e Java 21, e remover menções desatualizadas sobre segurança futura.

16.9 README como documento de integração inicial

Para a pessoa do frontend, o README também tem papel importante.

Ele deve indicar onde encontrar:

- Swagger;
- coleção Postman;
- credenciais ou dados de seed, se documentados;
- porta padrão da API;
- comando de execução local;
- comando de execução via Docker;
- fluxo de login;
- variáveis de ambiente relevantes.

Isso reduz o tempo necessário para começar a integrar.

A pessoa do frontend não deve precisar ler toda a documentação histórica antes de fazer a primeira requisição. O README deve permitir esse primeiro contato.

A documentação histórica entra depois, para explicar regras mais profundas.

16.10 docs/testing.md

Foi planejada e/ou criada uma documentação específica para testes em [docs/testing.md](#).

O objetivo desse documento é explicar a suíte de testes automatizados da API.

Ele deve cobrir:

- stack de testes;
- comandos de execução;
- estrutura das pastas de teste;
- tipos de teste;
- testes de controller;
- testes de service;
- testes de security;
- testes de repository;
- testes de domínio;
- testes de OpenAPI;
- testes de Localidade/LocalidadePendente;
- testes de paginação e filtros;
- contrato de erros;
- convenções adotadas;
- lacunas futuras.

Essa documentação é importante porque o estado final possui uma suíte de testes muito mais significativa que a inicial. O diagnóstico final cita 34 classes de teste documentadas, cobrindo controllers, services, security, OpenAPI, localidade, paginação/filtros, domínio e repository.

16.11 Papel dos testes no projeto

Os testes não foram adicionados apenas para aumentar número de cobertura. Eles servem para proteger decisões importantes.

Entre os comportamentos que os testes ajudam a proteger estão:

- endpoints públicos não exibirem recursos com localidade pendente;
- endpoints `/me` usem o usuário autenticado;
- endpoints admin exijam permissão;
- filtros funcionarem com paginação;
- empresas e oportunidades respeitem Localidade validada;
- reprocessamento de pendências não duplicar `LocalidadePendente`;
- Swagger documentar contratos esperados;
- regras de interesse, indicação, experiência e perfil funcionarem corretamente.

Essa suíte é importante porque vários comportamentos da API não são triviais.

Se uma alteração futura quebrar visibilidade pública, ownership ou tratamento de localidade, os testes devem ajudar a detectar.

16.12 Limitações conhecidas dos testes

Apesar da evolução, a suíte de testes ainda pode melhorar.

O diagnóstico final aponta lacunas reconhecidas, como:

- mais testes HTTP para corpos de erro 400, 404, 409 e 422;
- mais cobertura de writes em controllers;
- relatório de cobertura automatizado;
- testes mais explícitos de CORS, serialização e profiles;
- cobertura adicional para cenários de autenticação inválida ou expirada.

Essas lacunas não impedem o MVP, mas devem ser consideradas no roadmap de qualidade.

A regra para evolução futura é:

Fluxo crítico novo deve vir acompanhado de teste correspondente.

16.13 Dockerização

A dockerização foi adicionada para tornar a execução do projeto mais simples e reprodutível.

No estado inicial, não havia Dockerfile, docker-compose ou `.dockerignore`. Isso obrigava cada pessoa a configurar o ambiente manualmente.

No estado final, a API passou a contar com:

- `Dockerfile`;
- `docker-compose.yml`;
- `.dockerignore`.

O diagnóstico final registra um Dockerfile multi-stage, com build usando Maven e runtime em Java 21 JRE, além de docker-compose para subir a API em perfil local, com variáveis para JWT, Google Maps, CORS e reprocessamento.

Essa mudança facilita:

- avaliação do projeto;
- execução em outra máquina;
- demonstração;
- onboarding de novo dev;
- padronização mínima do ambiente local.

16.14 Papel do Docker no MVP

Docker não significa que o projeto está pronto para produção completa. Ele significa que o MVP ficou mais fácil de executar de forma padronizada.

A API ainda pode evoluir em aspectos de infraestrutura, banco externo, observabilidade e deploy real. Mas a presença de Docker já reduz bastante a fricção para quem vai rodar o projeto.

A regra é:

Docker é ferramenta de reprodutibilidade do ambiente, não substitui análise de produção.

Essa distinção é importante para não superestimar a entrega. O projeto está mais preparado, mas ainda pode precisar de ajustes antes de produção real.

16.15 Coleção Postman

A coleção Postman foi criada como apoio para testes manuais e exploração da API.

Ela cobre os endpoints principais, organiza requests por módulo e inclui configuração de autenticação com Bearer Token.

O fluxo de login foi configurado para capturar automaticamente o token retornado no header **Authorization** e salvá-lo em uma variável de ambiente do Postman.

Essa coleção é útil para:

- validar rapidamente endpoints públicos;
- testar login;
- testar endpoints protegidos;
- testar fluxos **/me**;
- testar fluxos admin;
- apoiar a pessoa do frontend;
- demonstrar a API.

A coleção não substitui os testes automatizados, mas ajuda muito em exploração manual e integração.

16.16 Dados de seed e Postman

O semeador do banco ajuda a coleção Postman porque cria usuários, perfis, tipos de emprego, oportunidades, interesses e indicações iniciais.

Com dados semeados, é mais fácil testar fluxos como:

- login com usuários existentes;
- listagem de oportunidades;

- listagem de empresas, quando aplicável ao estado final;
- tipos de emprego aprovados;
- interesses em vagas;
- indicações;
- fluxos de admin;
- fluxos de recrutador.

Isso reduz a necessidade de criar tudo manualmente antes de testar.

Para o frontend, esse conjunto de dados facilita desenvolver telas com exemplos reais de resposta.

16.17 Como cada artefato deve ser usado

Cada artefato final tem um papel específico.

O README deve ser o primeiro arquivo lido por quem recebe o projeto.

O Swagger deve ser usado para consultar contratos de endpoints.

O Postman deve ser usado para testar fluxos manualmente.

O `docs/testing.md` deve ser usado para entender e executar a suíte de testes.

O JavaDoc deve ser usado por quem vai alterar código sensível.

Esta documentação histórica deve ser usada para entender decisões, contexto e regras de continuidade.

A regra é:

Cada documentação responde a uma pergunta diferente.

O README responde como começar.

O Swagger responde qual é o contrato.

O Postman responde como testar manualmente.

Os testes respondem o que está protegido automaticamente.

O JavaDoc responde por que determinado código existe daquele jeito.

Esta documentação responde por que a API evoluiu para esse desenho e como continuar sem quebrar decisões importantes.

16.18 O que o frontend deve usar primeiro

Para a pessoa responsável pelo frontend, a ordem recomendada é:

1. ler o README para executar a API;
2. importar o ambiente e a coleção Postman;
3. testar `POST /login`;
4. consultar Swagger para contratos;
5. testar endpoints públicos;
6. testar endpoints `/me`;
7. testar endpoints admin se houver usuário admin;
8. ler as seções desta documentação sobre autenticação, paginação, erros e localidade.

Essa ordem permite começar de forma prática, sem precisar entender todo o histórico antes da primeira integração.

Depois, as regras mais profundas, como LocalidadePendente, DTO público vs interno e endpoints `/me`, devem ser lidas para evitar interpretações erradas.

16.19 Cuidados para manutenção da documentação

A documentação precisa evoluir junto com a API.

Algumas mudanças exigem atualização obrigatória de documentação:

- novo endpoint;
- novo campo em DTO;
- novo status de erro;
- mudança em regra de localidade;
- mudança em autenticação;
- novo filtro;

- novo comportamento de paginação;
- alteração em Docker;
- alteração em variáveis de ambiente;
- mudança em fluxos admin;
- implementação de funcionalidades pós-MVP.

A regra é:

Contrato mudou, documentação muda.

Se a documentação não acompanhar o código, ela perde confiança rapidamente.

16.20 Cuidados para o próximo backend

O próximo desenvolvedor deve manter alguns cuidados.

Primeiro, não adicionar endpoint sem documentação Swagger correspondente.

Segundo, não alterar DTO relevante sem revisar frontend, Swagger e testes.

Terceiro, não mudar erro padronizado sem considerar impacto no frontend.

Quarto, não alterar regra de LocalidadePendente sem revisar JavaDoc e documentação histórica.

Quinto, manter README atualizado quando mudar execução, Docker ou variáveis.

Sexto, atualizar Postman quando fluxos principais mudarem.

Sétimo, manter `docs/testing.md` atualizado quando a suíte crescer.

Esses cuidados evitam que a API volte ao estado inicial, em que documentação e código estavam desalinhados.

16.21 Síntese

A fase de documentação, testes, Docker e Postman foi essencial para transformar a API em uma entrega mais completa de MVP.

No início, o projeto tinha pouca documentação, nenhum Docker, testes mínimos e contratos pouco claros. No final, passou a contar com Swagger/OpenAPI, JavaDoc técnico, README atualizado, documentação de testes, Dockerização e coleção Postman.

Essa evolução melhora a continuidade do projeto.

A regra mais importante é:

Uma API não está pronta apenas quando o código funciona. Ela está mais pronta quando outra pessoa consegue entender, executar, testar, integrar e evoluir com segurança.

Essa foi a intenção da etapa final: deixar o projeto mais acessível para o próximo backend, para o frontend e para qualquer pessoa que precise avaliar ou dar continuidade à API.

17. Guia de integração para o frontend

Esta seção foi escrita especificamente para orientar a pessoa responsável pelo frontend na integração com a API **Quero Trabalhar Backend**.

A API possui documentação Swagger/OpenAPI, coleção Postman, README e documentação técnica, mas alguns comportamentos importantes precisam ser entendidos além do contrato formal dos endpoints. A principal diferença está no contexto de uso: existem endpoints públicos, endpoints autenticados do próprio usuário e endpoints administrativos. Cada grupo tem regras próprias de autenticação, visibilidade, retorno e tratamento de erro.

O objetivo desta seção é explicar como o frontend deve consumir a API de forma segura e coerente com as regras do backend.

A regra geral é:

O frontend deve usar o backend como fonte de verdade para autenticação, permissões, visibilidade pública, paginação, filtros, erros e status de localidade.

17.1 Visão geral para o frontend

A API está organizada para atender três contextos principais de tela:

- telas públicas;
- telas do usuário autenticado;
- telas administrativas.

As telas públicas usam endpoints que não exigem token e exibem apenas dados prontos para publicação. Exemplos: listagem de vagas, listagem de empresas, catálogo de tipos de emprego aprovados e catálogo de localidades.

As telas do usuário autenticado usam endpoints `/me`. Elas mostram dados do próprio usuário, suas experiências, seus interesses, seu perfil de candidato, seu perfil de recrutador e suas oportunidades. Nesses endpoints, o backend identifica o usuário pelo token JWT.

As telas administrativas usam endpoints `/api/admin/**`. Elas exigem usuário autenticado com permissão administrativa e servem para gestão global, moderação e aprovação de vínculos.

Essa divisão precisa ser refletida no frontend. Uma tela pública não deve depender de dados internos. Uma tela `/me` não deve pedir `usuarioId` manualmente. Uma tela admin não deve ser exibida para usuários comuns.

17.2 Autenticação no frontend

O login da API é feito por `POST /login`.

Quando o login é bem-sucedido, o backend retorna o JWT no header `Authorization`, no formato Bearer Token.

O frontend deve capturar esse header, guardar o token e reenviá-lo nas próximas requisições protegidas.

O fluxo esperado é:

1. usuário informa e-mail e senha;

2. frontend chama `POST /login`;
3. backend retorna `Authorization: Bearer <token>`;
4. frontend extrai o token;
5. frontend salva o token conforme estratégia da aplicação;
6. frontend envia o token em endpoints protegidos.

A coleção Postman foi configurada exatamente com essa lógica: depois do login, o script de pós-resposta captura o header `Authorization`, remove o prefixo `Bearer` e salva o token na variável `jwt_token`.

No frontend real, a mesma ideia deve ser aplicada no cliente HTTP usado pela aplicação.

A regra é:

O token não vem no corpo da resposta. Ele vem no header `Authorization`.

17.3 Envio do token nas requisições protegidas

Para endpoints protegidos, o frontend deve enviar o token como Bearer Token.

O backend espera o token no header `Authorization`.

A regra prática é:

Endpoints protegidos exigem `Authorization: Bearer <token>`.

Os endpoints públicos não precisam desse header.

Os endpoints `/me` e `admin` precisam.

Se o frontend deixar de enviar o token em um endpoint protegido, o backend deve responder 401.

Se enviar token válido, mas o usuário não tiver permissão, o backend deve responder 403.

17.4 Diferença entre 401 e 403

O frontend deve tratar 401 e 403 de formas diferentes.

401 Unauthorized significa que o usuário não está autenticado corretamente. Pode ocorrer por token ausente, token inválido, token expirado ou login inválido.

Quando receber 401, o frontend normalmente deve:

- limpar token inválido, se houver;
- redirecionar para login;
- pedir nova autenticação;
- exibir mensagem de sessão expirada, quando fizer sentido.

403 Forbidden significa que o usuário está autenticado, mas não tem permissão para aquela ação.

Quando receber 403, o frontend normalmente deve:

- mostrar mensagem de acesso negado;
- impedir ação;
- ocultar botões ou telas que exigem perfil não disponível;
- não necessariamente deslogar o usuário.

A regra é:

401 é problema de autenticação. 403 é problema de autorização.

17.5 Endpoints públicos

Endpoints públicos são consumidos sem autenticação.

Eles servem para telas abertas da aplicação, como página inicial, listagem pública de vagas, listagem pública de empresas, detalhe público de empresa, detalhe público de vaga, catálogo de localidades e catálogo de tipos aprovados.

Exemplos de endpoints públicos:

- **GET /api/oportunidades;**

- GET /api/oportunidades/{id};
- GET /api/empresas;
- GET /api/empresas/{id};
- GET /api/empresas/{id}/oportunidades;
- GET /api/empresas/{id}/recrutadores;
- GET /api/tipos-de-emprego/aprovados;
- GET /api/tipos-de-emprego/{id};
- GET /api/localidades/paises;
- GET /api/localidades/estados;
- GET /api/localidades/cidades;
- POST /api/usuarios/cadastrar;
- POST /login.

A regra mais importante é:

Endpoints públicos de empresas e oportunidades retornam apenas recursos com Localidade validada.

Isso significa que o frontend público não precisa filtrar manualmente recursos pendentes. O backend já não deve retorná-los.

17.6 Endpoints /me

Endpoints /me exigem autenticação e operam sobre o usuário logado.

Eles são usados em painéis internos, como área do candidato, área do recrutador e área de perfil do usuário.

Exemplos:

- GET /api/usuarios/me;
- PUT /api/usuarios/me;
- DELETE /api/usuarios/me;
- PATCH /api/usuarios/me/senha;

- endpoints de perfil candidato do usuário autenticado;
- endpoints de perfil recrutador do usuário autenticado;
- `GET /api/recrutadores/me/oportunidades`;
- `GET /api/candidatos/me/interesses/vagas`;
- endpoints de experiências do candidato autenticado;
- endpoints de indicações dadas e recebidas.

A regra principal é:

O frontend não deve enviar `usuarioId` para ações pessoais quando existir endpoint `/me`.

O backend descobre o usuário pelo token. Isso reduz risco de IDOR e simplifica integração.

17.7 Endpoints admin

Endpoints admin ficam sob `/api/admin/**`.

Eles exigem autenticação e perfil administrativo.

Exemplos:

- `GET /api/admin/usuarios`;
- `GET /api/admin/usuarios/{id}`;
- endpoints admin de perfil candidato e recrutador;
- `GET /api/admin/experiencias`;
- `GET /api/admin/tipos-emprego/nao-aprovados`;
- `PATCH /api/admin/tipos-emprego/{id}/aprovar`;
- `PATCH /api/admin/tipos-emprego/aprovar-lote`;
- `PATCH /api/admin/recrutadores/{recrutadorId}/empresa/aprovar`;
- `PATCH /api/admin/recrutadores/{recrutadorId}/empresa/recusar`.

A regra para o frontend é:

Tela admin só deve ser acessível para usuário com permissão administrativa, mas o backend continua sendo responsável por bloquear acesso indevido.

Mesmo que o frontend esconda a tela, a API ainda deve responder 403 para usuário sem permissão.

17.8 Diferença entre DTO público e DTO interno

Uma das coisas mais importantes para o frontend entender é que a API pode usar DTOs diferentes para o mesmo recurso, dependendo do contexto.

Um DTO público é usado em endpoints públicos. Ele deve conter apenas dados seguros para exibição geral.

Um DTO interno é usado em endpoints autenticados ou administrativos. Ele pode trazer informações adicionais, como status de localidade, texto original informado ou motivo de pendência.

Exemplos conceituais:

- empresa pública não expõe detalhes de `LocalidadePendente`;
- oportunidade pública não expõe detalhes de `LocalidadePendente`;
- oportunidade no painel do recrutador pode expor status de localidade;
- empresa em fluxo interno pode expor status de localidade.

A regra para o frontend é:

Tela pública consome DTO público. Painel interno consome DTO interno.

Não se deve esperar os mesmos campos em todos os contextos.

17.9 Localidade no frontend

Localidade é uma regra central para a integração.

Em endpoints públicos, se uma empresa ou vaga aparece, isso significa que ela possui Localidade validada.

O frontend público não precisa lidar com `LocalidadePendente`, porque recursos pendentes não devem aparecer em endpoints públicos.

Em endpoints internos, especialmente `/me`, o frontend pode receber informações indicando que uma empresa ou oportunidade está com localidade pendente.

Isso deve ser interpretado assim:

O recurso foi salvo, mas ainda não está apto para aparecer publicamente porque a localidade não foi validada com segurança.

Esse estado não deve ser tratado como erro fatal de cadastro.

Uma mensagem adequada para tela interna seria algo como:

“Este recurso foi salvo, mas a localidade ainda está em validação. Enquanto isso, ele pode não aparecer nas listagens públicas.”

O frontend não deve prometer confirmação manual de localidade se esse fluxo ainda não estiver implementado.

17.10 O que significa uma empresa ou vaga não aparecer no público

Uma dúvida provável no frontend é: “criei uma empresa ou vaga, mas ela não apareceu na listagem pública”.

Isso pode acontecer por regra de localidade.

Se a localidade ficou pendente, o recurso existe internamente, mas não aparece publicamente.

Portanto, a ausência na listagem pública não deve ser interpretada automaticamente como erro.

A lógica correta é:

- verificar o painel interno do usuário;
- observar o status de localidade;
- informar ao usuário que o recurso está aguardando validação, se for o caso.

A regra é:

Público mostra apenas recursos prontos. Painel interno mostra também recursos em processamento.

17.11 Paginação no frontend

Endpoints paginados retornam uma estrutura `Page`.

O array principal de dados fica em `content`.

Além de `content`, a resposta traz metadados úteis, como:

- número da página atual;
- tamanho da página;
- total de elementos;
- total de páginas;
- indicador de primeira ou última página;
- informações de ordenação.

O frontend deve usar esses metadados para montar paginação, botões de próximo/anterior, quantidade total e navegação.

A regra é:

Em endpoint paginado, a lista real está em `content`.

Não tratar a resposta inteira como se fosse um array.

17.12 Query params de paginação

Os principais parâmetros de paginação seguem o padrão Spring:

- `page`;
- `size`;
- `sort`.

O parâmetro `page` começa em zero.

Isso significa que a primeira página é `page=0`, não `page=1`.

O parâmetro `size` define quantos itens devem vir por página.

O parâmetro `sort` define ordenação, geralmente no formato `campo,direcao`.

A regra para o frontend é:

Ajustar a paginação visual para o usuário, mas lembrar que o backend usa índice de página começando em zero.

Se a interface mostrar página 1 para o usuário, internamente pode precisar enviar `page=0`.

17.13 Filtros no frontend

Filtros são enviados como query params.

Cada endpoint possui filtros próprios, documentados no Swagger.

Exemplos de filtros comuns:

- `termo`;
- `tipoDeEmpregoId`;
- `empresaId`;
- `recrutadorId`;
- `paisId`;
- `estadoId`;
- `cidadeId`;
- `modalidade`;

- `statusLocalidade`, em contextos internos;
- filtros de data, em contextos admin;
- filtros de perfil, em usuários admin.

O frontend deve sempre consultar o Swagger para confirmar quais filtros existem em cada endpoint.

A regra é:

Não aplicar filtro client-side em listas grandes quando o backend oferece filtro.

O ideal é enviar filtros para a API e deixar o backend consultar o conjunto correto no banco.

17.14 Filtros de localidade

Filtros por localidade devem usar IDs estruturados.

Em vez de filtrar por texto livre de cidade ou estado, o frontend deve usar:

- `paisId`;
- `estadoId`;
- `cidadeId`.

Esses IDs vêm dos endpoints de catálogo de Localidade.

O fluxo esperado no frontend é:

1. carregar países;
2. carregar estados filtrando por país;
3. carregar cidades filtrando por estado;
4. enviar os IDs escolhidos nos filtros ou formulários.

Essa abordagem evita inconsistência por texto e segue o modelo final da API.

A regra é:

Filtro confiável de localidade usa ID, não texto livre.

17.15 Formulários com localidade

Em formulários de empresa e oportunidade, a localidade pode ser informada de forma estruturada ou por texto livre, conforme o contrato do endpoint.

Quando o frontend usar IDs de país, estado e cidade, a chance de validação imediata é maior, porque a API consegue montar uma Localidade oficial.

Quando usar texto livre, a API pode tentar resolver, mas também pode criar uma pendência se o texto for ambíguo, incompleto ou não resolvido com segurança.

Por isso, para melhor experiência, o frontend deve preferir seleção estruturada sempre que possível.

A regra recomendada é:

Quando houver catálogo disponível, o frontend deve priorizar IDs estruturados de localidade.

Texto livre deve ser usado com cuidado e entendido como fluxo sujeito a pendência.

17.16 StatusLocalidade no frontend

Alguns endpoints internos podem aceitar ou retornar `statusLocalidade`.

Esse campo serve para distinguir recursos com localidade validada de recursos pendentes no contexto do usuário autenticado.

Em telas públicas, esse filtro não faz sentido, porque o público só vê validado.

Em telas internas, pode ser útil.

Exemplo de uso:

- painel do recrutador com filtro “todas”, “publicadas” e “aguardando validação”;
- painel do candidato para entender se uma vaga de interesse está em estado interno específico;
- futura área administrativa de acompanhamento.

A regra é:

`statusLocalidade` é conceito de tela interna, não de navegação pública.

17.17 Tratamento de erros no frontend

O frontend deve tratar erros de acordo com o status HTTP e o corpo padronizado retornado pela API.

Interpretação recomendada:

- `400`: erro de validação ou parâmetro inválido;
- `401`: usuário não autenticado ou token inválido;
- `403`: usuário autenticado sem permissão;
- `404`: recurso não encontrado;
- `409`: conflito, duplicidade ou recurso em uso;
- `422`: regra de negócio violada.

Quando o erro for de validação, o frontend deve procurar mensagens por campo.

Quando for regra de negócio, deve exibir a mensagem geral do backend.

A regra é:

Usar status HTTP para decidir o comportamento da interface e mensagem do backend para orientar o usuário.

17.18 Mensagens de erro

As mensagens do backend foram revisadas para português brasileiro e devem ser aproveitadas pelo frontend sempre que possível.

Exemplos de uso:

- mostrar toast para regra de negócio;
- mostrar mensagem próxima ao campo em erro de validação;

- mostrar acesso negado em 403;
- pedir login novamente em 401;
- mostrar mensagem amigável quando recurso não for encontrado.

O frontend não precisa reescrever todas as mensagens do backend, mas pode adaptar quando for necessário para melhorar a UX.

O importante é não ignorar a mensagem retornada pela API.

17.19 Fluxo de cadastro e login

Um fluxo comum do frontend será cadastro seguido de login.

O cadastro público cria o usuário.

Depois, o usuário deve fazer login para obter o token.

A API também trabalha com perfis. No estado final, o usuário pode ter perfil candidato, perfil recrutador ou ambos, conforme os fluxos disponíveis.

A regra para o frontend é:

Cadastro cria a conta. Login gera o token. Perfis definem o que o usuário pode fazer.

Se o usuário tentar acessar uma funcionalidade de recrutador sem ter perfil recrutador, a API pode retornar erro de regra de negócio ou permissão conforme o fluxo.

17.20 Fluxo do candidato

O frontend da área do candidato deve usar endpoints autenticados.

Funcionalidades principais:

- visualizar dados do usuário;
- gerenciar perfil candidato;
- gerenciar experiências profissionais;
- demonstrar interesse em vagas;

- remover interesse;
- listar vagas de interesse;
- consultar indicações dadas e recebidas.

A regra principal é:

O candidato não precisa informar seu próprio ID. O token define o usuário.

Isso deve simplificar bastante as telas.

17.21 Fluxo do recrutador

O frontend da área do recrutador também deve usar endpoints autenticados.

Funcionalidades principais:

- consultar perfil recrutador;
- criar ou gerenciar perfil recrutador;
- solicitar vínculo com empresa;
- visualizar empresa vinculada;
- listar próprias oportunidades;
- criar oportunidade;
- atualizar oportunidade;
- excluir oportunidade;
- publicar como empresa quando tiver vínculo aprovado.

A regra principal é:

Recrutador não informa `recrutadorId` para operar suas próprias oportunidades.

O backend identifica o recrutador pelo token.

17.22 Fluxo de empresa no frontend

O frontend pode ter telas públicas e internas relacionadas a empresa.

Telas públicas usam:

- listagem de empresas;
- detalhe de empresa;
- oportunidades da empresa;
- recrutadores públicos da empresa.

Essas telas só exibem empresas com localidade validada.

Telas internas podem permitir:

- cadastrar empresa;
- solicitar vínculo;
- acompanhar vínculo;
- visualizar status de localidade, quando disponível.

A regra é:

Empresa pública é empresa validada. Empresa interna pode estar em processamento.

17.23 Fluxo de oportunidade no frontend

O frontend deve separar oportunidade pública de oportunidade interna.

Na área pública:

- listar vagas;
- filtrar vagas;
- ver detalhe de vaga;
- demonstrar interesse, se usuário estiver autenticado como candidato.

Na área do recrutador:

- listar próprias vagas;
- criar vaga;
- editar vaga;
- excluir vaga;

- ver status de localidade;
- identificar vagas ainda não públicas.

A regra é:

A mesma vaga pode ter visibilidade diferente dependendo do endpoint.

Isso é especialmente importante por causa de LocalidadePendente.

17.24 Fluxo admin no frontend

O frontend admin deve consumir endpoints `/api/admin/**`.

Telas possíveis:

- gestão de usuários;
- gestão de experiências;
- moderação de tipos de emprego;
- aprovação de vínculos recrutador-empresa;
- aprovação em lote de tipos de emprego.

A interface deve considerar:

- usuário sem token recebe 401;
- usuário comum recebe 403;
- admin pode acessar;
- listagens admin geralmente são paginadas;
- filtros admin devem ser enviados por query param.

A regra é:

Admin é um contexto separado, não uma variação da tela comum.

17.25 Uso do Swagger pelo frontend

O Swagger deve ser usado como referência formal dos contratos.

A documentação histórica explica regras e contexto. O Swagger mostra os detalhes exatos de endpoint, parâmetros, DTOs, responses e status.

A pessoa do frontend deve consultar o Swagger para confirmar:

- nome correto dos campos;
- quais filtros existem;
- se o endpoint retorna **Page** ou **List**;
- quais parâmetros são obrigatórios;
- quais status são documentados;
- qual DTO é retornado.

A regra é:

Esta documentação explica o porquê. O Swagger confirma o contrato exato.

17.26 Uso da coleção Postman pelo frontend

A coleção Postman serve como apoio para testar os fluxos antes ou durante a implementação da interface.

Ela ajuda principalmente a:

- validar login;
- capturar token;
- testar endpoints protegidos;
- testar endpoints públicos;
- testar permissões admin;
- verificar exemplos de payload;
- entender sequência de chamadas.

A coleção usa variável de ambiente para **base_url** e **jwt_token**.

A regra é:

Antes de implementar uma tela complexa, valide o fluxo no Postman.

Isso reduz confusão entre erro de integração e erro de interface.

17.27 Cuidados importantes para o frontend

Alguns cuidados devem ser preservados na integração.

Primeiro, não assumir que todo recurso criado aparece imediatamente no público. Localidade pendente pode impedir a exposição.

Segundo, não usar endpoints admin para telas comuns.

Terceiro, não guardar lógica de permissão apenas no frontend. O backend continua validando.

Quarto, não filtrar client-side grandes listagens quando houver filtro backend.

Quinto, não tratar `Page` como array.

Sexto, não pedir `usuarioId` em fluxo `/me`.

Sétimo, não prometer funcionalidades futuras que ainda não existem, como confirmação manual de localidade, notificação de pendência ou ownership formal de empresa.

O frontend deve refletir o que a API entrega hoje.

17.28 Síntese

A integração com o frontend deve respeitar a divisão central da API:

- público;
- `/me`;
- admin.

Endpoints públicos mostram apenas dados prontos para publicação. Endpoints `/me` mostram dados do usuário autenticado e podem incluir estados internos. Endpoints admin servem para moderação e gestão.

A regra mais importante para o frontend é:

Não interpretar ausência no público como erro automático. A API pode ocultar recursos por regra de LocalidadePendente.

A segunda regra mais importante é:

Use o token para fluxos pessoais. Não use `usuarioId` quando existir endpoint `/me`.

Seguindo essas duas ideias, a integração tende a ficar mais segura, coerente e alinhada com as decisões do backend.

18. Contratos importantes para telas

Esta seção organiza os principais contratos da API do ponto de vista de telas do frontend. O objetivo não é substituir o Swagger/OpenAPI, mas ajudar a pessoa responsável pela interface a entender quais endpoints costumam alimentar cada tipo de tela, quais respostas esperar e quais regras de negócio precisam ser consideradas.

A API possui três grandes contextos de consumo:

- telas públicas;
- telas autenticadas do usuário comum;
- telas administrativas.

Essa divisão é mais importante do que decorar endpoints isolados. A mesma entidade pode aparecer de formas diferentes dependendo do contexto. Uma oportunidade pública, por exemplo, não expõe detalhes de pendência de localidade. Já uma oportunidade retornada no painel do recrutador pode trazer status interno de localidade, porque o dono precisa entender por que a vaga ainda não está pública.

A regra geral é:

O frontend deve escolher o endpoint conforme o contexto da tela: público, `/me` ou admin.

18.1 Tela de login

A tela de login deve usar o endpoint de autenticação da API.

Esse fluxo recebe credenciais do usuário e retorna o token JWT no header **Authorization**.

A regra importante é:

O token não vem no corpo da resposta. Ele vem no header **Authorization**.

Após o login, o frontend deve armazenar o token conforme a estratégia da aplicação e enviá-lo em endpoints protegidos como Bearer Token.

A tela de login deve tratar especialmente dois cenários:

- credenciais inválidas;
- usuário autenticado com sucesso.

Em caso de erro de autenticação, a API deve retornar status 401. O frontend pode exibir uma mensagem como “E-mail ou senha inválidos”, preferencialmente usando a mensagem retornada pelo backend.

18.2 Tela de cadastro de usuário

A tela de cadastro público deve usar o endpoint de criação de usuário.

Esse fluxo não exige autenticação, porque é o ponto de entrada de uma nova pessoa na plataforma.

A tela deve coletar os dados necessários do usuário e enviar para a API conforme o DTO documentado no Swagger.

Após cadastro bem-sucedido, o frontend pode direcionar o usuário para login ou, se futuramente existir login automático, esse fluxo deve ser implementado de forma explícita.

A regra importante é:

Cadastro cria a conta. Login gera o token.

A criação de usuário não deve depender de perfil admin. O frontend também não deve permitir que o usuário escolha manualmente uma role administrativa. Permissões administrativas pertencem ao backend e aos fluxos de gestão.

18.3 Tela de perfil do usuário autenticado

A tela de perfil do usuário deve usar endpoints `/me`.

Essa tela representa os dados da pessoa logada. Portanto, o frontend não deve buscar ou enviar `usuarioId` manualmente para consultar ou atualizar o próprio perfil.

A API deve identificar o usuário pelo token JWT.

Essa tela pode envolver:

- consultar dados do usuário autenticado;
- atualizar dados pessoais;
- alterar senha;
- remover conta, se o fluxo estiver disponível;
- consultar quais perfis o usuário possui.

A regra importante é:

Tela de perfil pessoal usa `/me`, não ID de usuário.

Isso evita risco de IDOR e simplifica a integração.

18.4 Tela pública de oportunidades

A tela pública de oportunidades deve usar a listagem pública de vagas.

Esse endpoint retorna oportunidades disponíveis para navegação geral e deve ser paginado.

A resposta paginada deve ser consumida a partir do campo `content`, junto com os metadados de paginação.

A tela pode oferecer filtros como:

- termo de busca;
- tipo de emprego;
- empresa;
- recrutador;
- país;
- estado;
- cidade;
- modalidade.

A regra mais importante dessa tela é:

A listagem pública de oportunidades retorna apenas vagas com Localidade validada.

Isso significa que o frontend não precisa esconder vagas pendentes nessa tela. O backend já deve garantir que elas não venham na resposta.

Se uma vaga criada por recrutador não aparece nessa tela, isso pode ser efeito da regra de localidade pendente, e não necessariamente erro de cadastro.

18.5 Tela de detalhe público da oportunidade

A tela de detalhe público de oportunidade deve usar o endpoint público de consulta por ID.

Mesmo que uma oportunidade exista internamente, ela só deve ser retornada nesse endpoint se estiver publicamente disponível.

A regra importante é:

Existir no banco não significa estar disponível publicamente.

Uma oportunidade com LocalidadePendente pode existir e aparecer no painel do recrutador, mas não deve aparecer no detalhe público.

Para o frontend, se o endpoint retornar 404, a tela deve tratar como oportunidade não encontrada ou indisponível publicamente.

18.6 Tela pública de empresas

A tela pública de empresas deve usar a listagem pública de empresas.

Esse endpoint retorna empresas publicamente visíveis e deve ser paginado.

Filtros esperados podem incluir:

- termo;
- país;
- estado;
- cidade.

A regra central é:

Empresa pública precisa ter Localidade validada.

Empresas com localidade pendente podem existir internamente, mas não devem aparecer na listagem pública.

O frontend não deve exibir empresas pendentes nessa tela, nem precisa filtrar isso manualmente.

18.7 Tela de detalhe público da empresa

A tela de detalhe público da empresa deve usar o endpoint público de consulta por ID.

Essa tela deve exibir apenas dados publicáveis da empresa.

O DTO público de empresa não deve conter detalhes técnicos de LocalidadePendente.

A regra é:

Detalhe público da empresa representa uma empresa validada e apta para exibição pública.

Se a empresa existir internamente, mas ainda estiver pendente por localidade, a consulta pública pode retornar não encontrado ou indisponível, conforme o comportamento da API.

18.8 Tela de oportunidades de uma empresa

A tela de oportunidades de uma empresa deve usar o endpoint público de oportunidades por empresa.

Esse endpoint é paginado e filtrável.

O ID da empresa vem do path. O frontend não deve tentar controlar o escopo da empresa por query param.

A regra é:

Em oportunidades de uma empresa, a empresa é definida pela URL, não pelo filtro.

Além disso, tanto a empresa quanto as oportunidades retornadas precisam estar publicamente válidas.

Isso significa que a tela não deve esperar vagas pendentes nesse endpoint.

18.9 Tela de recrutadores de uma empresa

A tela pública de recrutadores de uma empresa deve exibir apenas recrutadores com vínculo aprovado.

Recrutadores com vínculo pendente ou recusado não devem aparecer como representantes públicos da empresa.

A regra é:

Público vê apenas vínculos aprovados.

Para o frontend, isso significa que a lista de recrutadores da empresa deve ser tratada como uma lista de representantes já validados pelo fluxo administrativo.

18.10 Tela do painel do recrutador

O painel do recrutador deve usar endpoints autenticados.

Essa tela é diferente da listagem pública. Ela deve mostrar recursos do próprio recrutador, inclusive aqueles que ainda não estão disponíveis publicamente.

O endpoint principal é a listagem de oportunidades do recrutador autenticado.

Essa listagem é paginada, filtrável e pode incluir status de localidade.

A regra principal é:

O recrutador deve enxergar suas próprias oportunidades, inclusive pendentes.

Esse comportamento permite que o frontend informe ao recrutador que determinada vaga foi salva, mas ainda não aparece no público porque a localidade está em validação.

18.11 Tela de criação de oportunidade

A tela de criação de oportunidade deve ser usada por usuário autenticado com perfil de recrutador.

O frontend não deve enviar `recrutadorId`.

O backend identifica o recrutador pelo token.

A tela deve permitir informar:

- descrição ou dados principais da vaga;
- tipo de emprego aprovado;
- modalidade;

- localidade;
- opção de publicar como empresa, se aplicável.

A regra principal é:

Oportunidade nasce no contexto do recrutador autenticado.

Se a tela permitir **publicarComoEmpresa**, o frontend deve entender que essa opção só faz sentido quando o recrutador tem vínculo aprovado com uma empresa.

Mesmo que o frontend esconda a opção quando não houver vínculo, o backend continuará validando essa regra.

18.12 Tela de edição de oportunidade

A tela de edição de oportunidade deve operar sobre oportunidade do recrutador autenticado.

A API deve validar que a vaga pertence ao recrutador antes de permitir atualização.

Para o frontend, a regra é:

A edição deve ser feita a partir do painel do recrutador, não da tela pública da vaga.

Também é importante lembrar que mudar o contexto de publicação da vaga não deve ser tratado como uma simples alteração visual. Publicar como empresa envolve regra de vínculo aprovado.

18.13 Tela de empresa vinculada ao recrutador

O recrutador pode ter vínculo com uma empresa.

A tela interna do recrutador pode precisar mostrar:

- se existe empresa vinculada;
- se o vínculo está aprovado;
- se o vínculo está pendente;
- se o vínculo foi recusado;

- se a empresa está com localidade validada ou pendente.

A regra é:

Vínculo com empresa e visibilidade pública da empresa são coisas diferentes.

Um recrutador pode ter algum estado de vínculo com empresa, mas a empresa só aparece publicamente se também cumprir as regras de visibilidade, especialmente Localidade validada.

18.14 Tela de solicitação de vínculo com empresa

Essa tela permite que o recrutador solicite vínculo com uma empresa existente.

O frontend deve permitir selecionar uma empresa pública ou informar o ID da empresa conforme o fluxo disponível.

A regra principal é:

Solicitar vínculo não significa vínculo aprovado.

Depois da solicitação, a aprovação depende do fluxo administrativo.

O frontend não deve tratar solicitação como permissão imediata para publicar em nome da empresa.

18.15 Tela do candidato: interesses em vagas

A tela de interesses do candidato deve usar endpoint `/me`.

Ela deve listar as vagas em que o candidato autenticado demonstrou interesse.

Essa listagem é paginada e filtrável.

A regra principal é:

O candidato lista apenas os próprios interesses.

O frontend não deve enviar `usuarioId` para buscar interesses pessoais.

Também deve tratar erros de regra de negócio, como interesse duplicado ou remoção de interesse inexistente.

18.16 Ação de demonstrar interesse em vaga

A ação de demonstrar interesse deve ser feita pelo candidato autenticado.

O frontend deve chamar o endpoint correspondente usando o ID da vaga.

O backend identifica o candidato pelo token.

A regra é:

Interesse em vaga pertence ao candidato autenticado.

Se o usuário tentar demonstrar interesse novamente na mesma vaga, a API deve bloquear duplicidade e retornar erro de regra de negócio.

O frontend deve exibir a mensagem retornada pelo backend.

18.17 Ação de remover interesse em vaga

A remoção de interesse também deve ser feita pelo candidato autenticado.

O frontend informa a vaga, e o backend identifica o candidato pelo token.

Se o interesse não existir, a API pode retornar erro de regra de negócio.

A regra é:

O candidato só remove interesse próprio.

Essa ação não deve depender de `usuarioId`.

18.18 Tela de experiências profissionais do candidato

A tela de experiências profissionais do candidato deve usar endpoints `/me`.

Ela deve permitir:

- listar experiências;
- criar experiência;
- editar experiência;
- remover experiência.

A regra principal é:

Experiência profissional pertence ao candidato autenticado.

O frontend não deve enviar ID de usuário para gerenciar experiências pessoais.

Quando houver erro de validação, como campo obrigatório ausente ou data inválida, a tela deve usar as mensagens retornadas pelo backend.

18.19 Tela de indicações

As telas de indicação devem ser autenticadas.

O usuário pode visualizar:

- indicações dadas;
- indicações recebidas.

Também pode criar indicação para outro usuário, conforme o contrato disponível.

As regras principais são:

Usuário não pode indicar a si mesmo.

Exclusão de indicação deve respeitar autoria.

As listagens de indicações continuam como **List** no MVP, porque são fluxos pessoais menores.

Se o volume crescer, elas podem ser paginadas futuramente.

18.20 Tela de tipos de emprego aprovados

O frontend deve usar o catálogo de tipos aprovados em telas comuns.

Esse catálogo serve para:

- filtros de vagas;
- formulário de criação de oportunidade;
- formulário de experiência profissional;
- futuras preferências do candidato.

A regra é:

Telas públicas e formulários comuns usam apenas tipos aprovados.

Tipos não aprovados pertencem ao fluxo administrativo de curadoria.

18.21 Tela de sugestão de tipo de emprego

Quando o usuário não encontra um tipo de emprego adequado, pode existir fluxo de sugestão.

A sugestão não entra automaticamente no catálogo público.

A regra é:

Tipo sugerido nasce como não aprovado e depende de curadoria admin.

O frontend deve comunicar isso com clareza, evitando dar a entender que a sugestão ficará disponível imediatamente para todos.

18.22 Tela admin de tipos não aprovados

A tela admin de tipos não aprovados deve consumir endpoint administrativo paginado.

Ela serve para moderação do catálogo.

A regra principal é:

O backend impõe `aprovado=false` nesse endpoint.

O frontend pode filtrar por termo, navegar por páginas e acionar aprovação individual ou em lote.

A aprovação em lote usa uma lista simples de IDs, não objetos complexos.

18.23 Tela admin de usuários

A tela admin de usuários deve usar endpoint administrativo paginado.

Ela pode permitir filtros como:

- termo;
- nome;
- e-mail;
- CPF;
- existência de perfil candidato;
- existência de perfil recrutador.

Essa tela exige permissão admin.

A regra é:

Gestão global de usuários pertence ao contexto admin, não ao contexto `/me`.

18.24 Tela admin de experiências

A tela admin de experiências deve usar endpoint administrativo paginado e filtrável.

Ela permite visão global sobre experiências profissionais.

Filtros podem envolver:

- termo;
- tipo de emprego;
- experiência em andamento;
- datas.

Essa tela é diferente da tela pessoal de experiências do candidato.

A regra é:

Tela pessoal usa `/me`. Tela global usa `admin`.

18.25 Tela admin de vínculos recrutador-empresa

A tela admin de vínculos deve permitir aprovar ou recusar solicitações de recrutadores.

Esse fluxo é importante porque publicação em nome de empresa depende de vínculo aprovado.

A regra é:

Sem vínculo aprovado, recrutador não deve publicar em nome da empresa.

O frontend admin deve deixar clara a diferença entre solicitação, aprovação e recusa.

18.26 Tela de Localidade

Os endpoints de localidade funcionam como catálogo ou autocomplete.

Eles podem ser usados em formulários e filtros.

O fluxo recomendado é:

- carregar países;
- carregar estados a partir do país;
- carregar cidades a partir do estado;
- enviar IDs estruturados nos formulários ou filtros.

A regra é:

Sempre que possível, o frontend deve usar IDs de Localidade em vez de texto livre.

Isso reduz chance de pendência e melhora a qualidade dos filtros.

18.27 Telas que precisam lidar com LocalidadePendente

Telas públicas não devem lidar com LocalidadePendente porque a API não deve retornar recursos pendentes nelas.

As telas internas que podem precisar lidar com esse estado são principalmente:

- painel do recrutador;
- detalhe interno de oportunidade;
- dados internos de empresa;
- talvez painel de candidato em casos de vaga de interesse;
- futura tela admin de pendências, se implementada.

A regra de UX sugerida é:

Informar que o recurso foi salvo, mas ainda aguarda validação da localidade antes de aparecer publicamente.

Não tratar esse estado como erro fatal.

18.28 Contratos paginados

Telas que usam endpoints paginados devem tratar a resposta como página, não como array.

A lista está em `content`.

Os metadados devem ser usados para navegação.

Telas paginadas incluem:

- listagem pública de oportunidades;
- listagem pública de empresas;
- oportunidades de uma empresa;
- oportunidades do recrutador;
- vagas de interesse do candidato;
- usuários admin;
- experiências admin;

- tipos não aprovados admin.

A regra é:

Endpoint paginado alimenta componente de tabela/listagem com controle de página, tamanho e filtros.

18.29 Contratos não paginados

Alguns endpoints continuam retornando lista simples.

Isso é esperado no MVP.

Exemplos:

- localidades;
- tipos aprovados;
- experiências pessoais, se mantidas como lista;
- indicações dadas e recebidas;
- recrutadores públicos de uma empresa.

A regra é:

Nem toda lista é Page. O frontend deve consultar o Swagger e tratar cada contrato conforme o endpoint.

Essa diferença é importante para evitar erro de integração.

18.30 Como decidir qual endpoint usar

Quando houver dúvida, o frontend deve responder três perguntas.

Primeira: a tela é pública?

Se sim, usar endpoint público.

Segunda: a tela pertence ao usuário logado?

Se sim, usar endpoint `/me`.

Terceira: a tela é de gestão global ou moderação?

Se sim, usar endpoint admin.

Essa regra simples evita a maior parte das confusões.

Público mostra dados prontos para qualquer pessoa.

`/me` mostra dados do usuário autenticado.

Admin gerencia dados globais e moderação.

18.31 Síntese

Os contratos da API devem ser entendidos pelo contexto da tela.

A mesma entidade pode aparecer em formatos diferentes dependendo de onde está sendo usada. Empresa pública é diferente de empresa interna. Oportunidade pública é diferente de oportunidade do recrutador. Lista pública é diferente de lista admin. Dados pessoais usam `/me`.

A regra mais importante para integração é:

Escolha o endpoint pelo contexto da tela, não apenas pelo nome da entidade.

Essa decisão evita uso incorreto de endpoints, reduz risco de segurança e mantém o frontend alinhado com as regras de negócio do backend.

19. Decisões de MVP e trade-offs

Durante a evolução da API **Quero Trabalhar Backend**, algumas decisões foram tomadas com foco em entregar um MVP funcional, seguro e compreensível, sem tentar resolver todos os problemas possíveis de uma plataforma completa de empregabilidade.

Essas decisões são importantes porque explicam por que algumas funcionalidades foram implementadas de determinada forma e por que outras foram deixadas para depois. Nem tudo que ficou fora do MVP representa esquecimento ou falha. Em

muitos casos, foi uma escolha consciente para manter o escopo viável e evitar complexidade prematura.

A regra geral desta seção é:

O MVP priorizou segurança, coerência de domínio, contratos claros e visibilidade pública confiável. Funcionalidades mais complexas de operação, governança e refinamento foram documentadas como evolução futura.

19.1 O que significa MVP neste projeto

Neste projeto, MVP não significa uma API incompleta ou improvisada. Significa uma versão suficientemente organizada para demonstrar o fluxo principal do produto com segurança, documentação e regras de negócio coerentes.

O foco do MVP foi consolidar os fluxos centrais:

- cadastro e autenticação de usuários;
- perfis de candidato e recrutador;
- cadastro e consulta de empresas;
- publicação e consulta de oportunidades;
- interesse do candidato em vagas;
- experiências profissionais;
- indicações;
- catálogo moderado de tipos de emprego;
- localidade validada;
- ocultação pública de recursos pendentes;
- paginação e filtros nas listagens principais;
- documentação Swagger/OpenAPI;
- Dockerização e apoio à execução;
- testes para fluxos relevantes.

Ao mesmo tempo, algumas funcionalidades mais avançadas foram deixadas para pós-MVP, como ownership formal de empresa, confirmação manual de localidade, notificação de pendência e revogação de token.

Essa separação foi importante para evitar que o projeto crescesse demais antes de estabilizar sua base principal.

19.2 Prioridade: segurança antes de conveniência

Uma das principais decisões do MVP foi priorizar segurança e coerência do domínio, mesmo que isso exigisse mais trabalho de modelagem.

O exemplo mais claro foi a adoção de endpoints `/me`.

No estado inicial, alguns fluxos recebiam IDs de usuário no path. Isso era conveniente, mas criava risco de IDOR. No estado final, as ações pessoais passaram a derivar o usuário autenticado a partir do token JWT.

Essa decisão impactou candidato, recrutador, experiências, interesses, indicações e oportunidades.

O trade-off foi:

- menos liberdade para o frontend montar URLs com IDs;
- mais segurança e clareza de ownership no backend.

A decisão final foi correta para o MVP:

Em ações pessoais, segurança vale mais do que conveniência de contrato.

19.3 LocalidadePendente sem confirmação manual no MVP

Uma decisão importante foi implementar `LocalidadePendente` como fila técnica de resolução, mas não implementar ainda a confirmação manual de localidade sugerida pelo usuário.

A confirmação manual seria útil quando a API encontrasse uma possível localidade, mas precisasse que o usuário confirmasse se aquela sugestão está correta. Porém, esse fluxo exigiria novas telas, novos endpoints, estados adicionais, regras de expiração, UX de confirmação e possivelmente notificação.

Para o MVP, a decisão foi:

- permitir salvar o recurso internamente;
- registrar a pendência;
- ocultar o recurso do público;
- tentar reprocessar em segundo plano;
- deixar confirmação manual para uma etapa futura.

O trade-off foi:

- o sistema ainda não oferece ao usuário uma ação direta para confirmar uma sugestão;
- por outro lado, o MVP evita publicar localidade errada e evita bloquear todo cadastro.

A regra consolidada foi:

Ambiguidade não deve ser resolvida automaticamente. Confirmação manual fica para pós-MVP.

19.4 Ocultar recurso pendente em vez de bloquear todo cadastro

Outra decisão importante foi não bloquear necessariamente o cadastro quando a localidade não puder ser validada.

A alternativa mais rígida seria rejeitar qualquer empresa ou oportunidade cuja localidade não fosse resolvida imediatamente. Isso garantiria que só entrassem dados validados, mas criaria fricção alta para o usuário e tornaria o sistema dependente demais da base interna ou de serviços externos.

A alternativa oposta seria aceitar tudo e publicar mesmo com localidade incerta. Isso melhoraria a fluidez do cadastro, mas comprometeria a qualidade das listagens públicas.

A solução escolhida foi intermediária:

- salvar o recurso internamente;
- criar **LocalidadePendente**;

- não exibir publicamente;
- reprocessar depois.

O trade-off foi:

- o usuário pode criar um recurso que não aparece imediatamente no público;
- mas a API preserva a qualidade pública e não perde o cadastro.

Essa é uma das decisões mais importantes do MVP.

19.5 Sem notificação automática de localidade pendente

Foi reconhecido que, quando uma empresa ou oportunidade fica oculta por localidade pendente, o usuário deveria idealmente ser informado.

No entanto, implementar notificação exigiria decisões adicionais:

- canal de notificação;
- e-mail ou notificação interna;
- template;
- disparo síncrono ou assíncrono;
- retry;
- registro de envio;
- preferências do usuário;
- infraestrutura de e-mail.

Para o MVP, essa frente foi deixada para depois.

A decisão foi:

O backend registra a pendência e permite que endpoints internos mostrem status, mas não envia notificação automática no MVP.

O trade-off foi:

- a comunicação depende da tela interna do frontend;
- mas o backend não assume uma infraestrutura de notificação antes da hora.

19.6 Sem ownership formal de Empresa no MVP

A modelagem de empresa evoluiu bastante: empresa deixou de ser texto e virou entidade, recrutadores podem solicitar vínculo e admins podem aprovar ou recusar.

Porém, ownership formal de empresa ficou para pós-MVP.

Isso significa que, no MVP, existe vínculo entre recrutador e empresa, mas ainda não existe uma modelagem completa de:

- dono da empresa;
- administradores da empresa;
- conta empresarial;
- login por CNPJ;
- gestão de permissões internas da empresa;
- transferência de ownership;
- convite de recrutadores por um dono.

Essa decisão foi tomada porque ownership empresarial exigiria uma frente grande de produto e segurança.

O trade-off foi:

- a empresa já funciona como entidade pública e pode ter recrutadores vinculados;
- mas a governança completa da empresa ainda não existe.

A regra atual é:

Recrutador vinculado não é automaticamente dono da empresa.

Essa distinção deve ser preservada até que a issue de ownership seja implementada.

19.7 Vínculo moderado em vez de conta empresarial

Relacionado ao ponto anterior, o MVP escolheu um modelo mais simples: vínculo moderado entre recrutador e empresa.

Nesse modelo:

- empresa existe como entidade;
- recrutador solicita vínculo;
- admin aprova ou recusa;
- recrutador aprovado pode publicar em nome da empresa.

A alternativa seria criar conta empresarial, login por CNPJ, donos e administradores. Isso seria mais completo, mas também muito mais complexo.

A decisão de MVP foi:

Para validar o fluxo principal, vínculo moderado é suficiente. Conta empresarial fica para depois.

Esse trade-off permitiu entregar publicação como empresa sem abrir toda a complexidade de governança empresarial.

19.8 Page direto do Spring aceito temporariamente

Na frente de paginação, a API passou a retornar `Page` diretamente.

Essa decisão acelerou a entrega e manteve consistência com o padrão Spring usado no backend. Porém, existe um trade-off conhecido: a serialização direta de `PageImpl` pode não ser o contrato mais estável para longo prazo.

O próprio diagnóstico final registrou esse ponto como limitação conhecida do MVP.

A decisão foi:

Para o MVP, usar `Page` direto é aceitável. Futuramente, pode-se criar um DTO próprio de resposta paginada.

O trade-off foi:

- menos código agora;
- contrato de paginação menos controlado para evolução futura.

Se o frontend já estiver integrado ao formato atual, uma mudança futura deve ser planejada com cuidado.

19.9 Nem todos os endpoints foram paginados

Outra decisão foi não paginar todos os endpoints.

Foram priorizadas listagens com maior risco de crescimento ou necessidade de navegação:

- oportunidades;
- empresas;
- oportunidades da empresa;
- oportunidades do recrutador;
- interesses do candidato;
- usuários admin;
- experiências admin;
- tipos não aprovados.

Alguns endpoints continuaram retornando `List`, como catálogos, autocompletes e listas pessoais menores.

O trade-off foi:

- menos padronização absoluta;
- mais simplicidade onde paginação ainda não era necessária.

A decisão foi consciente:

Paginar onde há necessidade real. Manter `List` onde o volume esperado é pequeno ou o endpoint funciona como catálogo simples.

Isso pode ser revisado se o volume crescer.

19.10 DTO interno ainda expõe alguns detalhes técnicos

No MVP, DTOs públicos foram protegidos para não expor `LocalidadePendente`.

Porém, DTOs internos ainda podem expor informações como:

- status de localidade;
- texto original;
- motivo da pendência;
- status técnico de validação.

Essa exposição foi aceita porque ajuda o usuário autenticado a entender o estado do próprio recurso.

O trade-off foi:

- o frontend interno recebe informações úteis;
- mas alguns detalhes ainda têm caráter técnico e podem ser refinados futuramente.

A decisão atual é:

DTO público não expõe pendência. DTO interno pode expor status de localidade quando isso ajuda o dono do recurso.

No futuro, pode ser interessante transformar esses detalhes técnicos em mensagens mais orientadas à experiência do usuário.

19.11 Estados futuros em `StatusValidacaoLocalidade`

Durante a documentação, foi observado que `StatusValidacaoLocalidade` possui estados que apontam para evolução futura, como confirmação ou recusa manual pelo usuário.

Em vez de remover esses estados ou alterar o contrato, a decisão foi documentar explicitamente que alguns deles são reservados para pós-MVP.

O trade-off foi:

- manter compatibilidade e intenção futura;
- evitar que o Swagger pareça prometer fluxo que ainda não existe.

A regra é:

Estados reservados devem ser documentados como evolução futura, não como funcionalidade já disponível.

Isso evita confusão para frontend e próximos desenvolvedores.

19.12 Google Maps como detalhe interno, não contrato público

A integração com Google Maps foi encapsulada dentro do fluxo de resolução de localidade.

A alternativa seria expor endpoints ou comportamentos diretamente dependentes do Google Maps. Isso poderia até facilitar alguns testes manuais, mas aumentaria acoplamento e faria a API pública depender de uma integração externa.

A decisão foi:

Google Maps é fallback interno de resolução, não contrato público.

O trade-off foi:

- menos exposição direta da integração;
- mais controle arquitetural;
- menor acoplamento do frontend;
- maior responsabilidade do backend em resolver e validar.

Essa decisão deve ser preservada.

19.13 Ambiguidade tratada com cautela

Quando há mais de uma localidade possível, a API não escolhe automaticamente.

Essa decisão evita publicar recurso em localidade errada.

O trade-off foi:

- menos automação imediata;
- mais segurança e confiabilidade.

Para o MVP, a decisão foi:

Melhor manter pendente do que validar errado.

Essa regra é especialmente importante porque localidade controla visibilidade pública.

19.14 TipoDeEmprego com curadoria em vez de criação livre

Outro trade-off importante foi tratar **TipoDeEmprego** como catálogo moderado.

Permitir criação livre e uso imediato seria mais simples, mas prejudicaria a qualidade do catálogo público.

A decisão foi:

- usuários podem sugerir tipos;
- sugestões entram como não aprovadas;
- admin aprova;
- público consome apenas aprovados;
- oportunidades usam tipos aprovados.

O trade-off foi:

- mais trabalho administrativo;
- catálogo mais limpo e confiável.

Para o MVP, essa decisão protege filtros, formulários e listagens públicas.

19.15 Swagger local e não exposto em todos os ambientes

A documentação Swagger/OpenAPI foi configurada para ajudar desenvolvimento e integração, mas não ficar exposta indiscriminadamente.

A decisão foi manter Swagger disponível no ambiente local e desabilitado em test/prod.

O trade-off foi:

- facilidade para desenvolvimento local;
- menor exposição em ambientes sensíveis.

A regra é:

Swagger é ferramenta de desenvolvimento e integração, não necessariamente recurso público de produção.

19.16 Testes suficientes para MVP, mas não cobertura completa

A suíte de testes evoluiu bastante em comparação com o estado inicial, mas não foi tratada como cobertura perfeita.

Foram cobertos fluxos importantes de services, controllers, security, OpenAPI, Localidade, LocalidadePendente, paginação, filtros, domínio e repository. O diagnóstico final identificou 34 classes de teste documentadas.

Ainda assim, ficaram lacunas:

- mais testes HTTP de corpos de erro;
- mais writes em controllers;
- cobertura automatizada;
- mais cenários de JWT inválido/expirado;
- testes de CORS e profiles.

O trade-off foi:

- proteger os fluxos mais críticos;
- deixar refinamento de cobertura para evolução futura.

A regra é:

O MVP precisa de testes nos pontos críticos, mas a suíte deve continuar evoluindo.

19.17 Docker como apoio ao MVP, não solução completa de produção

A dockerização foi adicionada para facilitar execução e demonstração.

Isso não significa que toda a infraestrutura de produção está resolvida.

O Docker atual ajuda a:

- rodar a API localmente;
- padronizar ambiente;
- facilitar avaliação;
- reduzir dependência de configuração manual.

Mas produção real ainda pode exigir:

- banco externo configurado;
- gestão de secrets;
- logs;
- métricas;
- healthchecks;
- pipeline de deploy;
- observabilidade;
- estratégia de backup;
- configuração de domínio e HTTPS.

O trade-off foi:

- melhorar muito a execução do MVP;
- deixar produção completa para etapa posterior.

19.18 Preferências como base, não fluxo completo

A entidade de preferências aparece no domínio, mas não foi consolidada como fluxo HTTP completo no MVP.

Essa decisão foi tomada porque outras frentes eram mais críticas: autenticação, perfis, empresa, oportunidade, localidade, paginação, erros e documentação.

O trade-off foi:

- o domínio já tem base para preferências;
- o produto ainda não oferece fluxo completo para isso.

No futuro, preferências podem ser usadas para recomendação, match, filtros personalizados e alertas.

19.19 Revogação de token ficou para pós-MVP

A API usa JWT stateless.

Foi reconhecido que revogação de token via blacklist ou estratégia equivalente é uma melhoria importante, mas ela ficou para pós-MVP.

Implementar revogação exigiria uma decisão adicional sobre armazenamento, expiração, logout, invalidação e impacto na arquitetura stateless.

O trade-off foi:

- manter autenticação simples e funcional para o MVP;
- deixar controle mais avançado de sessão para depois.

A regra atual é:

JWT resolve autenticação do MVP. Revogação/blacklist é evolução futura.

19.20 Trade-offs para o frontend

Algumas decisões afetam diretamente o frontend.

A primeira é que recurso criado pode não aparecer imediatamente no público se estiver com localidade pendente.

A segunda é que endpoints paginados retornam formato **Page** do Spring.

A terceira é que alguns endpoints retornam **List**.

A quarta é que DTO público e DTO interno podem ter campos diferentes.

A quinta é que status de localidade só faz sentido em telas internas.

A sexta é que confirmação manual de localidade ainda não existe.

A sétima é que ownership formal de empresa ainda não existe.

Esses pontos devem ser comunicados no frontend para evitar expectativas erradas.

19.21 Trade-offs para o backend

Para o próximo backend, os principais trade-offs são:

- não simplificar demais LocalidadePendente;
- não tratar ausência de ownership como bug simples;
- não expor pendentes publicamente para facilitar tela;
- não remover estados futuros sem avaliar roadmap;
- não trocar /me por IDs de usuário no path;
- não padronizar tudo como Page sem avaliar contexto;
- não implementar Google Maps direto no controller;
- não resolver ambiguidade automaticamente por conveniência.

Essas decisões foram tomadas para equilibrar segurança, escopo e qualidade do MVP.

19.22 O que ficou conscientemente para o pós-MVP

As principais evoluções futuras são:

- confirmação manual de localidade sugerida;
- notificação de recurso oculto por localidade pendente;
- ownership formal de Empresa;
- conta empresarial ou login por CNPJ, se fizer sentido;
- painel operacional de LocalidadePendente;
- estabilização do contrato JSON de paginação;
- revisão dos DTOs internos de localidade;
- revogação de token por blacklist;
- preferências como fluxo completo;
- maior cobertura de testes;
- melhoria de infraestrutura para produção.

Esses pontos devem ser tratados como roadmap, não como falha do MVP.

19.23 Síntese

As decisões de MVP da API **Quero Trabalhar Backend** foram tomadas para equilibrar entrega, segurança e coerência.

O projeto não tentou resolver todos os problemas de uma plataforma completa. Em vez disso, consolidou os fluxos essenciais e registrou claramente os limites.

As principais escolhas foram:

- validar localidade antes de expor publicamente;
- salvar pendências sem publicar;
- usar `/me` para reduzir IDOR;
- moderar tipos de emprego;
- manter empresa sem ownership formal por enquanto;
- aceitar `Page` do Spring no MVP;
- manter algumas listas simples;
- expor status técnico apenas em DTOs internos;
- documentar funcionalidades futuras sem fingir que já existem.

A regra mais importante é:

Trade-off documentado não é dívida invisível. É decisão consciente de escopo.

Essa seção deve ajudar o próximo desenvolvedor e a pessoa do frontend a entenderem o que está pronto, o que foi adiado e por que o MVP foi desenhado dessa forma.

20. Roadmap pós-MVP

O roadmap pós-MVP reúne as evoluções que foram identificadas durante a refatoração, mas que não foram implementadas na entrega atual por decisão de escopo. Esses pontos não devem ser tratados como esquecimentos. Em geral, são funcionalidades que exigem novas decisões de produto, segurança, frontend, banco de dados, infraestrutura ou experiência do usuário.

A API chegou a um estado mais organizado para MVP: possui autenticação com JWT, endpoints `/me`, administração, empresas, oportunidades, Localidade validada, LocalidadePendente, paginação, filtros, tratamento global de erros, Swagger/OpenAPI, JavaDoc técnico, Docker, testes e coleção Postman.

Mesmo assim, ainda existem frentes naturais de evolução.

A regra principal desta seção é:

O pós-MVP deve evoluir a API sem desfazer as decisões centrais do MVP: segurança por contexto autenticado, visibilidade pública apenas com Localidade validada, LocalidadePendente como fila técnica temporária e separação entre público, `/me` e admin.

20.1 Confirmação manual de localidade sugerida

A confirmação manual de localidade é uma das evoluções mais importantes para o módulo de Localidade.

Hoje, quando uma localidade não pode ser resolvida com segurança, a API pode criar uma `LocalidadePendente`. Essa pendência impede que o recurso apareça publicamente até que a localização seja resolvida.

No MVP, o sistema tenta resolver automaticamente pela base interna, pelo fallback Google Maps e pelo reprocessamento em segundo plano. Porém, quando há ambiguidade ou uma sugestão que precisa de confirmação humana, ainda não existe fluxo para o usuário escolher a localidade correta.

A evolução futura seria permitir que o sistema apresente uma sugestão ao dono do recurso e peça confirmação.

Esse fluxo poderia envolver:

- registrar sugestão encontrada;
- expor sugestão em endpoint interno;
- permitir aceitar sugestão;
- permitir recusar sugestão;

- aplicar Localidade validada após confirmação;
- remover ou finalizar a pendência;
- atualizar a visibilidade pública do recurso.

A regra mais importante é:

A API não deve escolher automaticamente uma localidade ambígua. Se houver dúvida, a decisão deve continuar pendente até confirmação segura.

Essa evolução impacta backend, frontend e UX.

20.2 Notificação de recurso oculto por localidade pendente

Outra evolução importante é notificar o usuário quando uma empresa ou oportunidade ficar oculta publicamente por causa de LocalidadePendente.

Hoje, o recurso pode ser salvo internamente e ficar fora das listagens públicas. O dono consegue visualizar esse estado em endpoints internos, mas o sistema ainda não envia uma notificação ativa.

No futuro, faria sentido informar o usuário quando isso acontecer.

Essa notificação poderia dizer, por exemplo, que o recurso foi salvo, mas ainda não aparecerá publicamente porque a localidade precisa ser validada.

Essa frente exige decisões sobre:

- canal de notificação;
- e-mail;
- notificação interna;
- template;
- momento do disparo;
- retry;
- histórico de envio;
- preferências do usuário;
- integração com serviço externo de e-mail.

A regra é:

O usuário deve ser informado sem transformar LocalidadePendente em erro fatal de cadastro.

Esse ponto é especialmente importante para o frontend, porque melhora a experiência do usuário e reduz confusão quando uma vaga criada não aparece no público.

20.3 Ownership formal de Empresa

A API atual possui empresa como entidade e vínculo moderado entre recrutador e empresa. O recrutador pode solicitar vínculo, e o admin pode aprovar ou recusar.

Porém, ainda não existe ownership formal de empresa.

Isso significa que o MVP ainda não define completamente:

- quem é o dono da empresa;
- quem pode editar dados da empresa;
- quem pode aprovar recrutadores;
- se a empresa terá administradores;
- se haverá conta empresarial;
- se haverá login por CNPJ;
- se um recrutador aprovado pode gerenciar a empresa;
- como transferir ownership;
- como remover vínculo de recrutador;
- como auditar alterações.

Essa evolução é grande porque envolve regra de negócio, segurança, UX e modelagem.

A regra atual deve ser preservada até que essa evolução seja feita:

Recrutador vinculado não é automaticamente dono da empresa.

O próximo desenvolvedor não deve assumir que vínculo aprovado equivale a permissão total sobre a empresa.

20.4 Conta empresarial ou login por CNPJ

Relacionado ao ownership formal, pode surgir a necessidade de permitir conta empresarial ou login por CNPJ.

Essa decisão ainda não foi tomada no MVP.

Há algumas possibilidades:

- manter apenas usuários pessoas físicas com perfis;
- criar conta empresarial separada;
- permitir login por CNPJ;
- associar múltiplos usuários administradores a uma empresa;
- criar convites para recrutadores;
- criar permissões internas por empresa.

Essa frente deve ser pensada com cuidado porque muda a forma como o sistema entende identidade.

Hoje, a identidade central é **Usuario**. Perfis representam papéis de candidato e recrutador.

Se uma conta empresarial for criada, será necessário decidir se ela é um novo tipo de usuário, um agregado separado, uma organização com membros ou outro modelo.

A regra é:

Login por CNPJ não deve ser adicionado como simples campo sem revisar a arquitetura de identidade e permissões.

20.5 Controle operacional avançado de LocalidadePendente

O MVP implementou LocalidadePendente como fila técnica temporária, com reprocessamento em segundo plano.

No futuro, pode ser útil criar uma visão operacional mais completa dessa fila.

Essa evolução poderia incluir:

- endpoint admin para listar pendências;
- filtros por tipo de recurso;
- filtros por data;
- filtros por motivo;
- quantidade de tentativas;
- status de processamento;
- ação manual de reprocessar;
- ação manual de descartar;
- ação manual de vincular a uma Localidade;
- histórico de resolução;
- auditoria;
- dashboard de falhas recorrentes.

Essa frente não foi priorizada no MVP porque a intenção inicial era manter LocalidadePendente como mecanismo técnico interno, não como fluxo administrativo público.

A regra deve continuar sendo:

LocalidadePendente é fila técnica. Se ganhar tela admin, ainda não deve virar recurso público normal.

20.6 Histórico terminal de pendências resolvidas

No MVP, quando uma pendência é resolvida, ela pode ser removida. Isso mantém a fila limpa e reforça a ideia de que a pendência é temporária.

Porém, essa escolha tem um trade-off: perde-se histórico detalhado do ciclo de vida da pendência.

No futuro, pode fazer sentido registrar estados terminais ou histórico de resolução.

Possíveis informações úteis:

- quando a pendência foi criada;
- quantas tentativas foram feitas;
- por qual origem foi resolvida;

- quando foi resolvida;
- qual Localidade final foi aplicada;
- se houve confirmação manual;
- se houve falha permanente;
- se o usuário foi notificado.

Essa evolução é útil para auditoria e suporte.

A regra é:

Se o produto precisar de auditoria, remover pendência resolvida pode deixar de ser suficiente.

20.7 Estabilização do contrato JSON de paginação

No MVP, os endpoints paginados retornam `Page` diretamente, usando a serialização padrão do Spring.

Isso funcionou bem para acelerar a entrega e manter consistência com Spring Data. Porém, existe uma limitação conhecida: o formato de `PageImpl` não é necessariamente o contrato ideal para longo prazo.

Uma evolução futura seria criar um DTO próprio de paginação.

Esse DTO poderia padronizar explicitamente:

- lista de itens;
- página atual;
- tamanho;
- total de elementos;
- total de páginas;
- se é primeira página;
- se é última página;
- ordenação;
- filtros aplicados, se fizer sentido.

A vantagem seria ter um contrato controlado pela aplicação, e não pela serialização interna do Spring.

Essa mudança deve ser planejada com cuidado porque impacta o frontend.

A regra é:

Se o formato de página mudar, o frontend precisa ser avisado e adaptado.

20.8 Revisão da exposição de LocalidadePendente em DTOs internos

No MVP, DTOs públicos não expõem LocalidadePendente. Isso está correto e deve ser preservado.

Porém, DTOs internos podem expor alguns detalhes técnicos, como status de localidade, texto original, motivo da pendência e status de validação.

Isso foi aceito no MVP porque ajuda o dono do recurso a entender por que uma empresa ou oportunidade ainda não aparece publicamente.

No futuro, pode ser interessante revisar essa exposição.

A evolução poderia transformar detalhes técnicos em uma estrutura mais amigável ao frontend, por exemplo:

- status simplificado;
- mensagem pronta para usuário;
- ação disponível;
- indicação se aguarda reprocessamento;
- indicação se precisa de confirmação;
- ocultação de motivos técnicos internos.

A regra é:

DTO interno pode informar o estado ao usuário, mas não precisa expor toda a mecânica técnica da fila.

Essa revisão melhora UX e reduz acoplamento do frontend a detalhes internos.

20.9 Revogação de token via blacklist

A API usa JWT stateless.

Esse modelo é simples e adequado para o MVP, mas tem uma limitação: depois que um token é emitido, ele continua válido até expirar, a menos que exista uma estratégia de revogação.

Uma evolução futura é implementar blacklist ou outro mecanismo de revogação.

Isso poderia permitir:

- logout efetivo;
- invalidação de token após troca de senha;
- invalidação de token após exclusão de conta;
- bloqueio de token comprometido;
- controle administrativo de sessão.

Essa frente exige decisões sobre:

- onde armazenar tokens revogados;
- tempo de retenção;
- impacto em performance;
- integração com filtros de segurança;
- limpeza de registros expirados;
- comportamento em ambientes distribuídos.

A regra é:

Revogação de token deve ser planejada como evolução de segurança, não adicionada de forma improvisada.

20.10 Preferências do candidato como fluxo completo

A entidade de preferências existe como base de domínio, mas não foi consolidada como fluxo completo no MVP.

No futuro, ela pode se tornar uma parte importante da experiência do candidato.

Possíveis evoluções:

- cadastro de preferências de tipo de emprego;
- preferência por modalidade;
- preferência por localidade;
- disponibilidade para trabalho remoto;
- faixa salarial, se fizer sentido;
- alertas de novas vagas;
- recomendações automáticas;
- match entre candidato e oportunidade.

Essa evolução deve usar a base já criada:

- tipos de emprego aprovados;
- localidade validada;
- perfil candidato;
- endpoints [/me](#).

A regra é:

Preferências devem ser modeladas como dados do candidato autenticado, não como filtros soltos sem ownership.

20.11 Melhorias em TipoDeEmprego

O módulo de TipoDeEmprego já possui curadoria básica: tipos aprovados, sugestões não aprovadas, aprovação individual e aprovação em lote.

Futuramente, esse módulo pode evoluir.

Possibilidades:

- recusar sugestão com motivo;
- desativar tipo em vez de excluir;
- mesclar tipos duplicados;
- registrar quem aprovou;
- registrar data de aprovação;

- criar histórico de curadoria;
- agrupar por área;
- criar hierarquia de categorias;
- impedir sugestões muito parecidas;
- melhorar busca textual.

Essas melhorias ajudam a manter o catálogo limpo conforme a plataforma cresce.

A regra é:

O catálogo de tipos de emprego deve continuar curado para não perder qualidade.

20.12 Paginação em endpoints pessoais menores

No MVP, alguns endpoints continuaram retornando **List**, especialmente listas pessoais pequenas e catálogos.

Isso é aceitável no estágio atual.

Porém, se o volume crescer, alguns endpoints podem precisar migrar para **Page**.

Candidatos naturais:

- indicações dadas;
- indicações recebidas;
- experiências pessoais;
- recrutadores de uma empresa;
- outros recursos subordinados que crescerem com o uso.

A regra é:

Se uma lista começar a crescer ou precisar de filtros, migrar para o padrão **Page + FilterDTO + Specification**.

Essa evolução deve ser coordenada com o frontend porque altera o formato da resposta.

20.13 Melhoria da cobertura de testes

A suíte de testes evoluiu bastante, mas ainda pode melhorar.

Possíveis melhorias pós-MVP:

- testes HTTP mais detalhados para corpos de erro;
- testes negativos de JWT inválido, expirado ou malformatado;
- testes de CORS;
- testes de profiles local, test e prod;
- relatório de cobertura;
- mais testes de writes em controllers;
- testes de fluxo completo para LocalidadePendente;
- testes de performance básica em listagens paginadas;
- testes de contrato para DTOs críticos.

A regra é:

Nova regra crítica deve vir acompanhada de teste.

Essa evolução aumenta segurança para futuras refatorações.

20.14 Observabilidade e monitoramento

A API já evoluiu em logs estruturados principalmente no fluxo de Localidade e Google Maps. Porém, observabilidade pode crescer bastante.

Possíveis evoluções:

- padronização de logs em outros módulos;
- métricas de reprocessamento de LocalidadePendente;
- métricas de falha do Google Maps;
- quantidade de empresas/vagas pendentes;
- tempo médio para resolução de pendência;
- métricas de autenticação;
- métricas de erros 4xx/5xx;
- integração com ferramenta de monitoramento;

- healthchecks mais completos.

A regra é:

Fluxos assíncronos e integrações externas precisam ser observáveis.

Isso é especialmente importante para LocalidadePendente, porque ela pode impactar diretamente a visibilidade pública de empresas e vagas.

20.15 Infraestrutura de produção

Docker melhora muito a execução local e a demonstração do MVP, mas produção real exige mais decisões.

Possíveis evoluções de infraestrutura:

- banco externo persistente;
- profile de produção revisado;
- secrets fora do repositório;
- variáveis de ambiente obrigatórias;
- pipeline CI/CD;
- build automatizado;
- deploy;
- HTTPS;
- logs centralizados;
- backups;
- healthchecks;
- migração de banco;
- estratégia de rollback.

A regra é:

Dockerização é passo importante, mas não substitui arquitetura completa de produção.

Antes de colocar a API em produção real, esses pontos devem ser revisados.

20.16 Migrações de banco

O projeto evoluiu muito o modelo de dados, especialmente Localidade, Empresa e perfis.

No MVP, foi possível avançar com a configuração atual de banco e testes. Porém, uma evolução natural seria adotar uma ferramenta formal de migração, como Flyway ou Liquibase.

Isso ajudaria a controlar:

- criação de tabelas;
- alteração de colunas;
- remoção de legado;
- seeds;
- versionamento de schema;
- evolução entre ambientes;
- rollback planejado.

Essa frente é especialmente relevante se o projeto sair de H2/local para banco persistente real.

A regra é:

Em ambiente persistente, evolução de schema precisa ser versionada.

20.17 Melhorias no frontend a partir das regras atuais

O backend já prepara alguns comportamentos que o frontend pode evoluir.

Possibilidades:

- painel do recrutador com filtro de vagas pendentes;
- mensagem clara para localidade em validação;
- tela para vínculo com empresa;
- tela admin para aprovar vínculos;
- tela admin para tipos não aprovados;
- filtros de oportunidade por localidade estruturada;

- uso correto de paginação;
- feedback de erros 400, 401, 403, 409 e 422;
- integração com Postman/Swagger durante desenvolvimento.

O frontend deve continuar respeitando a divisão:

- público;
- /me;
- admin.

Essa divisão deve guiar a organização das telas.

20.18 Documentação contínua

A documentação criada para o MVP não deve ser congelada.

Ela precisa acompanhar a API.

Pontos que devem ser atualizados quando houver mudança:

- README;
- Swagger/OpenAPI;
- JavaDoc técnico;
- docs/testing.md;
- coleção Postman;
- documentação histórica/evolutiva;
- roadmap.

A regra é:

Se a decisão mudar, a documentação precisa mudar junto.

Isso evita que o projeto volte a ter documentação desatualizada como no estado inicial.

20.19 Priorização sugerida pós-MVP

Uma ordem razoável para evolução pós-MVP seria:

1. confirmação manual de localidade sugerida;
2. notificação de recurso oculto por localidade pendente;
3. ownership formal de Empresa;
4. estabilização do contrato de paginação;
5. revisão de DTOs internos de localidade;
6. painel operacional de LocalidadePendente;
7. revogação de token;
8. preferências do candidato;
9. melhoria de testes e cobertura;
10. infraestrutura de produção e migrações.

Essa ordem não é obrigatória, mas reflete o impacto no produto.

Localidade e ownership de Empresa aparecem cedo porque afetam diretamente a experiência do usuário e a confiabilidade pública da plataforma.

20.20 O que não deve ser feito apressadamente

Algumas evoluções parecem simples, mas não devem ser feitas sem planejamento.

Não deve ser feito apressadamente:

- adicionar login por CNPJ como campo simples;
- permitir que qualquer recrutador edite empresa;
- publicar recurso pendente para “resolver rápido” uma tela;
- escolher localidade ambígua automaticamente;
- trocar `/me` por IDs de usuário no frontend;
- expor detalhes de LocalidadePendente em endpoints públicos;
- mudar formato de Page sem alinhar com frontend;
- remover estados de enum sem verificar contratos;
- adicionar endpoint admin sem proteger com role;
- mudar mensagens de erro sem considerar frontend.

Esses pontos tocam decisões centrais do projeto.

20.21 Síntese

O roadmap pós-MVP mostra que a API já possui uma base sólida, mas ainda tem frentes naturais de evolução.

As prioridades futuras giram em torno de cinco temas principais:

- melhorar a experiência em torno de LocalidadePendente;
- formalizar governança de Empresa;
- estabilizar contratos e DTOs;
- fortalecer segurança e testes;
- preparar infraestrutura para produção.

A regra mais importante é:

O pós-MVP deve evoluir o produto sem desfazer as decisões que tornaram o MVP mais seguro e coerente.

Isso significa preservar endpoints `/me`, visibilidade pública apenas com Localidade validada, LocalidadePendente como fila técnica, curadoria de TipoDeEmprego, contratos DTO e separação entre público, usuário autenticado e admin.

21. Guia para o próximo desenvolvedor backend

Esta seção foi escrita para a próxima pessoa que assumir a manutenção ou evolução da API **Quero Trabalhar Backend**.

A ideia aqui não é repetir toda a documentação anterior, mas destacar os principais cuidados práticos que devem ser observados antes de modificar o projeto. A API chegou ao estado atual depois de uma refatoração grande, com várias decisões de arquitetura e negócio que precisam ser preservadas para evitar regressões.

A regra geral para quem continuar o backend é:

Antes de alterar um módulo, entenda qual problema ele resolveu, qual regra de negócio ele protege e qual impacto a mudança terá no frontend, nos testes e na documentação.

21.1 Entenda a divisão principal da API

A API está organizada em três grandes contextos:

- endpoints públicos;
- endpoints `/me`;
- endpoints administrativos.

Essa divisão não é apenas estética. Ela representa regras de segurança, visibilidade e integração com frontend.

Endpoints públicos retornam apenas dados prontos para qualquer pessoa ver. Empresas e oportunidades públicas, por exemplo, só devem aparecer se tiverem Localidade validada.

Endpoints `/me` operam sobre o usuário autenticado. Eles devem usar o token JWT para descobrir quem está executando a ação, evitando que o cliente informe IDs sensíveis.

Endpoints admin ficam em `/api/admin/**` e exigem permissão administrativa. Eles são usados para gestão global, moderação e aprovação.

A primeira regra para manutenção é:

Não misture público, `/me` e admin. Cada contexto tem uma finalidade diferente.

21.2 Preserve controllers finos

Controllers devem continuar sendo a borda HTTP da aplicação.

Eles podem receber DTOs, parâmetros, paginação, validações de entrada e retornar `ResponseEntity`, mas não devem concentrar regra de negócio.

Quando uma regra começar a crescer dentro do controller, provavelmente ela deve ir para um service.

A responsabilidade do controller é:

- expor rota;
- receber request;
- validar entrada;
- delegar ao service;
- devolver response;
- documentar o endpoint.

A responsabilidade do controller não é:

- decidir ownership;
- consultar repository diretamente;
- resolver localidade;
- validar vínculo entre recrutador e empresa;
- aplicar regra de publicação;
- montar filtros complexos manualmente;
- capturar exceptions de domínio uma a uma.

A regra é:

Controller fino facilita teste, manutenção e documentação.

21.3 Coloque regra de negócio nos services

Services são o centro da regra de negócio.

Eles devem continuar responsáveis por decisões como:

- buscar o usuário autenticado;
- validar perfil candidato ou recrutador;
- validar ownership;
- impedir duplicidade;
- validar vínculo com empresa;
- validar tipo de emprego aprovado;
- acionar resolução de localidade;
- decidir se recurso pode ser público;
- lançar exception específica;
- controlar transações de escrita.

Se uma nova funcionalidade for criada, o service deve responder às perguntas de domínio.

Por exemplo:

- esse usuário pode fazer essa ação?
- esse recurso pertence a ele?
- esse vínculo está aprovado?
- esse tipo está liberado?
- essa localidade é válida?
- esse recurso pode aparecer publicamente?

A regra é:

Service protege o domínio. Controller não deve substituir essa função.

21.4 Não volte a expor entidade JPA como contrato HTTP

Uma das principais melhorias da refatoração foi separar entidade de DTO.

Entidade JPA representa persistência. DTO representa contrato HTTP.

Não retorne entidade diretamente em endpoint novo. Também não receba entidade diretamente em request body.

Isso evita:

- vazamento de campos internos;
- acoplamento do frontend ao banco;
- problemas de serialização;
- quebra de contrato quando a entidade mudar;
- exposição indevida de relacionamentos;
- confusão entre DTO público e DTO interno.

A regra é:

Novo endpoint deve ter request/response DTO quando expõe contrato HTTP relevante.

Essa regra vale especialmente para Empresa, Oportunidade, Usuário, Perfil, Localidade, Experiência e Indicação.

21.5 Preserve a lógica de /me

Endpoints `/me` existem para reduzir risco de IDOR.

Quando a ação pertence ao próprio usuário, o backend deve descobrir o usuário pelo token JWT, não por um `usuarioId` enviado pelo cliente.

Não crie endpoints pessoais novos como:

- `/api/usuarios/{usuarioId}/...`;
- `/api/candidatos/{candidatoId}/...`;
- `/api/recrutadores/{recrutadorId}/...`;

se a ação puder ser feita pelo usuário autenticado.

Prefira o padrão:

- `/api/usuarios/me`;
- `/api/candidatos/me/...`;
- `/api/recrutadores/me/...`

A regra é:

Em fluxo pessoal, identidade vem do token.

Isso deve ser preservado em experiências, interesses, oportunidades do recrutador, perfis e indicações.

21.6 Não confie apenas no frontend para segurança

O frontend pode esconder botões, telas e ações, mas isso não substitui validação no backend.

Toda ação sensível precisa ser validada no backend.

Exemplos:

- candidato só remove o próprio interesse;
- candidato só altera a própria experiência;
- recrutador só altera a própria oportunidade;
- recrutador só publica como empresa se tiver vínculo aprovado;
- admin precisa ter permissão admin;
- usuário não pode se autoindicar;
- tipo não aprovado não deve ser usado em oportunidade;
- recurso público precisa ter Localidade validada.

A regra é:

Frontend melhora UX. Backend garante segurança.

21.7 Antes de mexer em Localidade, leia a documentação do módulo

Localidade é o módulo mais sensível da API.

Ele não é apenas cadastro de cidade, estado e país. Ele define se empresas e oportunidades podem ou não aparecer publicamente.

Antes de alterar qualquer coisa nesse módulo, entenda bem estas regras:

- Localidade validada é a fonte oficial;
- LocalidadePendente é fila técnica temporária;
- recursos com LocalidadePendente não aparecem publicamente;
- endpoints `/me` podem mostrar pendência ao dono;
- Google Maps é fallback encapsulado;
- ambiguidade não deve ser resolvida automaticamente;
- reprocessamento não deve criar pendências duplicadas;
- Empresa e Oportunidade não devem voltar a ter FK direta para LocalidadePendente.

A regra mais importante é:

LocalidadePendente não é estado público normal do produto.

Se essa regra for quebrada, a API pode voltar a exibir empresas ou vagas com localização incerta.

21.8 Não exponha LocalidadePendente em endpoints públicos

DTOs públicos de Empresa e Oportunidade não devem expor dados de LocalidadePendente.

Se um recurso está pendente, ele não deve aparecer no público.

Não tente resolver isso no frontend escondendo registros. A consulta pública deve sair correta do backend.

A regra é:

O backend deve garantir que endpoint público só retorna recurso com Localidade validada.

Isso precisa estar nos services, repositories ou specifications correspondentes.

21.9 Preserve Google Maps como detalhe interno

Google Maps é usado como fallback no fluxo de resolução de localidade. Ele não deve virar dependência de controller, listagem pública ou frontend.

Evite criar fluxo em que:

- controller chama Google Maps diretamente;
- frontend chama Google Maps para decidir o que o backend deve publicar;
- listagem pública depende de chamada externa;
- LocalidadeService deixa de ser catálogo oficial e vira integração externa.

A regra é:

Google Maps ajuda a resolver localidade, mas não é contrato público da API.

21.10 Não escolha localidade ambígua automaticamente

Se o sistema encontra mais de uma localidade possível, não deve escolher a primeira por conveniência.

Isso pode publicar empresa ou vaga na cidade errada.

A decisão do MVP foi manter pendente quando houver ambiguidade.

No futuro, essa pendência pode ser resolvida com confirmação manual do usuário, mas esse fluxo ainda precisa ser implementado com cuidado.

A regra é:

Melhor manter pendente do que validar errado.

21.11 Antes de mexer em Empresa, entenda vínculo e ownership

Empresa evoluiu de texto para entidade real, mas ainda não possui ownership formal completo.

Hoje existe vínculo entre recrutador e empresa, com aprovação administrativa. Isso não significa que o recrutador aprovado é dono da empresa.

A regra atual é:

Vínculo aprovado permite representar a empresa em alguns fluxos, mas não equivale a ownership formal.

Antes de permitir que recrutadores editem dados da empresa, aprovem outros recrutadores ou administrem a empresa, defina primeiro o modelo de ownership.

Possíveis decisões futuras incluem:

- dono da empresa;
- administradores da empresa;
- conta empresarial;
- login por CNPJ;
- convite de recrutadores;

- permissões por empresa.

Não implemente isso como simples campo improvisado.

21.12 Preserve a regra de publicação como empresa

O recrutador só pode publicar oportunidade em nome de empresa se tiver vínculo aprovado.

Não basta receber `empresaId` ou `publicarComoEmpresa=true`.

O backend precisa validar se o recrutador autenticado pode representar aquela empresa.

A regra é:

Publicar como empresa é permissão de negócio, não apenas campo de request.

Essa regra protege a credibilidade da plataforma.

21.13 Antes de mexer em Oportunidade, preserve ownership

Oportunidade nasce no contexto do recrutador autenticado.

O cliente não deve controlar `recrutadorId`.

Ao criar, editar, excluir ou listar oportunidades internas, o backend deve respeitar o recrutador autenticado.

A regra é:

Toda oportunidade tem recrutador responsável, derivado do contexto autenticado.

Também preserve:

- tipo de emprego aprovado;
- modalidade válida;

- localidade obrigatória;
- visibilidade pública apenas com Localidade validada;
- separação entre DTO público e DTO interno;
- publicação como empresa dependente de vínculo aprovado.

21.14 Preserve diferença entre oportunidade pública e interna

Uma oportunidade pode existir internamente e não aparecer publicamente.

Isso acontece principalmente por LocalidadePendente.

Não trate isso como inconsistência.

A regra é:

Existir no banco não significa estar disponível publicamente.

Essa regra vale para Empresa também.

Endpoints públicos devem retornar apenas recursos publicáveis. Endpoints internos `/me` podem retornar recursos próprios em processamento.

21.15 Antes de mexer em TipoDeEmprego, preserve a curadoria

TipoDeEmprego funciona como catálogo moderado.

Não misture tipos aprovados e não aprovados no catálogo público.

A regra é:

Público consome apenas tipos aprovados.

Também preserve:

- sugestão de usuário nasce como não aprovada;
- admin modera tipos pendentes;
- oportunidade só usa tipo aprovado;
- exclusão deve respeitar recurso em uso;
- aprovação em lote recebe array simples de IDs.

Se no futuro for necessário melhorar curadoria, trate como evolução explícita, por exemplo com recusa, motivo, merge de duplicados ou desativação.

21.16 Antes de mexer em paginação e filtros

Paginação e filtros não são apenas detalhe técnico. Eles preservam performance, integração com frontend e regras de visibilidade.

Ao criar listagem nova, avalie se ela pode crescer. Se puder, use o padrão:

- FilterDTO;
- Pageable;
- @PageableDefault;
- @ParameterObject;
- Page<ResponseDTO>;
- JpaSpecificationExecutor;
- Specification.

Não pague em memória quando o banco pode pagar.

Não aplique regra de visibilidade depois da paginação.

A regra é:

Filtre o conjunto correto no banco antes de pagar.

Isso é especialmente importante para excluir recursos com LocalidadePendente em endpoints públicos.

21.17 Não deixe query param controlar escopo sensível

Alguns escopos devem vir do path ou do token, não de query param.

Exemplos:

- em `/api/empresas/{id}/oportunidades`, o `empresaId` vem do path;
- em `/api/recrutadores/me/oportunidades`, o recrutador vem do token;

- em `/api/candidatos/me/interesses/vagas`, o candidato vem do token;
- em `/api/usuarios/me`, o usuário vem do token.

Não permita que um filtro opcional sobrescreva esse escopo.

A regra é:

Query param filtra dentro do escopo. Ele não define o escopo sensível.

21.18 Preserve tratamento global de erros

Não volte a lançar `RuntimeException` genérica para regra de negócio esperada.

Use exceptions específicas e deixe o handler global transformar isso em resposta HTTP padronizada.

A regra é:

Erro também é contrato da API.

Ao criar nova regra, pense em qual status faz sentido:

- 400 para entrada inválida;
- 401 para não autenticado;
- 403 para sem permissão;
- 404 para recurso inexistente ou indisponível;
- 409 para conflito;
- 422 para regra de negócio.

Também preserve mensagens em português correto, com acentuação real e sem mojibake.

21.19 Atualize Swagger quando mudar contrato

Swagger/OpenAPI é a referência formal para o frontend.

Se você alterar endpoint, DTO, filtro, status ou autenticação, atualize a documentação.

Não deixe Swagger desatualizado.

A regra é:

Contrato mudou, Swagger muda.

Também revise testes de OpenAPI quando aplicável.

21.20 Atualize README, docs e Postman quando necessário

A documentação faz parte da entrega do projeto.

Se mudar forma de executar, atualize README.

Se mudar teste ou estrutura da suíte, atualize `docs/testing.md`.

Se mudar endpoint importante, atualize coleção Postman.

Se mudar decisão arquitetural sensível, atualize JavaDoc ou documentação histórica.

A regra é:

Código e documentação precisam evoluir juntos.

Esse cuidado evita voltar ao problema inicial de documentação desatualizada.

21.21 Preserve testes ao alterar regra crítica

A suíte de testes foi ampliada para proteger fluxos importantes.

Ao alterar regra crítica, atualize ou adicione testes.

Isso vale especialmente para:

- Localidade;
- LocalidadePendente;

- Empresa;
- Oportunidade;
- endpoints `/me`;
- endpoints admin;
- paginação;
- filtros;
- segurança;
- tratamento de erro.

A regra é:

Mudou comportamento importante, teste precisa acompanhar.

Não confie apenas em teste manual via Postman.

21.22 Cuidado com encoding

Durante a documentação, houve esforço para corrigir acentuação e evitar mojibake.

Mantenha arquivos em UTF-8.

Não troque acentos por Unicode escapes em JavaDoc ou Markdown.

Evite textos como:

- `NÃ£o`;
- `descriÃ§Ã£o`;
- `invÃ;lido`;
- `pend\u00eancia`.

A regra é:

Texto humano deve estar legível com acentuação real.

Isso vale para mensagens de erro, Swagger, JavaDoc, README e documentação em Markdown.

21.23 Cuidado com escopo de MVP

Nem tudo que não existe hoje é bug.

Alguns pontos ficaram conscientemente para pós-MVP:

- confirmação manual de localidade;
- notificação de recurso oculto por LocalidadePendente;
- ownership formal de Empresa;
- conta empresarial ou login por CNPJ;
- contrato próprio de paginação;
- revisão de DTOs internos de localidade;
- revogação de token;
- painel operacional de pendências;
- preferências completas do candidato.

Antes de implementar qualquer um desses pontos, revise a documentação e trate como evolução planejada.

A regra é:

Não confunda decisão de escopo com esquecimento.

21.24 Ao implementar pós-MVP, não quebre MVP

Quando uma funcionalidade pós-MVP for implementada, ela deve evoluir o desenho atual, não desfazê-lo sem necessidade.

Por exemplo:

- confirmação manual de localidade deve trabalhar com LocalidadePendente, não contornar a fila;
- notificação deve comunicar pendência, não transformar pendência em erro fatal;
- ownership de Empresa deve complementar vínculo, não assumir que vínculo já é ownership;
- contrato próprio de paginação deve ser alinhado com frontend;

- revogação de token deve ser compatível com a estratégia JWT existente.

A regra é:

Evoluir não é apagar a decisão anterior; é entender por que ela existia e melhorar a partir dela.

21.25 Ordem recomendada para entender o projeto

Para um novo desenvolvedor backend, a ordem recomendada de estudo é:

1. README, para entender execução e visão geral;
2. Swagger, para entender contratos;
3. esta documentação, para entender decisões;
4. módulo de segurança, para entender autenticação e `/me`;
5. módulo de Localidade, para entender a regra mais sensível;
6. Empresa e Recrutador;
7. Oportunidade;
8. Candidato, experiências, interesses e indicações;
9. TipoDeEmprego;
10. testes;
11. Docker e configuração.

Essa ordem evita começar por módulos isolados sem entender as regras centrais.

21.26 Checklist antes de abrir pull request ou commit relevante

Antes de finalizar uma mudança, verifique:

- a regra de negócio ficou no service?
- o controller continua fino?
- foi criado ou atualizado DTO quando necessário?
- a mudança respeita `/me`, admin ou público?
- ownership foi validado?
- LocalidadePendente continua oculta do público?
- filtros estão na Specification?

- paginação não foi feita em memória?
- exceptions estão padronizadas?
- Swagger foi atualizado?
- testes foram atualizados?
- README/docs/Postman precisam de ajuste?
- não há mojibake ou Unicode escape em texto humano?
- a mudança não promete funcionalidade futura que ainda não existe?

Esse checklist ajuda a manter o padrão atual do projeto.

21.27 Síntese

O próximo desenvolvedor backend deve entender que a API atual é resultado de uma refatoração orientada por segurança, clareza de domínio, visibilidade pública controlada e integração com frontend.

As regras mais importantes são:

- identidade vem do token em fluxos pessoais;
- endpoints públicos só mostram recursos publicamente válidos;
- Localidade validada controla visibilidade pública;
- LocalidadePendente é fila técnica temporária;
- Empresa é entidade, mas ainda sem ownership formal;
- Oportunidade pertence ao recrutador autenticado;
- publicação como empresa exige vínculo aprovado;
- TipoDeEmprego é catálogo moderado;
- DTO protege contrato HTTP;
- service protege regra de negócio;
- documentação e testes devem acompanhar o código.

Se essas regras forem preservadas, a API pode continuar evoluindo sem perder a coerência conquistada no MVP.

22. Guia para o desenvolvedor frontend

Esta seção foi escrita para a pessoa responsável por desenvolver ou dar continuidade ao frontend da aplicação **Quero Trabalhar**. O objetivo é explicar como consumir a API de forma coerente com as decisões tomadas no backend, evitando interpretações erradas sobre autenticação, paginação, visibilidade pública, localidade pendente, permissões e contratos.

A API foi organizada para separar claramente três contextos de uso: público, usuário autenticado e administração. Essa separação precisa aparecer também no frontend. Muitas regras do backend foram criadas justamente para proteger dados, evitar acesso indevido e impedir que recursos incompletos apareçam publicamente antes da validação.

A regra principal para o frontend é:

Escolha o endpoint conforme o contexto da tela: público, `/me` ou admin.

Não escolha apenas pelo nome da entidade.

22.1 Entenda os três contextos da API

A API possui três grupos principais de endpoints.

O primeiro grupo é o dos endpoints públicos. Eles não exigem autenticação e são usados por telas abertas, como listagem pública de oportunidades, listagem pública de empresas, detalhe de vaga, detalhe de empresa, catálogo de tipos de emprego aprovados e catálogo de localidades.

O segundo grupo é o dos endpoints `/me`. Eles exigem autenticação e operam sobre o usuário logado. São usados em telas pessoais, como perfil do usuário, área do candidato, área do recrutador, experiências, interesses em vagas e oportunidades criadas pelo recrutador.

O terceiro grupo é o dos endpoints administrativos. Eles ficam em `/api/admin/**`, exigem autenticação e perfil administrativo. São usados para gestão de usuários, aprovação de vínculos, moderação de tipos de emprego e outras operações globais.

Essa divisão é essencial para a integração.

Público mostra dados prontos para qualquer pessoa.

`/me` mostra dados do usuário autenticado.

Admin gerencia dados globais e moderação.

22.2 Não use ID de usuário em fluxos pessoais

Sempre que a API oferecer um endpoint `/me`, o frontend deve usá-lo.

Isso significa que, em ações pessoais, o frontend não deve pedir, guardar ou enviar manualmente o `usuarioId` para o backend. O token JWT já identifica o usuário.

Essa regra vale para fluxos como:

- consultar dados do usuário logado;
- atualizar dados do usuário logado;
- alterar senha;
- gerenciar perfil de candidato;
- gerenciar perfil de recrutador;
- listar interesses do candidato;
- demonstrar interesse em uma vaga;
- remover interesse em uma vaga;
- listar oportunidades do recrutador;
- gerenciar experiências profissionais;
- listar indicações dadas e recebidas.

O motivo dessa decisão é segurança. Usar `/me` reduz risco de um usuário tentar manipular dados de outro apenas alterando um ID na URL.

A regra para o frontend é:

Se a tela é do usuário logado, use `/me`.

22.3 Login e armazenamento do token

O login é feito em `POST /login`.

Quando o login dá certo, a API retorna o JWT no header `Authorization`, no formato Bearer Token. O token não vem no corpo da resposta.

O frontend deve capturar esse header, extrair o token e armazená-lo conforme a estratégia da aplicação.

Depois disso, todas as requisições protegidas devem enviar:

`Authorization: Bearer <token>`

A coleção Postman foi configurada seguindo essa lógica, capturando automaticamente o token após o login e salvando em uma variável de ambiente. No frontend, a mesma ideia deve ser implementada no cliente HTTP.

A regra é:

Login gera token. Endpoints protegidos usam esse token no header Authorization.

22.4 Trate 401 e 403 de formas diferentes

O frontend precisa diferenciar autenticação de autorização.

`401 Unauthorized` significa que o usuário não está autenticado corretamente. Pode ser token ausente, token inválido, token expirado ou credenciais inválidas no login.

Nesse caso, o comportamento esperado costuma ser limpar a sessão local, redirecionar para login ou pedir nova autenticação.

`403 Forbidden` significa que o usuário está autenticado, mas não tem permissão para aquela ação.

Nesse caso, o comportamento esperado é exibir acesso negado, ocultar a funcionalidade ou impedir a ação, sem necessariamente deslogar o usuário.

A regra é:

401 pede autenticação. 403 indica falta de permissão.

22.5 Use endpoints públicos apenas para telas públicas

Endpoints públicos devem alimentar telas abertas da aplicação.

Eles não exigem token e retornam apenas recursos prontos para exposição pública.

Exemplos de telas públicas:

- listagem de oportunidades;
- detalhe de oportunidade;
- listagem de empresas;
- detalhe de empresa;
- oportunidades de uma empresa;
- recrutadores públicos de uma empresa;
- catálogo de tipos de emprego aprovados;
- seleção de país, estado e cidade.

A regra mais importante é:

Endpoints públicos de empresa e oportunidade só retornam recursos com Localidade validada.

Se uma vaga ou empresa não aparece no público, isso não significa automaticamente que ela não existe. Ela pode estar oculta porque a localidade está pendente.

22.6 Use endpoints **/me** para painéis internos

Endpoints **/me** devem alimentar telas internas do usuário logado.

Eles podem retornar informações que não aparecem publicamente. Isso é importante principalmente para recursos com localidade pendente.

Por exemplo, uma vaga criada por um recrutador pode aparecer no painel interno dele, mas não aparecer na listagem pública de vagas enquanto a localidade estiver em validação.

Isso é comportamento esperado.

A regra é:

O público vê apenas o que está pronto. O dono vê também o que está em processamento.

O frontend deve comunicar esse estado em telas internas, especialmente no painel do recrutador.

22.7 Use endpoints admin apenas em telas administrativas

Endpoints `/api/admin/**` devem ser usados apenas em telas administrativas.

Essas telas exigem usuário autenticado com permissão de admin.

Exemplos de telas admin:

- gestão de usuários;
- gestão global de experiências;
- moderação de tipos de emprego;
- aprovação individual ou em lote de tipos de emprego;
- aprovação ou recusa de vínculo entre recrutador e empresa.

Mesmo que o frontend esconda telas admin para usuários comuns, o backend continuará validando permissão. Portanto, o frontend deve tratar 403 corretamente.

A regra é:

Tela admin não é tela comum com mais botões. É um contexto separado.

22.8 Entenda LocalidadePendente

`LocalidadePendente` é um dos pontos mais importantes para o frontend.

Ela não significa erro fatal. Ela significa que o recurso foi salvo, mas a localização ainda não foi validada com segurança.

Isso pode acontecer quando a localidade informada é ambígua, incompleta, não encontrada ou depende de reprocessamento.

Em endpoints públicos, recursos pendentes não aparecem.

Em endpoints internos, o dono pode ver o recurso e seu status.

A mensagem ideal para o usuário interno deve seguir esta ideia:

“Este recurso foi salvo, mas a localidade ainda está em validação.
Enquanto isso, ele pode não aparecer nas listagens públicas.”

O frontend não deve prometer confirmação manual de localidade, porque esse fluxo ainda não está implementado no MVP.

22.9 Não trate ausência no público como erro automático

Um caso comum será o recrutador criar uma oportunidade e depois não vê-la na listagem pública.

Isso pode ser normal.

Se a vaga ficou com localidade pendente, ela existe internamente, mas ainda não está pública.

O fluxo correto para o frontend é:

- mostrar a vaga no painel do recrutador;
- exibir status de localidade, se o DTO trouxer;
- informar que a vaga pode ainda não aparecer publicamente;
- não tratar automaticamente como erro de cadastro.

A mesma lógica vale para empresa.

A regra é:

Se existe em `/me`, mas não no público, verifique `LocalidadePendente` antes de assumir bug.

22.10 Diferencie DTO público e DTO interno

A API pode retornar versões diferentes do mesmo recurso dependendo do endpoint.

DTO público é usado em telas públicas. Ele não deve expor detalhes técnicos internos.

DTO interno é usado em `/me` ou `admin`. Ele pode trazer dados adicionais para apoiar o usuário ou a gestão.

Exemplos:

- oportunidade pública não mostra detalhes de `LocalidadePendente`;
- oportunidade do recrutador pode mostrar status de localidade;
- empresa pública não mostra motivo de pendência;
- empresa interna pode mostrar status da pendência.

A regra é:

Não espere os mesmos campos em endpoint público e endpoint interno.

Sempre consulte o Swagger para saber o contrato exato.

22.11 Consuma paginação corretamente

Endpoints paginados não retornam um array diretamente.

Eles retornam um objeto de página. A lista de itens fica dentro de `content`.

Além de `content`, a resposta traz metadados, como página atual, tamanho da página, total de elementos, total de páginas e informações de ordenação.

O frontend deve usar esses metadados para montar paginação.

Atenção: no padrão usado pelo backend, `page` começa em zero. Então a primeira página enviada para a API é `page=0`.

A regra é:

Em endpoint paginado, leia os dados em `content` e use os metadados para navegação.

22.12 Saiba quais telas usam Page e quais usam List

Nem todos os endpoints retornam `Page`.

Listagens principais e com maior potencial de crescimento foram paginadas, como:

- oportunidades públicas;
- empresas públicas;
- oportunidades de uma empresa;
- oportunidades do recrutador;
- vagas de interesse do candidato;
- usuários admin;
- experiências admin;
- tipos de emprego não aprovados.

Alguns endpoints continuam retornando `List`, como catálogos, autocompletes ou listas pessoais menores.

Isso é esperado no MVP.

A regra para o frontend é:

Consulte o Swagger e trate cada endpoint pelo contrato real. Não assuma que toda lista é `Page` nem que toda lista é array.

22.13 Envie filtros como query params

Filtros devem ser enviados como query params.

Cada endpoint tem filtros próprios.

Exemplos comuns:

- `termo`;
- `tipoDeEmpregoId`;
- `empresaId`;
- `recrutadorId`;

- `paisId`;
- `estadoId`;
- `cidadeId`;
- `modalidade`;
- `statusLocalidade`, em telas internas;
- filtros de datas, em telas admin;
- filtros de perfil, em telas admin de usuários.

A regra é:

Não carregue tudo para filtrar no frontend quando o backend já oferece filtros.

Isso melhora performance e mantém a busca consistente.

22.14 Use localidade estruturada sempre que possível

O frontend deve priorizar IDs estruturados de localidade.

O ideal é usar os endpoints de catálogo para carregar:

- países;
- estados de um país;
- cidades de um estado.

Depois, em filtros e formulários, enviar os IDs correspondentes.

Texto livre pode existir em alguns fluxos, mas aumenta a chance de a API criar uma pendência se não conseguir resolver a localidade com segurança.

A regra recomendada é:

Sempre que houver catálogo disponível, use `paisId`, `estadoId` e `cidadeId`.

Isso reduz pendências e melhora a qualidade da experiência.

22.15 Entenda o fluxo de candidato

Na área do candidato, o frontend deve trabalhar com o usuário autenticado.

Principais ações:

- visualizar dados do usuário;
- gerenciar perfil candidato;
- gerenciar experiências profissionais;
- demonstrar interesse em vaga;
- remover interesse;
- listar vagas de interesse;
- listar indicações dadas e recebidas;
- criar indicação, quando aplicável.

A regra principal é:

Candidato não envia o próprio ID. O token define quem ele é.

Erros comuns nesse fluxo podem envolver perfil candidato ausente, interesse duplicado, interesse inexistente ou autoindicação.

22.16 Entenda o fluxo de recrutador

Na área do recrutador, o frontend também deve trabalhar com o usuário autenticado.

Principais ações:

- visualizar perfil recrutador;
- criar ou atualizar perfil recrutador;
- solicitar vínculo com empresa;
- consultar empresa vinculada;
- listar próprias oportunidades;
- criar oportunidade;
- editar oportunidade;
- excluir oportunidade;
- acompanhar status de localidade das vagas;

- publicar como empresa quando houver vínculo aprovado.

A regra principal é:

Recrutador não envia `recrutadorId` para operar suas próprias oportunidades.

O backend identifica o recrutador pelo token.

22.17 Entenda publicação como empresa

`publicarComoEmpresa` não é apenas um campo visual.

Ele depende de regra de negócio.

O recrutador só pode publicar em nome de empresa se tiver vínculo aprovado com essa empresa.

Se não houver vínculo aprovado, o backend deve impedir a ação.

O frontend pode esconder ou desabilitar a opção quando o vínculo não existir, mas não deve depender apenas disso.

A regra é:

Publicar como empresa depende de vínculo aprovado, validado pelo backend.

22.18 Entenda o fluxo de empresa

No frontend, empresa pode aparecer em dois contextos.

No contexto público, empresa representa uma organização visível para qualquer pessoa. Ela precisa ter Localidade validada.

No contexto interno, uma empresa pode existir em estado ainda não público, por exemplo com localidade pendente ou vínculo em análise.

Também existe o fluxo de vínculo entre recrutador e empresa:

- recrutador solicita vínculo;
- admin aprova ou recusa;
- vínculo aprovado permite representação pública;
- vínculo aprovado pode permitir publicação como empresa.

A regra é:

Solicitar vínculo não significa vínculo aprovado.

22.19 Entenda TipoDeEmprego no frontend

Telas públicas e formulários comuns devem usar o catálogo de tipos aprovados.

Tipos não aprovados pertencem ao fluxo admin.

O usuário pode sugerir um novo tipo, mas a sugestão não entra automaticamente no catálogo público.

A regra é:

Tipo sugerido precisa de curadoria antes de virar opção pública.

Para criação de oportunidade e experiência profissional, o frontend deve preferir tipos aprovados.

22.20 Trate erros de validação e regra de negócio

O frontend deve diferenciar erro de validação de erro de regra de negócio.

Erro de validação normalmente vem com campos inválidos. Ele deve ser exibido junto ao formulário.

Erro de regra de negócio normalmente vem com mensagem geral. Ele deve ser exibido como feedback da ação, por exemplo em toast ou alerta.

Exemplos de regra de negócio:

- interesse duplicado;
- autoindicação;

- publicar como empresa sem vínculo aprovado;
- usar tipo de emprego não aprovado;
- tentar alterar recurso que não pertence ao usuário.

A regra é:

Use erros de campo para formulário e mensagem geral para regra de negócio.

22.21 Use Swagger como fonte de contrato

Esta documentação explica contexto e decisões, mas o Swagger deve ser a fonte principal para detalhes de contrato.

Antes de implementar uma tela, consulte o Swagger para confirmar:

- método HTTP;
- URL;
- parâmetros;
- body;
- response;
- status possíveis;
- se exige autenticação;
- se retorna Page ou List;
- campos exatos dos DTOs.

A regra é:

Esta documentação explica o porquê. O Swagger confirma o contrato.

22.22 Use Postman para validar fluxos antes de implementar telas

A coleção Postman foi criada para ajudar nos testes manuais.

Ela já organiza endpoints por módulo, possui ambiente local e captura token após login.

Antes de implementar uma tela mais complexa, vale testar o fluxo no Postman.

Isso ajuda a diferenciar:

- erro de contrato;
- erro de autenticação;
- erro de permissão;
- erro de regra de negócio;
- erro de implementação da tela.

A regra é:

Se um fluxo não funciona no Postman, provavelmente também não vai funcionar no frontend.

22.23 Não implemente no frontend funcionalidades que o MVP ainda não tem

Alguns fluxos foram deixados para pós-MVP.

O frontend não deve prometer essas funcionalidades como se já existissem.

Exemplos:

- confirmação manual de localidade sugerida;
- notificação automática de recurso oculto por localidade pendente;
- ownership formal de empresa;
- conta empresarial;
- login por CNPJ;
- painel operacional completo de LocalidadePendente;
- revogação avançada de token.

Esses pontos podem aparecer no roadmap, mas não devem ser tratados como recurso atual.

A regra é:

Tela não deve vender funcionalidade que a API ainda não entrega.

22.24 Cuidados de UX alinhados ao backend

Algumas decisões de backend precisam ser traduzidas em UX.

Quando uma localidade ficar pendente, a tela interna deve explicar que o recurso foi salvo, mas pode não aparecer publicamente ainda.

Quando um usuário não tiver permissão, a tela deve indicar acesso negado, não erro genérico.

Quando um token expirar, a tela deve pedir login novamente.

Quando o usuário tentar ação duplicada, como demonstrar interesse duas vezes, a tela deve mostrar a mensagem da API.

Quando uma sugestão de tipo de emprego for enviada, a tela deve indicar que ela depende de aprovação.

A regra é:

Boa UX não ignora regra de backend; ela explica a regra para o usuário.

22.25 Checklist para implementar uma tela

Antes de implementar uma tela, a pessoa do frontend deve responder:

- a tela é pública, /me ou admin?
- o endpoint exige token?
- o usuário precisa de perfil específico?
- o retorno é Page ou List?
- quais filtros existem?
- a tela precisa lidar com LocalidadePendente?
- o recurso pode existir internamente e não aparecer publicamente?
- quais erros precisam ser tratados?
- o Swagger confirma o contrato?
- existe request equivalente no Postman?
- a funcionalidade já existe no MVP ou é roadmap?

Esse checklist evita boa parte dos erros de integração.

22.26 Síntese

O frontend deve consumir a API respeitando os contextos definidos pelo backend.

As regras mais importantes são:

- público só mostra recursos publicamente válidos;
- `/me` usa o usuário autenticado;
- admin exige permissão administrativa;
- token vem no header `Authorization`;
- endpoints paginados usam `content`;
- localidade pendente não é erro fatal;
- DTO público e DTO interno podem ser diferentes;
- filtros devem ser enviados ao backend;
- localidade estruturada deve ser preferida;
- funcionalidades pós-MVP não devem ser prometidas como prontas.

Se essas regras forem respeitadas, a integração tende a ficar segura, previsível e alinhada com as decisões do backend.